



National University of Computer and Emerging Sciences, Lahore



Cassandra: All You Need in a Class Companion

Mannan Ul Haq	22L-7556	BS(DS)
Huzaifa Bin Amir	22L-7518	BS(DS)
Muhammad Abdullah	22L-7522	BS(DS)

Supervisor: Ms. Umme Ammarah

Final Year Project

May 12, 2026

Anti-Plagiarism Declaration

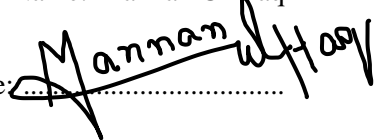
This is to declare that the above publication was produced under the:

Title: Cassandra: All You Need in a Class Companion

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: March 25, 2025

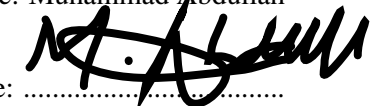
Name: Mannan Ul Haq

Signature: 

Name: Huzaifa Bin Amir

Signature: 

Name: Muhammad Abdullah

Signature: 

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

Cassandra is an intelligent, web based educational platform designed to resolve the critical student challenge of divided attention during classroom lectures, where the necessity of manual note taking compromises the full absorption of complex content. The multimodal pipeline processes lectures using Faster-Whisper and Resemblyzer for audio transcription and analysis. It uses BLIP-2 for captioning the media images extracted from slides and a custom table-equation-classifier to identify specific visual elements, while DeepSeek LLM Model organizes the reading flow. Whiteboard data is structured into text and LaTeX using Qwen3-VL-4B-Instruct. Finally, SentenceTransformers and DeepSeek-V3 Large Language Model integrations combine everything into structured academic notes. The RAG reinforces all raw content with the addition of authoritative details from uploaded course materials and textbooks. The completed system replaces the incomplete, fragmented output of manual note-taking with a fully cohesive, multi-source, contextually rich study resource. This aggregate data is compiled into one coherent, logically organized study document by the DeepSeek-V3 Large Language Model. Cassandra packages this core capability together, along with a full Learning Management System and dual AI Chatbot system, to make the dynamic lecture a constant, accessible, high-quality learning resource.

Executive Summary

The Cassandra: All You Need in a Class Companion project delivers a groundbreaking intelligent, web based platform specifically created to solve the most significant functional gap in modern digital education: the complete and effective capture of the live, multi sensory classroom lecture. Whereas current Learning Management Systems (LMS) manage administrative functions such as course registration and assignment tracking well, they are not equipped to capture dynamic, real time content produced during lectures. This requires that the students have to be continually distracted between listening to complicated lectures and paying attention to their note-taking, resulting in inconsistent and incomplete notes. Cassandra offers a one-stop solution, as it automatically transcribes the spoken lecture, the visual information present on the slides, as well as the written information presented on the whiteboard, compiling all these different components into a cohesive and structured set of notes which are then presented once the class has been concluded.

The technology behind the Cassandra system is its Automated Note Generation pipeline, which is a high-level, multi-stage approach to generating notes that has the capability to adapt to the teaching approach being adopted, whether the teacher is conducting his/her lecture using slides, a classroom board, or through an audio only format. This process begins with the Multimodal Data Acquisition stage, whereby the live audio feed, presentation content, whiteboard content, and other external information from pre-uploaded course material and textbooks are all acquired simultaneously. In order to ensure the authenticity and quality of the resulting transcript, the system initially enrolls a unique Teacher Voice Signature (utilizing the Resemblyzer tool) in order to separate the instructor's voice, actively eliminating all student interruptions and background noises. The pure audio is then transcribed into text using a customized Faster Whisper model for code-switched speech-embedded Urdu and English words, making the end text exact and contextually correct.

Simultaneously, special AI techniques, such as Google ViT 16-Patch 224 and Qwen3-VL-4B-Instruct for extremely accurate printed and handwritten text identification, pull out all the visual components of the lecture, inclusive of but not limited to text, tables, mathematical expressions, and in-slide-embedded images, and handwritten material comprising text, tables, mathematical expressions from the whiteboard. Furthermore, the obtained content is accurately synchronized with its respective segment of the transcribed audio so that full contextualization of what was being said and what was being displayed is achieved. The integrated content has been further enhanced and validated by applying a Retrieval Augmented Generation system, which intelligently selects and extracts any contextual explanations, definitions, and details right from the source i.e., the textbooks and other documents provided. This process enriches the factual accuracy and completeness of the generated content. The synthesis of this massive

content and its structuring is accomplished on the robust DeepSeek-chat Large Language Model. Ultimately, the result is a thoroughly curated, ready-to-study lecture document. Apart from its primary function as an automatic note-taking tool, Cassandra acts as a full educational platform. It has been integrated with a comprehensive Learning Management System (LMS) for managing all the logistical aspects related to classes. Teachers can create classes, register students, assign tasks, and also send out messages within the system. Furthermore, Cassandra provides a Dual AI Assistant Chatbot System. The chatbot designated for the Instructor allows the instructor to search through all the instructional materials uploaded by them for preparing lessons. On the other hand, the student chatbot can only answer queries from students based on reading the created lecture content only. Such an approach ensures factual accuracy and greatly promotes meaningful revision efforts. All components are developed following the decoupling and modularity principle with the aim of achieving Maintainability and Extensibility regarding future seamless incorporation of different AI models and types of inputs. However, above all else, all data belonging to users, their lecture videos, and notes are protected by RBAC and encryption measures in order to create a safe, reliable, and wise educational environment. The pinnacle of Cassandra, without a doubt, is the conversion of the ephemeral, multisource lecture into the durable learning material that can be permanently archived and verified. The provision of a consolidated and automatically processed document is precisely what allows this project to overcome the old conflict between student attention and manual note-taking. Thus, an achieved combination of all advanced AI technologies from multimodal processing to RAG within a holistic LMS makes this project meet all requirements as a true intelligent class companion.

Table of Contents

List of Figures	xiii
List of Tables	xv
1 Introduction	1
1.1 Purpose of this Document	1
1.2 Intended Audience	2
1.3 Definitions, Acronyms, and Abbreviations	2
1.4 Conclusion	2
2 Project Vision	4
2.1 Problem Domain Overview	4
2.2 Problem Statement	5
2.3 Problem Elaboration	5
2.4 Goals and Objectives	7
2.5 Project Scope	7
2.5.1 In Scope Features	8
2.5.2 Project Deliverables	9
2.5.3 Out of Scope	9
2.6 Sustainable Development Goal (SDG)	10
2.7 Constraints	10
2.8 Business Opportunity	12
2.9 Stakeholders Description/User Characteristics	12
2.9.1 Stakeholders Summary	12
2.9.2 Key High-Level Goals and Problems of Stakeholders	13
2.10 Conclusion	13
3 Literature Review	14
3.1 Definitions, Acronyms, and Abbreviations	14

3.2	Detailed Literature Review	14
3.2.1	The Landscape of Digital Learning Platforms	14
3.2.2	Studies on Automated Educational Content Generation	16
3.2.3	State of the Art in Core Enabling Technologies	17
3.3	Literature Review Summary Table	18
3.4	Conclusion	19
4	Software Requirement Specifications	21
4.1	List of Features	21
4.2	Functional Requirements	21
4.2.1	Teacher's Functional Requirements	21
4.2.2	Functional Requirements of Students	22
4.2.3	System Functional Requirements	22
4.3	Quality Attributes	23
4.3.1	Reliability	23
4.3.2	Maintainability	23
4.3.3	Correctness	23
4.3.4	Usability	23
4.3.5	Efficiency	24
4.4	Non Functional Requirements	24
4.4.1	Performance	24
4.4.2	Security	24
4.4.3	Extensibility	24
4.5	Assumptions	25
4.6	Use Cases	25
4.7	Hardware and Software Requirements	29
4.7.1	Hardware Requirements	29
4.7.2	Software Requirements	29
4.8	Graphical User Interface	30
4.8.1	Teacher Registration/Login Page	30
4.8.2	Teacher Dashboard Interface	31
4.8.3	Teacher's Course Management	31
4.8.4	Teacher's RAG Knowledge Base Management	31
4.8.5	Lecture Initiation for Teachers	32
4.8.6	Teacher's Live Lecture Interface	33

4.8.7	Student's Registration/Login Page	33
4.8.8	Student's Dashboard Interface	34
4.8.9	Student's Course Management	35
4.8.10	Student Notes Library	35
4.8.11	Student-Teacher Messaging Interface	36
4.9	Database Design	36
4.9.1	ER Diagram	36
4.9.2	Data Dictionary	38
4.10	Risk Analysis	41
4.10.1	Technical Risks	41
4.10.2	Content and Quality Risks	42
4.10.3	User Adoption and Engagement Risks	42
4.10.4	Competition Risks	42
4.11	Conclusion	42
5	Proposed Approach and Methodology	43
5.1	Multimodal Data Acquisition Framework	43
5.2	Audio Recording	43
5.2.1	Teacher Voice Signature Enrollment	43
5.2.2	Lecture Audio Recording	44
5.2.3	Lecture Audio Transcription	44
5.3	Slide Content Decomposition and Extraction Pipeline	45
5.3.1	Preliminary Document Conversion and Preprocessing	45
5.3.2	Complete PPTX Decomposition	45
5.3.3	Mathematical Equations Extraction	45
5.3.4	Table Extraction	46
5.4	Whiteboard Content Decomposition and Extraction Pipeline	46
5.4.1	Snapshot Acquisition	46
5.4.2	Image Preprocessing	46
5.4.3	Model Loading and Inference	47
5.4.4	Structured JSON Output Schema	47
5.5	AI-Driven Content Structuring and Semantic Integration	48
5.5.1	Logical Content Structuring	48
5.5.2	Semantic Audio-to-Slide Alignment	48
5.5.3	Semantic Audio-to-Whiteboard Alignment	49

5.5.4	Semantic Audio-to-Combined Slides and Whiteboard Alignment	49
5.6	Knowledge Augmentation and Final Note Generation	49
5.6.1	RAG-Based Content Explanation	49
5.6.2	Summarized Note Generation	50
5.7	Conclusion	50
6	High-Level and Low-Level Design	51
6.1	System Overview	51
6.2	Design Considerations	51
6.2.1	Assumptions and Dependencies	52
6.2.2	General Constraints	53
6.2.3	Goals and Guidelines	53
6.2.4	Development Methods	54
6.3	System Architecture	54
6.3.1	Subsystem Architecture	56
6.4	Architectural Strategies	62
6.4.1	Strategy 1: Pipeline and Modular Specialization	63
6.4.2	Strategy 2: Hardware Aware AI Deployment and Concurrency	63
6.4.3	Strategy 3: Semantic Enhancement and Dynamic Integration	63
6.4.4	Strategy 4: Reuse of Pre-trained Models and External APIs	63
6.5	Domain Model / Class Diagram	64
6.6	Policies and Tactics	65
6.6.1	Tools and Libraries	65
6.6.2	Coding Guidelines and Conventions	65
6.6.3	User Interface Design	65
6.6.4	Error Handling and Recovery	66
6.6.5	Model Management and Performance	66
6.6.6	System Testing Approach	66
6.6.7	Software Maintenance	66
6.7	Conclusion	66
7	Implementation and Test Cases	68
7.1	Implementation	68
7.1.1	Teacher Authentication and Voice Signature Enrollment	68
7.1.2	Construction of RAG Knowledge Base	69

7.1.3	Teacher's AI Chatbot and Retrieval-Augmented Generation (RAG)	70
7.1.4	PPTX Conversion Service	72
7.1.5	Slides Only Lecture Session Mode	72
7.1.6	Whiteboard Only Lecture Session Mode	73
7.1.7	Hybrid Slides and Whiteboard Lecture Session Mode	75
7.1.8	Audio Only Lecture Session Mode	76
7.1.9	Audio Preprocessing Pipeline	77
7.1.10	Visual Content Classification	78
7.1.11	Qwen3-VL-4B-Instruct Fine-Tuning for Slides Equations and Tables Extraction	80
7.1.12	Slides Content Decomposition and Structuring	81
7.1.13	Whiteboard Processing Pipeline	83
7.1.14	Visual Content and Audio Alignment	85
7.1.15	Retrieval-Augmented Explanation Generation	86
7.1.16	Final Summarization and Markdown Compilation	87
7.1.17	Final HTML Rendering and Notes Normalization	88
7.1.18	Interactive Student AI Chatbot and RAG Framework	89
7.1.19	Web Integration and Full-Stack Cloud Architecture	89
7.2	Test Case Design and Description	90
7.3	Test Metrics	99
7.4	Conclusion	99
8	User Manual	100
8.1	Getting Started: Registration and Authentication	100
8.1.1	Teacher Registration and Voice Enrollment	100
8.1.2	Student Registration	100
8.2	Teacher Dashboard & Operations	100
8.2.1	Class Creation & Knowledge Base Uploads	100
8.2.2	Lecture Recording Modes	101
8.2.3	Notes Management	101
8.2.4	Teacher AI Chatbot	101
8.3	Student Dashboard & Operations	102
8.3.1	Joining a Class	102
8.3.2	Accessing Generated Notes	102
8.3.3	Interactive AI Chatbot (Cassandra Tutor)	102
8.4	Direct Messaging & Announcements & Assignments	102

8.5	Conclusion	102
9	Experimental Results and Discussion	103
9.1	Audio Transcription and Translation Quality Assessment	103
9.2	Visual Content Classification Accuracy (ViT)	104
9.3	Qwen3-VL-4B-Instruct Fine-Tuning for Structured OCR	106
9.4	Retrieval Augmented Explanation Generation Assessment	107
9.5	Final Summarization and Markdown Compilation Quality	108
9.6	Conclusion	109
10	Conclusion	110

List of Figures

2.1	SDG 4	10
4.1	Teacher Registration Interface of the Cassandra System	30
4.2	Teacher Login Interface of the Cassandra System	30
4.3	Dashboard For Course and Profile Management	31
4.4	Course Management interface	31
4.5	Teacher Interface For Managing the RAG Knowledge Base	32
4.6	Lecture Session Initiation Interface with Delivery Mode Selection	32
4.7	Live Lecture Interface of Slides Only Mode	33
4.8	Student Registration Interface of the Cassandra System	33
4.9	Student Login Interface of the Cassandra System	34
4.10	Student Dashboard Showing Enrolled Courses and Course Enrollment Option	34
4.11	Student Course Interface With Announcements and AI Assistant	35
4.12	Student Notes Library With View and Download Options	35
4.13	Student-Teacher Messaging Interface For Direct Communication	36
4.14	ER Diagram for Cassandra Database	37
6.1	System Architecture of Cassandra	55
6.2	Voice Verification and Processing	57
6.3	Slides Decomposition steps	58
6.4	Whiteboard Text Extraction Pipeline	59
6.5	Content Refinement Step through AI Integration	60
6.6	Semantic Audio Alignment Process	61
6.7	Knowledge Augmentation Process	62
6.8	Class Diagram of Cassandra	64
7.1	Mandatory Voice Enrollment Interface for Speaker Isolation	69
7.2	Storing Vector Embeddings for the RAG System	70
7.3	AI Teacher Assistant Chat Interface	71

7.4	Lecture Recorder Interface in Slides-Only Mode	73
7.5	QR Code for Connecting External Mobile Camera	74
7.6	Camera Mobile Interface for Whiteboard Synchronization	74
7.7	Whiteboard-Only Mode displaying Captured Handwriting	75
7.8	Audio Only Lecture Session Mode Interface	76
7.9	Transcription Pipeline Showing Speaker Identification and Noise Filtering	77
7.10	Dataset Samples for Classifying Equations, Tables, and Images	79
7.11	Mathematical Equation Extraction and LaTeX Conversion	80
7.12	Slide Element Detection and Bounding Box Segmentation	81
7.13	JSON Output Structure representing Processed Slide Data	82
7.14	Handwritten Whiteboard Input Snapshot	83
7.15	Preprocessed Whiteboard Image	83
7.16	JSON Output Structure Representing Processed Whiteboard Data	85
7.17	Compiled Markdown Output after Final Summarization	87
7.18	Normalized Final Notes with HTML Conversion	88
9.1	Slide 1 Audio Transcription Analysis	103
9.2	Slide 2 Audio Transcription Analysis	104
9.3	Training and Validation Loss	105
9.4	Classification Report and Confusion Matrix	106
9.5	Training Loss Convergence over 2500 steps	107
9.6	Comparison of Base and Fine-tuned Model	107

List of Tables

1.1	Commonly Used Abbreviations and Their Definitions	2
3.1	Commonly Used Abbreviations and Their Definitions	14
3.2	Comparative Analysis of Commercial Educational Platforms	16
3.3	Summary of Literature Review	19
4.1	Teacher Initiates Lecture Session	25
4.2	System Generates Final Lecture Note	26
4.3	Student Queries AI Assistant on Notes	28
4.4	Database Entities and Attributes	38
7.1	Teacher Voice Enrollment Test	90
7.2	Student Registration and Enrollment	91
7.3	Teacher Secure Login Test	91
7.4	Student Secure Login Test	92
7.5	RAG Knowledge Base Construction	92
7.6	Teacher AI Chatbot Query	93
7.7	Slides-Only Mode Recording	93
7.8	Whiteboard-Only Mode Recording	94
7.9	Hybrid Slides and Whiteboard Mode	94
7.10	Audio-Only Mode Recording	95
7.11	Notes Markdown Compilation	95
7.12	Final HTML Rendering Normalization	96
7.13	Student AI Chatbot Query	96
7.14	Course Announcements System Test	97
7.15	Assignment Distribution Submission Test	97
7.16	Private Student-Teacher Messaging Test	98
7.17	Teacher Notes Edit Test	98
7.18	Test cases Matric	99

9.1	Audio Pipeline Performance Metrics Across Sample Slides	104
9.2	ViT Classifier Performance Evaluation	105
9.3	RAG Explanation Generation Performance Metrics	108
9.4	Markdown Compilation and Summarization Metrics	108

Chapter 1 Introduction

While Learning Management Systems (LMS) such as Moodle or Google Classroom are effective tools for disseminating course content and checking on homework, they normally lack a crucial component in this regard, which are the lectures that occur inside the classroom. During the classes, the students should be able to listen carefully to what the lecturer says, try to comprehend complex topics, and even take extensive notes all at once. As a consequence, their notes become messy and incomplete; thus, making them difficult to use as study material in the future. In response, we have designed a unique and innovative learning platform called Cassandra, which is capable of recording and organizing the lecture material systematically so that classes may become a source of valuable study material.

1.1 Purpose of this Document

The main emphasis of this FYP report is the complete development process of our innovative web platform entitled All You Need in a Class Companion: Cassandra. Cassandra is a new educational web portal that can automatically create well-organized lecture notes using different sources of information such as audio recording from the class, PPTX presentations, images captured from the whiteboards, and books.

In this respect, there exists only one crucial research question, whether an AI model can efficiently capture and transcribe real-time lecture content to create an exhaustive learning resource, thus reducing the requirement for making manual notes:

- Automatic transcription of speech to text in which the system would identify the speaker and handle multiple languages such as English and Urdu.
- Multimodal content extraction through slides and whiteboards to extract text, tables, equations, etc.
- Large language models for summarizing content.
- The RAG framework for knowledge enrichment in the transcribed content by incorporating relevant content from textbooks and other course resources to ensure contextual accuracy.

This report highlights the overall design of the Cassandra framework, the technology deployment, the testing process, and performance evaluation of the proposed system. The final part discusses the limitations of the platform and potential future modifications.

1.2 Intended Audience

Knowing who will read this report is essential to getting the right tone, level of detail, and overall presentation. Keeping the audience in mind allows us to ensure that the purpose of the project, technical effort, and assessment are described in appropriate terms that are clear and understandable.

This report is mainly intended for Ms. Umme Ammarah, the project advisor, and the faculty assessment board at the Department of Data Science, FAST-NUCES. That said, it is also prepared with the students and instructors in mind, especially those who are interested in how AI can be used to enhance the learning and teaching process.

The report for lecturers shows how Cassandra will ease the management of lectures and delivery of content, while for the students, it shows how the system will enhance learning by automatically converting the lectures into nicely organized notes.

1.3 Definitions, Acronyms, and Abbreviations

Familiarity with these terms will aid in understanding the technical descriptions of the Cassandra platform. This can be best understood by Table 1.1.

Table 1.1: Commonly Used Abbreviations and Their Definitions

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
FYP	Final Year Project
LLM	Large Language Model
LMS	Learning Management System
NLP	Natural Language Processing
OCR	Optical Character Recognition
PDF	Portable Document Format
PPTX	PowerPoint Presentation
RAG	Retrieval-Augmented Generation
ViT	Vision Transformer

1.4 Conclusion

The report is structured in such a way as to guide the reader through the entire process of the project from start to finish, from idea to implementation and assessment.

It initiates with Chapter 2: Project Vision, where the problem being solved is described and the scope of the project is clearly outlined. Chapter 3: Literature Review goes over the current research and

technology on automated note-taking and AI for education to lay the foundation for our strategy.

Then, Chapter 4: Software Requirement Specifications defines the functional and technical requirements of the system as a lead-in to Chapter 5: Proposed Approach and Methodology, where the main algorithms and workflow are fully elaborated. Chapter 6: High Level and Low Level Design then gives an overview of the structure of the system. The implementation and test cases are explained in Chapter 7: Implementation and Test Cases that shows how the system was implemented and tested and verified. In addition, Chapter 8: User Manual provides a user-friendly, step-by-step tutorial of using Cassandra in order to help users understand how to use the platform.

Chapter 9: Experimental Results and Discussion presents the results of the project, where system performance and accuracy of the notes generated are tested. Finally, Chapter 10: Conclusions summarizes the main conclusions, points out some of the existing limitations, and describes further work that can extend and improve Cassandra.

Chapter 2 Project Vision

This chapter presents the foundation needed for the “Cassandra” project by defining the general vision and scope of the project. It serves as a clear guideline that describes what problem we are trying to solve, what goals we want to accomplish, and what boundaries exist within which the project will operate. Given these building blocks, this chapter makes sure everyone is on the same page regarding what Cassandra is intended to achieve and how it will be developed from concept to finished product. First, there is a broad overview of the system’s domain to help the readers understand the context and intended role of the platform.

2.1 Problem Domain Overview

Cassandra is an intelligent, web-based educational platform that automatically generates comprehensive and well-structured lecture notes. The goal of the system is to convert the lively, fast-paced live lecture setting involving voice, presentations, and text written on the whiteboard into structured learning material. For this, Cassandra uses artificial intelligence in collecting and processing the data available from various resources.

The following are types of inputs processed by the system at once:

- **Live Audio Inputs:** Captures the voice of the teacher during a live lecture.
- **Whiteboard Inputs:** Captures text, diagrams, tables, formulas written on the whiteboard.
- **Presentation Inputs:** Extracts information from presentation slides such as PowerPoint (pptx).
- **Course Materials Knowledge:** Provides additional information for related course material.

The Cassandra end-to-end workflow consists of several stages that have been automated to generate meaningful lecture notes in a structured format:

1. **Voice Isolation:** It involves recording a sample of the instructor’s voice, which generates a vocal signature, allowing Cassandra to ignore background noises and student voices.
2. **Audio Transcription:** Transcribing the isolated audio lecture into a textual form. If the speech contains code switching (e.g., Urdu-English), it will convert it to English.
3. **Content Decomposition:** Extracting all elements of content contained within presentation slides and whiteboards.
4. **Intelligent Structuring:** Cassandra employs the LLM to organize the content in an orderly fashion with the inclusion of headings, paragraphs, and bullet points.

5. **Semantic Integration:** Aligning the transcription of the lecture with the corresponding slide content, thus linking the teacher's explanation to any visual information.
6. **RAG Integration:** Applying the textbook or course material to provide more elaborate information in note generation using RAG capability.
7. **Note Generation:** Lastly, the AI models generate lecture notes from the integrated information in a succinct yet comprehensive manner.

In addition to automatically creating lecture notes, Cassandra can act as a full-fledged LMS, offering options for creating courses, sending and receiving assignments, communicating, and issuing announcements. Moreover, there is a double AI Chatbot service intended to help teachers manage the uploading process and one dedicated to helping students understand the lecture through the answers to the questions related to the lecture notes generated.

In other words, the presence of such capabilities makes Cassandra an all-around educational tool, as it not only facilitates the note-taking process but also transforms the way teachers teach and students learn.

2.2 Problem Statement

The fundamental problem this research project is aiming to solve is students' inability to take comprehensive notes in class and, simultaneously, grasp lecture material during online and traditional classes. These two equally important processes often clash, drawing students' attention away from both and causing incompleteness or incoherence of the study materials prepared by them.

As for existing LMS like Moodle and Google Classroom, they have shown a remarkable capability to organize various processes associated with teaching, like sending announcements and assignments and uploading documents. Yet, none of these tools has managed to embrace a lecture as a complex process of interaction, incorporating multiple senses.

As such, a lot of context and valuable information about what is actually being taught by the teacher becomes lost. The problem of disconnect arises from here, leading to students having a partial picture of what has transpired during lectures.

2.3 Problem Elaboration

One of the biggest problems a student faces when attending a classic lecture is a high mental load. A student comes to attend a class where the information comes through three channels simultaneously: the teacher's speech, what is written on the board, and what is presented through slides. Thus, the

student has to listen, comprehend, and take notes all at once. Inevitably, this results in many instances of omissions in key information by students and subsequently leads to poorly taken notes which may not necessarily correspond to what had been taught.

In order to solve these problems, Cassandra tackles several tough technical challenges, each addressing a different aspect of the problem:

- **Multi Modal Data Synchronization:** One of the key problems associated with taking good notes involves aligning various types of data such as audio recordings, whiteboard scribbles during classes, and PowerPoint slides in one document.
- **Accurate Speech to Text Conversion:** The first thing that Cassandra does is transcribe the audio of the lecture into text. This requires a highly advanced speech recognition model that not only transcribes the audio but can handle audio recordings in two different languages such as Urdu and English, and then converts both of them into precise English text.
- **Speaker Identification:** One of the biggest challenges that a real classroom poses is noise in the environment. There are multiple speakers in the environment, so it becomes difficult to capture a clean version of the audio. Cassandra uses an existing model of speaker verification to filter out irrelevant voices from the audio by sampling the instructor's voice.
- **Complex Document Decomposition:** Presentation slides are usually unstructured and hence, difficult for machines to read. Cassandra decomposes the .pptx file into its sub-components such as text, tables, images, and equations among others.
- **Whiteboard Content:** This captures the handwritten content in the system and integrates it into the multi-modal processing pipeline. Intelligent segmentation and classification is used to align the content.
- **Semantic Content Integration:** The most advanced step among all is the integration of slides in the lecture's transcription. First, it integrates the appropriate slide content with the lecture's transcription and then integrates the whiteboard content according to the contextual information. Unlike other models, Cassandra doesn't just integrate texts but uses the semantic similarity between the spoken and written content to match the explanations with the correct context and visuals.
- **Retrieval-Augmented Generation (RAG):** RAG plays an important role in enhancing knowledge, thereby making the generated lecture notes more informative by incorporating appropriate external context, explanations, and examples from uploaded textbook PDFs.

By solving these interconnected challenges, Cassandra brings together AI, natural language processing, computer vision and multimodal data handling in a seamless automated note generation system.

2.4 Goals and Objectives

The Cassandra project is designed to tackle the challenges identified earlier by having a set of clearly defined goals and objectives that define what it is and what it will deliver. These goals serve as guidelines for development, showing the direction the project takes, and can also be used to measure its success.

1. Automated Note Generation System

It can be designed as an intelligent platform that records live lectures, transcribes the instructor's voice, captures whiteboard content, and extracts information from presentation slides. Using a Retrieval Augmented Generation approach, it would also incorporate relevant course materials knowledge. Finally, AI models would merge all these sources into structured, easy-to-read lecture notes.

2. Flexible Teaching Support

The system should be flexible enough to accommodate various teaching methodologies ranging from using pre-uploaded slides, use only of the whiteboard during the class, and uploading outlines before lectures.

3. Dual AI Assistant System

Use two chatbots powered by artificial intelligence for different objectives; one will assist the teacher in finding and explaining information in any uploaded material like books and PDFs, while the other will work as a helper to the student answering their questions based on lecture notes prepared using the previous material.

4. Complete Learning Management Tools

Add the full functionalities of the LMS system including course generation, file uploads/submissions, private messaging, and announcements. Provide a solution similar to Google Classroom but equipped with AI-powered assistance.

Altogether, these goals join to create a comprehensive educational ecosystem automated note taking is but the first step in transforming how teachers teach and how students learn.

2.5 Project Scope

This project will result in a complete, responsive web application called Cassandra, built to meet the goals described in the previous section.

2.5.1 In Scope Features

The features of the final application will be:

- **Web Based Platform with User Accounts:** Separate teacher and student dashboards with personalized access and communication.
- **Teacher Tools:** The ability to create and manage courses as well as upload educational materials (PDF textbooks), where the latter provides context in which the RAG system operates. Teachers can start the lecture recording, select the lecture mode (Slides, Whiteboard, Slides + Whiteboard, Audio only), and gain access to a special AI assistant chatbot that allows them to query only their uploaded course materials.
- **Student Tools:** Enroll in classes and access a repository of all automatically created and organized lecture notes. See announcements and message teachers. Students can also submit assignments and access an AI Chatbot dedicated to answering questions based strictly on the generated lecture notes.
- **Automated Lecture Processing Pipeline:** A backend system that handles the data of lectures intelligently at multiple stages:
 - Recording a sample of the instructor's voice for speaker identification.
 - Capturing audio during lectures.
 - Transcribing and translating the audio.
 - Filtering and isolating the instructor's voice.
 - Text, tables, images and equation extraction from slides and whiteboard content.
 - Performing semantic alignment between the lecture transcript, slide text and whiteboard content.
 - Application of RAG to generate explanations for the notes.
 - Generating well structured notes, which refines and organizes the content into readable study material.
- **Dual AI Assistant System:** Two AI-powered chatbots, one to assist teachers in improving upon their uploaded content and another to assist students by answering their questions from the generated lecture notes.
- **Communication Tools:** Communication tools within this solution will cover built-in messaging services and announcements within the course, allowing for interactions between the teacher and

students.

2.5.2 Project Deliverables

Output for this project will include::

- Web application with all functions mentioned above fully operational.
- Manual with detailed description of how to use the app for teachers and students.
- Documentation describing the design process, implementation process and testing result.

2.5.3 Out of Scope

The following items are not included in the project to maintain focus on core objectives:

- Hosting or providing server infrastructure for deployment.
- Development of native mobile apps for iOS or Android.
- Integration with external services or APIs beyond those explicitly mentioned for the main processing pipeline (for example, third party calendars or external LMS platforms).
- Functionality of creating final lecture notes in languages other than English
- Teacher recording hardware is provided, such as a microphone for audio recording, a mobile camera for capturing whiteboard images, and a projector for slides. The system pre-supposes that the instructor already has appropriate hardware to record high quality audio.
- Offline functionality, since the design assumes that all users (Teachers and Students) have stable, high speed internet connectivity for streaming lecture material and communicating with the web application.
- Acceptance of slide formats other than PPTX (PowerPoint).
- Extracting diagrams, charts, animations, or content in languages other than English from PPTX slides. The system is limited to extracting text, images, mathematical equations, and tables.
- Extraction of drawings, complex shapes, content not in English language, or low-quality hand-written content from whiteboard images. The system is limited to extracting text, mathematical equations, and tables from whiteboard images.
- Usage of document formats other than PDF or DOCX, such as HTML or PPTX, by the RAG component.
- Support for recorded online classes is provided (example: Zoom, Google Meet, MS Teams, etc.).

The recordings of on-campus lectures only are supported.

2.6 Sustainable Development Goal (SDG)

Cassandra is highly relevant to SDG 4: Quality Education, which aims to “ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.” This project is, therefore, primarily aimed at making learning effective, accessible, and more equitable by automating the processes involved in generating high-quality, structured, and easy-to-understand study materials.

By doing so, Cassandra supports the education of all students inclusively, irrespective of note taking skills, learning pace, or background as shown in the (figure 2.1). Each learner is presented with the same robust resources, which enable them to pay more attention to understanding rather than rush to take notes. Besides, this reduces stress, it enhances engagement during lectures. Ultimately, Cassandra converts classroom discussions into permanent, well-organized knowledge resources that help bridge educational gaps and make quality learning more achievable for all.



Figure 2.1: SDG 4

2.7 Constraints

Constraints will be related to the range of deployment, hardware requirement and functionality limitations:

- **Platform and Deployment Scope:** The range of deployment for this project includes only the development of the web application while development of native mobile apps for iOS and Android devices is excluded from this project.
- **Integration Constraints:** The solution excludes integration with any external services or APIs

other than those required for the core processing pipeline. Examples include but are not limited to third-party calendars and Learning Management Systems. The functionality of the core pipeline is critically dependent on access to and the commercial availability of the needed Large Language Models (LLMs) and high-accuracy third-party APIs, such as Faster-Whisper for Speech-to-Text.

- **Input and Output Constraints:**

- Final structured lecture notes are created only in English. Although the system can take code-switched input like, mixed Urdu and English speech the final structured output must be constrained to clear English text only.
- The system accepts only lecture slides in .PPTX format. Also, content extraction from slides is limited to English text, images, mathematical equations, and tables while diagram, chart, and animation extractions are beyond the scope.
- The specialized whiteboard recognition feature is designed to only process the English language, mathematical equations, and tables. It cannot extract items such as drawings, complex shapes, or poor quality handwritten content.
- The RAG system requires reference materials that are uploaded to be in formatted, searchable PDF or DOCX. It does not work with reference books that are only images and with any other document type such as PPTX or HTML.

- **Operational and Hardware Assumptions:**

- The system design assumes that all users both Teachers and Students are on stable and high-speed internet connectivity, since offline functionality is not supported.
- It is assumed that teachers have the right microphone or recording hardware for recording high quality audio and an appropriate mobile phone for capturing whiteboard images.

- **Performance and Processing Constraints:** With the complexity of the AI pipeline involving transcription, extraction, embedding generation, RAG retrieval, and final LLM synthesis, the system gives priority to accuracy over instantaneous real time in outputting the complete note generation process. Also, the core AI pipeline requires, especially the Large Language Model components along with the image processing, dedicated access to the Graphics Processing Units for timely and effective inference.

2.8 Business Opportunity

The Cassandra project represents an important advance in the capability of educational platforms to capture and transform the rich, dynamic, multi-sensory environment of a live lecture into durable, structured study resources.

- **Dealing With the Limitation of Traditional LMS:** While traditional learning management systems (like Moodle and Google Classroom) are efficient enough in terms of administrative capabilities that is, uploading documents and assignments. They do not include features related to interactive, multi-sensory elements of live lecture, thus becoming merely places where material is stored. And thus missing important context, such as explanations that a teacher can give off-hand and link with visuals.
- **Solving the Core Student Problem:** The Core Student Problem is the mental load of having to listen and understand in detail a complex matter while making detailed organized notes. It forces them into incomplete, messy, or disjointed notes, which are time-consuming to revise effectively. The business opportunity free's students from this task with Cassandra's system, that automatically prepares proper notes as the student is freed to focus on the material.
- **Creating an All in One Companion:** Integrating the Automated Note Generation System with Cassandra completes all LMS features such as course creation, direct messaging, and assignments, thereby making it an all-in-one educational companion. This duality redefines how teachers deliver knowledge and students interact with it.

2.9 Stakeholders Description/User Characteristics

The Cassandra platform is intended as a web-based educational ecosystem that supports two main user roles, each with personalized accounts and specialized tools.

2.9.1 Stakeholders Summary

Teachers are course creators and instructors who manage the creation of courses, conduct live lectures with slides, whiteboard, or audio they upload reference material in the form of textbooks and slides, this platform auto-creates the lecture notes for them and also utilize the Teacher's AI Assistant based on their uploaded content and the textbooks. Students are learners enrolled in courses, requiring comprehensive and accurate study material. They use the generated notes repository, submit assignments, and utilize the Student's AI assistant for study support.

2.9.2 Key High-Level Goals and Problems of Stakeholders

The system is intended to solve distinct high-level problems and help achieve pre-determined goals for the following two primary user groups:

2.9.2.1 Problem Solved for Teachers

Traditional LMS lacks the tools to capture the rich context of live teaching.

Goals Achieved: To use a platform that would provide Flexible Teaching Support for varied four delivery modes like slides only, classroom whiteboard, slides + whiteboard, or audio only. To have access to a complete LMS for course creation, management of the students, assignment distribution, and communication. To get access to an AI Assistant Chatbot, which will fetch and explain the content directly from their uploaded course material or textbooks. To make the management of lecture contents easier and enhance the overall content delivery.

2.9.2.2 Problem Solved for Students

Students battle with the mental load of multitasking including note taking and comprehension in fast-paced lectures, along with fragmented notes.

Goals Achieved: Receive Automated, Comprehensive Study Notes that compile speech, slides, whiteboard content, and external knowledge into a structured document. Leverage an AI Assistant Chatbot strictly scoped to answer questions based solely on the generated lecture notes for review and clarification. Enroll in courses and manage coursework, including viewing a repository of generated notes and submitting assignments.

2.10 Conclusion

This chapter set up the basic roadmap for the Cassandra platform: from the central problem that students cannot create complete and organized notes in real time while understanding the complex lecture content, to the project's vision of solving this by creating a web-based intelligent platform that will automatically capture and synthesize multimodal inputs, live audio recordings, presentation slides, classroom whiteboard content, and external knowledge, into clear, structured study resources.

The crux of the solution is the Automated Note Generation System, which relies on advanced techniques like RAG for knowledge enrichment and LLMs to logically structure the final output. Apart from note-taking, the scope of this project goes on to provide a complete LMS and a dual AI assistant system.

Chapter 3 Literature Review

This chapter provides an overview of the current landscape of digital learning technologies and the relevant academic literature. This review critically assesses functionality and shortcomings of existing solutions in order to outline the considerable research gap that the “Cassandra” platform is aimed at overcoming. The review will also consider commercial systems and seminal scholarly work to establish a precise context for the innovative contributions of the platform to learning technology. The review is broken down thematically into an account of existing platforms, an overview of scholarly work in automated content generation, and enabling technologies assessment at the core.

3.1 Definitions, Acronyms, and Abbreviations

To facilitate clarity throughout this chapter, the following table provides definitions for key terms and abbreviations utilized in the discussion of educational technology and artificial intelligence. The definitions are placed within the context of the Cassandra project and are intended to act as an immediate reference for the reader.

Table 3.1: Commonly Used Abbreviations and Their Definitions

Abbreviation	Definition
AI	Artificial Intelligence
ASR	Automated Speech Recognition
HTR	Handwritten Text Recognition
LLM	Large Language Model
LMS	Learning Management System
Multimodal AI	Artificial intelligence that processes and interprets multiple types of data, such as text, audio, and images.
NLP	Natural Language Processing
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation

3.2 Detailed Literature Review

3.2.1 The Landscape of Digital Learning Platforms

Before surveying the specialist academic scholarship, critical attention must be given to the dominant commercial platforms in use within education today. These systems determine the dominant digital experience of both teachers and students and their deficiencies establish the practical problems that next generation tools need to address [1]. The following section critically assesses traditional Learning Management Systems and newer lecture capture and transcription services for their ability to support

and enable the live lecture experience as well as where they fail [2].

3.2.1.1 Traditional Learning Management Systems (LMS)

Classical Learning Management Systems (LMS) such as Moodle and Google Classroom are found everywhere in present day education. They are excellent administrative tools, functioning as the central location for distribution of course material, assignment submission, and announcements. These are important organizational resources, functioning as the electronic archive for storing course syllabus, course readings, and drop off sites [3].

However, their fundamental flaw lies in the immobility of these systems. An LMS is basically an electronic file cabinet, an archive for the pre-arranged static information, and not a medium through which the dynamic and multi-modal aspects of a live lecture may be recorded. All the wealth of information, including verbal explanations of the professor, impromptu diagrams drawn on the whiteboard, and relationships between visual and verbal information, get utterly wiped away. This leads to the gap between the actual learning taking place during a lecture, and study material prepared for students, giving rise to confusion among students regarding actual happenings in the classroom. Thus, although these websites provide a good administrative base for a course, there is nothing much in capturing knowledge produced in the process; thus, ways to preserve this ephemeral information become essential.

3.2.1.2 Commercial Lecture Capture and Transcription Services

There have been new innovations in the form of commercial tools to address the content gap left open by old LMS technologies. Unfortunately, these technologies tend to be one-and-a-half solutions, insofar as they only capture one modality of lecture data independently, and cannot present a complete package.

1. **Lecture Recording Platforms:** Technologies like Panopto and Echo360 are a major step up when it comes to capturing audio-visual lecture content. The purpose of these tools is to create video recordings of the lecture including synchronization with visual presentation materials. The result is a resource which can help students go back over their lectures whenever they choose [4]. It is certainly helpful, yet the format is an unorganized, searchable video, which contains no organized, condensed text for easy referencing. A student still has to watch many hours of video content to pick out the relevant pieces of information from the recording [1].
2. **Automated Transcription Services:** Personal automated transcription technologies such as Otter.ai are made possible by advancements in Automated Speech Recognition (ASR). These programs provide text transcriptions of lectures with very high accuracy rates. Yet in an academic context their limitations are keenly felt: these tools do not integrate visual content from white-

boards or slides, breaking the critical association between what was said and what was shown [2]. The transcript has no semantic association with material shared, and the general purpose design of such tools is ill-suited to distilling educational material into coherent notes. Finally, these tools cater to only one modality sound alone, without the rich multi-modal interaction that typifies good teaching. [1].

Taken together, these commercially available products leave a student with a dis-articulated set of holdings a video file and a raw text transcript rather than a single synthesized and structured study material.

Table 3.2: Comparative Analysis of Commercial Educational Platforms

Feature	Moodle / Canvas [3]	Otter.ai [1]	Glean (Genio)	Cassandra
Full LMS Functionality	Yes	No	No	Yes
Live Audio Transcription & Diarization	No	Yes	Yes (Audio Recording)	Yes
Presentation Slide Decomposition (Semantic)	No (Static Hosting)	Partial (Image Capture Only)	No (Manual Import)	Yes
Live Whiteboard HTR (Text, Math, Tables)	No	No	No	Yes
Semantic Audio Visual Alignment	No	No	No	Yes
RAG based Knowledge Enrichment	No	No	No	Yes
Integrated AI Assistant	Partial (Auxiliary Tasks)	Yes (Transcript Q&A)	Yes (AI Outline / Quiz)	Yes (Dual System)

3.2.2 Studies on Automated Educational Content Generation

The academic society has sought for a long time to leverage technology for learning indeed, there has been an increasing number of research studies related to the automation of education resource generation. This section describes important research papers that make up the science involved in systems like Cassandra by pointing out the research in speech recognition for academia, multimodal content fusion, and building AI-based student aid systems.

3.2.2.1 Speech Recognition and Transcription in Academia

Success for any automated note-taking system requires its ability to effectively transcribe a lecturer's speech. Smith, Thompson, and Davis through their comparison of educational speech recognition accuracy, provide a critical platform in this regard. Their work intentionally tested a number of ASR models against lecture recordings with the conclusion that state-of-the-art systems now have an accuracy high enough for automatic transcription to be extremely feasible in academic contexts [5]. The research is important in that it establishes the viability of the heart of the audio processing module of a system such

as Cassandra [1]. Meanwhile, the research also recognized ongoing issues, including managing domain specific terminologies and inconsistent audio quality, thus emphasizing the importance of stronger, integrated solutions that can overcome these restrictions.

3.2.2.2 Multimodal Content Integration

Since realizing that a lecture represents more than just the words spoken, researchers have been working on ways to combine multiple sources of information. Particularly relevant to this research is the 2022 paper by Kumar et al., in which they developed a deep learning approach toward the summarization of lectures, taking into consideration both audio and visual inputs. Their research clearly showed that the use of multiple modalities like transcription and images used in the presentation would enrich the resulting summary [6]. This gives very convincing evidence supporting the architecture of Cassandra, one of the most important tenets of which is mentioned above. Nevertheless, the research by Kumar et al. also highlights serious technical challenges in aligning the semantics of the spoken utterances and the specific visual objects referred to in them [7]. This problem explains the necessity for advanced semantic alignment addressed explicitly by the architecture of Cassandra.

3.2.2.3 AI-Powered Student Support Systems

In addition to its ability to create content for learning purposes, AI has already been able to demonstrate its ability to give students immediate assistance. The scoping review of Rodriguez, Kim, and Johnson (2022) on the usage of AI chatbots in educational platforms examines their contribution, stating that AI assistants can easily pique the students' interest and provide priceless help through a button-press answer [8]. However, the review also makes an important observation: success depends ultimately on the quality and completeness of the underlying knowledge base. An AI assistant can be no better than the material it is built upon. This finding provides a central issue that Cassandra is designed to solve [9]. The fact that Cassandra automatically creates a lecture specific knowledge base first, of high quality, forms the best foundation upon which this AI assistant is built so as to provide accurate, context-sensitive, and truly useful help to students.

3.2.3 State of the Art in Core Enabling Technologies

Inventing an end-to-end system like Cassandra requires overcoming serious technical difficulties in a host of different areas of artificial intelligence [10]. The success of the platform depends not only on the novelty of the system design but also on the strategic combination of state-of-the-art technologies. This section summarizes briefly the state-of-the-art and limitation of two key enabling technologies, namely Handwritten Text Recognition and Retrieval Augmented Generation to outline the difficulty of the task

and provide the necessary context for Cassandra’s suggested technical solution [11].

3.2.3.1 Challenges in Handwritten Text Recognition (HTR)

The ability to digitize whiteboard content properly is an integral component of an advanced lecture capture system. HTR research in recent years has evolved from early models with CNN-LSTM architectures (Nikitha A. et al., 2020) to advanced vision transformer models (Yuting Li et al., 2024) [12]. Although progress has been made, there are still quite a few limitations discussed in the literature. HTR models show poor performance with cursive, styled, and/or noisy handwriting [10]. They rely heavily on large training datasets for each target language. They can thus hardly be adapted to different languages or fields like mathematics. Finally, while advanced ViT models show state-of-the-art performance, they require tremendous computational power that makes them incompatible with real-time applications [13]. These challenges are all clear indicators that a general-purpose HTR solution will fall short and Cassandra’s specialized method will be required, where whiteboard content is first classified by category as text, math expressions or table formats and recognized via separate models for each domain.

3.2.3.2 Knowledge Enrichment with Retrieval-Augmented Generation (RAG)

For generating fully inclusive study notes, it becomes essential to augment lecture information with credible external knowledge. According to Lewis et al. (2020), it can be done efficiently with the help of the Retrieval Augmented Generation paradigm. The RAG method boosts the Large Language Models capabilities by initially retrieving knowledge from a document corpus, and then utilizing it to guide the generated text. The approach significantly improves the generated text’s factuality and contextualization. However, the main drawback of the RAG method in such cases is obvious: its effectiveness relies entirely on having a credible domain-specific knowledge base [11]. This makes it evident that Cassandra could play a crucial part in filling in this gap the system needs to intake and create such a high-fidelity knowledge base out of an ever-evolving multi-modal flow of data, which is a live lecture and is thus initially unavailable in any form for querying.

3.3 Literature Review Summary Table

The table 3.3 below summarizes the systems and technologies described in this chapter. It allows quick assessment of the characteristics and gaps of each of the approaches, and highlights the research areas where Cassandra fills the existing gaps.

Table 3.3: Summary of Literature Review

System / Author(s)	Key Features / Method	Strengths	Limitations / Research Gap
Moodle / Google Classroom [3]	Web based LMS platforms for course administration.	Effective for file sharing, assignment management, and announcements.	Static content repositories; do not capture or structure live lecture content.
Panopto / Echo360 [4]	Lecture capture systems that record video and synchronized slides.	Provides a replayable video record of the lecture.	Output is a passive video, not structured, searchable, or summarized notes.
Otter.ai / Transcription Services [1]	Automated Speech Recognition (ASR) services for audio to text conversion.	High accuracy in transcribing spoken language.	Lacks integration with visual content (slides/whiteboard); context unaware.
Smith, Thompson, and Davis (2022) [5]	Comparative study on speech recognition accuracy in educational settings.	Confirms high potential for ASR in academic environments.	Highlights remaining accuracy challenges and the need for robust systems.
Kumar, Singh, and Patel (2022) [7]	Deep learning approach for multimodal lecture transcription and summarization.	Represents that combining audio and visuals improves content quality.	Identifies semantic alignment between modalities as a major technical hurdle.
Rodriguez, Kim, and Johnson (2022) [8]	Systematic review of AI chatbots in educational platforms.	Shows AI chatbots improve student engagement and support.	Effectiveness is limited by the quality of the underlying knowledge base.
HTR Research (e.g., Nikitha, 2020; Li et al., 2024) [10, 13, 12]	Research into recognizing handwritten text using CNN, LSTM, and Transformers.	Increasing accuracy on benchmark datasets like IAM and RIMES.	Struggles with diverse styles, noise, and language dependency; requires specialized pipelines.
Lewis et al. (2020) [11]	Retrieval Augmented Generation (RAG) framework for NLP tasks.	Improves factual accuracy by grounding responses in provided documents.	Dependent on a pre existing, high quality, domain specific knowledge source.

3.4 Conclusion

From the above review of literature and commercial applications, there is an evident gap for a more comprehensive approach to digital learning. Several approaches have been developed for different aspects of digital learning but they all operate in isolation: Learning Management Systems control courses, transcription systems transcribe audio files, while lecture capture systems record videos [1]. Although each system operates on one specific aspect, they cannot combine these separate channels into a structured learning tool. Simultaneously, studies on education have proved that AI can produce enriched learning resources. Studies have demonstrated the feasibility of automatic transcription, the benefits of multimodal integration, and the necessity of AI tutoring [6]. However, the studies have also brought out several technological challenges that need to be resolved, including semantic alignment, handwriting

recognition, and dependence on high-quality input data for AI assistants.

The research gap that needs to be filled, therefore, is quite evident: There is a lack of comprehensive systems that can automatically record, align, and merge all the multimodal aspects of live lectures to create a single set of enhanced learning notes.

Cassandra is a new solution specifically tailored to bridge this gap. By building an end to end pipeline that intelligently combines these fragmented streams of data, it seeks to take fleeting moments of teaching and turn them into enduring, interactive learning resources [11]. This puts the project not in the position of incremental advancement on top of what is already being done, but as a complete redesign of what technology can do to enhance the fundamental educational experience. Here is the comparison of Commercial Educational Platforms and Cassandra illustrated by table 3.2. The following chapter outlines the particular technical approach engineered to make this vision a reality.

Chapter 4 Software Requirement Specifications

This chapter describes the functional and technical specifications of the Cassandra: All You Need in a Class Companion web application. It outlines all the necessary system functionalities, quality attributes, non-functional constraints, and assumptions to give an overall account of the software requirements. The aim is to outline the external system behavior and required design constraints concisely.

4.1 List of Features

Cassandra is an interactive learning platform that combines a complete Learning Management System (LMS) with state of the art AI based note creation. The primary features of the system include:

- **User Management and Dashboards:** Customized accounts and dashboards for Teachers and Students.
- **Course and Material Management:** Instructors can upload required course content, such as lecture presentation (PPTX) and reference books. Make announcements and upload assignments for students.
- **Automatic Lecture Recording & Processing:** Complete backend pipeline to capture live lecture audio, transcribe, isolate the teacher's voice, extract content from slides, whiteboard and text books, and sync all data.
- **Note Generation Powered by AI:** The core functionality based on a Retrieval Augmented Generation (RAG) architecture, and Large Language Models (LLMs) to merge lecture material, and textbook material into organized, readable notes.
- **Double AI Assistant Chatbot System:** Two separate chatbots one for instructors to ask questions of their posted content, and another for students to question the content of lecture notes generated.
- **Communication tools in LMS:** Built in functionalities for announcements, one-on-one private messaging.
- **Mode Support:** The system supports creating notes from different combinations of inputs: Audio only, Slides based, Whiteboard based, or Slides + Whiteboard.

4.2 Functional Requirements

4.2.1 Teacher's Functional Requirements

System will enable teachers to:

- Register, log in securely, and edit their profile details and security settings.
- Build new courses, manage course details, generate enrollment keys, and view enrolled students.
- Post course materials (PDF textbooks or DOCX) to set the RAG context for a course.
- Start and end a new lecture session.
- Create, edit, and publish course specific announcements to all registered students.
- Create, distribute, and monitor student assignments for coursework within the course dashboard.
- Engage with the only AI chatbot approved to ask and obtain information from them using exclusively their course material postings.

4.2.2 Functional Requirements of Students

System shall enable students to:

- Register, securely log in and control their basic profile data.
- Enroll in courses via a course key sent by the instructor.
- View a repository of all automatically generated and organized lecture notes for their enrolled courses.
- Check announcements posted by the teacher.
- Ask questions from teachers through private messaging.
- Submitting assigned coursework and tracking the submission status of such.
- Access and use only the AI chatbot assigned for asking questions on the exclusive content of the generated lecture notes. Utilize the knowledge of uploaded course materials

4.2.3 System Functional Requirements

The system will:

- Record teacher's audio. Apply the teacher's voice signature to eliminate noise, separate their speech, and accurately transcribe the audio, including code switched speech translation.
- Extract text, tables, images, and equations from PPTX slides.
- Extract text, tables, and equations from classroom whiteboard images.
- Align semantically and match the spoken transcript with the corresponding content from slides / whiteboard / slides + whiteboard.

- Utilize the knowledge of uploaded course materials by querying with the RAG framework to contextualize, explain, and enrich the transcribed lecture material.
- Employ LLMs to summarize, optimize, and logically structure the merged material into a structured final lecture note document.

4.3 Quality Attributes

The quality attributes specify the characteristics and non-functional requirements of the Cassandra platform such that it delivers a strong and worthwhile user experience.

4.3.1 Reliability

The application should focus on reliability to provide a constant and consistent user experience, particularly during live lecture recording. The processing and recording pipeline for lectures should be designed with fault tolerance such that any intermediate failures do not result in the loss of the entire lecture data. All user data such as course material and created notes need to be stored securely and recoverable in the event of system failure.

4.3.2 Maintainability

Maintaining the project will be quite simple due to the high documentation and application of a modular and decoupled architecture. This will help the application be designed using industry standards and can always improve further to fix any problems and incorporate features that help it remain sustainable despite technological developments.

4.3.3 Correctness

Accuracy is very important in the system's effectiveness. There needs to be high levels of: transcription accuracy (especially for code-switched speech), semantic alignment (between the spoken words and the image), and quality of the RAG enrichment (the information from the textbooks should be factual and true). The generated notes should clearly and precisely summarize the lecture in question.

4.3.4 Usability

It is important that the application has an intuitive interface since this will make the users understand how to manage their lectures and get the relevant notes. The interface must have an appealing and easy-to-use design. The generated notes should also be easy to read for inclusive education.

4.3.5 Efficiency

Efficiency is key owing to the nature of AI computations. The system needs to meet two main requirements: Note Generation Latency (notes generated must be available within a certain time limit) and System Responsiveness (UI of the web application should always be responsive to all users regardless of heavy loads).

4.4 Non Functional Requirements

This part explains all the non-functional requirements that specify how the system works and among them are operational constraints and quality characteristics. The system should be capable of supporting 50 active lecture sessions and 1,000 logged-in users at the same time during the busy hours without losing its responsiveness.

4.4.1 Performance

- **Concurrency:** The system should be capable of supporting 50 active lecture sessions and 1,000 logged-in users at the same time during the busy hours without losing its responsiveness.
- **Latency:** The user should not have to wait more than 5 seconds to get a response from the AI Assistant for a query related to the notes.

4.4.2 Security

- **Data Confidentiality:** Users personal data, course material uploaded, raw audio, and generated notes must be encrypted when they are being transmitted via HTTPS/SSL and stored in the Supabase (Database/Storage).
- **Access Control:** Role-Based Access Control will be strictly enforced, allowing students to access only course materials and notes for which they have enrolled.
- **Authentication:** Secure session management and strong password hashing should be implemented.

4.4.3 Extensibility

System architecture is such that it will incorporate loose coupling of modules making it possible to later bring in more than one multimodal input, for example, video feed analysis, or just to change a specific LLM or RAG vector database to switch or replace the AI core functionalities.

4.5 Assumptions

The following assumptions were made in the specification with the intention of constraining the problem domain:

- **Internet Connectivity:** It is assumed that all users (Teachers and Students) will have reliable internet connectivity with high speed connection capability for streaming lectures and interfacing with the web application.
- **Whiteboard Data Input:** The project assumes the presence of a preliminary system (e.g., a capture device) that handles the extraction of content from whiteboards.
- **External Service Reliability:** The project relies on the continued commercial availability, accessibility, and operational performance of required Large Language Models (LLMs) and high-accuracy Speech-to-Text APIs / libraries (Python-based) necessary for the core AI pipeline.
- **Teacher Recording Hardware:** Teachers are presumed to possess and utilize the appropriate microphone and recording hardware necessary for recording high-quality audio for the lecture sessions.
- **Reference Material Format:** Uploaded reference materials (e.g., textbooks, supplementary documents) utilized for the Retrieval-Augmented Generation (RAG) framework are assumed to be PDF or DOCX format.
- **Visual Content Simplicity:** Lecture slides (PPTX) and whiteboard images are assumed to primarily contain English content that the system is equipped to process: text, images, tables, and mathematical equations. It is assumed that the content does not contain complex or highly unstructured visual elements like intricate diagrams, flowcharts, or animations, as extraction of these elements is not supported.

4.6 Use Cases

This section lists use cases that represent the significant, central functionalities of the final system, particularly those related to the core AI-driven note generation. The use cases as shown in figure 4.1, 4.2 and 4.3 illustrate the interaction between the actors and the system.

Table 4.1: Teacher Initiates Lecture Session

Name	Teacher Initiates Lecture Session
------	-----------------------------------

Actors	Teacher
Summary	The Teacher after register its dashboard do logs in, navigates to a course, in course management section and click on the lecture recording, selects lecture mode and upload lecture slides or lecture agenda (topics), and starts a lecture session, beginning the automated data capture and processing pipeline.
Pre-Conditions	The Teacher must be logged in. The course must be created and selected. Necessary materials (textbooks) must be already uploaded.
Post-Conditions	A new lecture session is created. The system is actively recording audio and preparing for content alignment.
Special Requirements	None.
Basic Flow	
Actor Action	System Response
1. The Teacher navigates to the Course Dashboard and clicks Start New Lecture.	2. The system displays the Lecture Initiation Form, prompting for final materials and asking for lecture mode.
2. The Teacher confirms materials and start providing the voice lecture start formally.	4. The system begins live audio recording and content capture (slides / whiteboard / slides + whiteboard).
3. The Teacher begins the lecture.	6. The system displays the lecture control interface, confirming recording status.
Alternative Flow	
	None

Table 4.2: System Generates Final Lecture Note

Name	System Generates Final Lecture Notes
Actors	Teacher, System

Summary	Upon lecture completion, the system automatically triggers the complex AI pipeline to transcribe, integrate, enrich using RAG, and summarize all recorded multimodal data into a final set of structured lecture notes.
Pre-Conditions	The Teacher has successfully ended a lecture session. All raw data (audio, slides, whiteboard content) is available in the backend storage.
Post-Conditions	A well structured, final set of lecture notes is generated, saved to the database (supabase), and made accessible on the Course Dashboard for students and the teacher.
Special Requirements	Requires high performance computing for LLM and RAG framework processing, must handle data synchronization across multiple services.
Basic Flow	
Actor Action	System Response
1. The Teacher clicks End Lecture.	2. The system compiles all raw lecture data and queues the session for backend AI processing.
2. System begins the processing pipeline	4. The system displays a Processing status to all users and begins the resource intensive RAG and LLM summarization.
3. System performs RAG enrichment and final LLM summarization/structuring	6. The final lecture notes are generated, converted to a web readable format, and saved to the course database.
4. Processing is complete.	8. The system updates the course dashboard status to Notes Available and sends an automatic notification to enrolled students.
Alternative Flow	
5. Semantic alignment fails due to a major data corruption (transcription error, missing whiteboard data).	8-A. The system halts automated processing, saves the raw/partial data, flags the session with a status of Manual Review Required, and notifies the Teacher.

Table 4.3: Student Queries AI Assistant on Notes

Name	Student Queries AI Assistant on Notes
Actors	Student
Summary	The Student accesses the AI Assistant to ask a specific question about the content covered in a generated lecture note document, receiving an answer strictly scoped to the available notes.
Pre-Conditions	The Student must be logged in and enrolled in the course. The final lecture notes for the session must be available.
Post-Conditions	The Student receives a relevant and contextually accurate answer, and the query / response are logged for performance tracking.
Special Requirements	The AI Assistant must be strictly confined to the document context and must not provide external knowledge.
Basic Flow	
Actor Action	System Response
1. The Student accesses a specific lecture's notes and clicks the Ask AI button.	2. The system loads the full text of the notes as the context for the AI Assistant interface.
2. The Student types a question (e.g., Summarize the key takeaways from slide 7.) and submits it	4. The system processes the query against the loaded note context using the AI model (LLM).
3. The System generates a relevant response based only on the available note text.	6. The system displays the answer to the Student in the chat interface.
Alternative Flow	
4. The question is outside the scope of the lecture notes (e.g., What is the capital of France?).	6-A. The system responds with an automated message: I can only answer questions based on the content of this lecture's notes. Please rephrase your question to focus on the lecture material.

4.7 Hardware and Software Requirements

4.7.1 Hardware Requirements

- **Server CPU/RAM:** Minimum 16 GB RAM per core and a high-core-count CPU for handling multiple users and document processing.
- **Server GPU:** High-performance GPU (e.g., NVIDIA A100/V100, 40GB+ VRAM) for LLM and image processing.
- **Server Storage:** Fast SSD storage for quick access to models and user files.
- **Development Machine:** Quad-core Intel i5/AMD Ryzen 5 with at least 16 GB RAM for local development and testing.
- **Client Devices:** Modern web browser, microphone for audio recording, projector for displaying slides and a mobile camera for capturing whiteboard content.

4.7.2 Software Requirements

- **Frontend Framework:** We are using HTML, CSS, and JavaScript to develop responsive and interactive single page web app interfaces for both user dashboards.
- **Backend Framework:** We are using Python with Flask to handle the core processing pipeline and model serving.
- **AI/ML Libraries:** Hugging Face, PyTorch/TensorFlow, integration of LLM, ViT, RAG vector embedding/similarity search, and other core AI capabilities.
- **Database and Deployment:** We use Supabase for storing table data and files. The backend is deployed on Azure (CPU tasks) and Modal (GPU tasks), while the frontend is deployed via Netlify.
- **External APIs:** Supabase API for storing data, DeepSeek API for large language model tasks, and ConvertAPI for converting PPTX to PDF for the slide decomposition process.
- **Version Control:** Git and GitHub for source code maintenance, team development, and version tracking.

4.8 Graphical User Interface

4.8.1 Teacher Registration/Login Page

This is the Registration/Login page for teachers, where a teacher can register for the first time and then use the login option to access the dashboard. This is shown in figures 4.1 and 4.2 .

Cassandra

Join as a Teacher!

Create your account to manage courses, start AI-powered lectures, and provide enriched learning experiences to your students.

- Create and manage courses
- Start AI-powered lecture capture
- Auto-generate structured notes
- Query course materials with AI

Your All-in-One Class Companion.

Teacher Registration Form:

Login Register

Create Teacher Account

First Name: First name

Last Name: Last name

Email Address: you@example.com

Institute Name: Enter your institute name

Password: Create a password

✕ At least 8 characters
 ✕ At least 1 uppercase letter
 ✕ At least 1 lowercase letter
 ✕ At least 1 number
 ✕ At least 1 special character (!@#\$%^&*~)

Confirm Password: Confirm your password

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#)

[Create Account](#)

Already have an account? [Sign In](#)

Are you a Student? [Register here](#)

[Back to Home](#)

Figure 4.1: Teacher Registration Interface of the Cassandra System

Cassandra

Welcome, Teacher!

Manage your courses, start lectures, and create AI-enriched notes effortlessly. Transform your teaching experience with intelligent automation.

Your All-in-One Class Companion.

Teacher Login Form:

Login Register

Teacher Access Portal

Email Address: you@example.com

Password: Enter your password [Show](#)

[Secure Login](#)

[Forgot Password?](#)

Are you a Student? [Click here](#)

[Back to Home](#)

Figure 4.2: Teacher Login Interface of the Cassandra System

4.8.2 Teacher Dashboard Interface

The teacher's dashboard page, where they can view courses, create new courses, view profile info, and edit specific course material (as in figure 4.3).

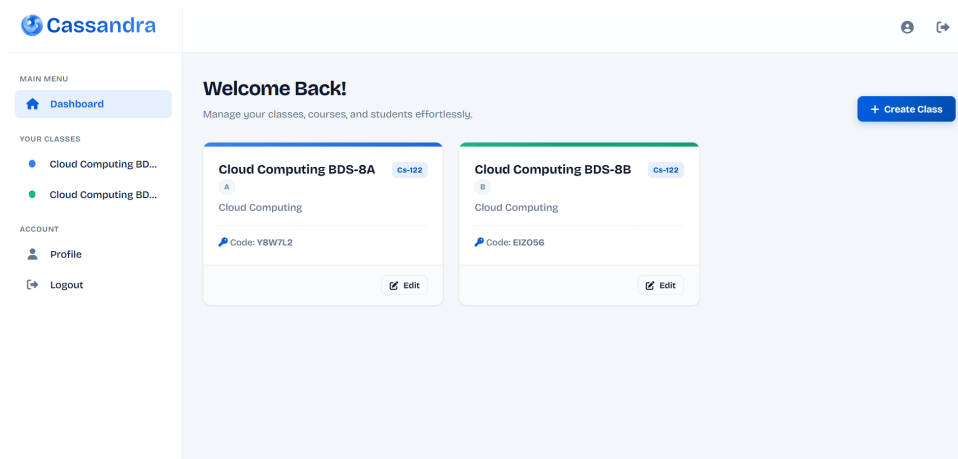


Figure 4.3: Dashboard For Course and Profile Management

4.8.3 Teacher's Course Management

A dedicated interface for a selected course, enabling the teacher to access all AI-generated notes, start the lecture recording, private chat with the selected student, do announcements and request assistance from the implemented RAG system (figure 4.4).

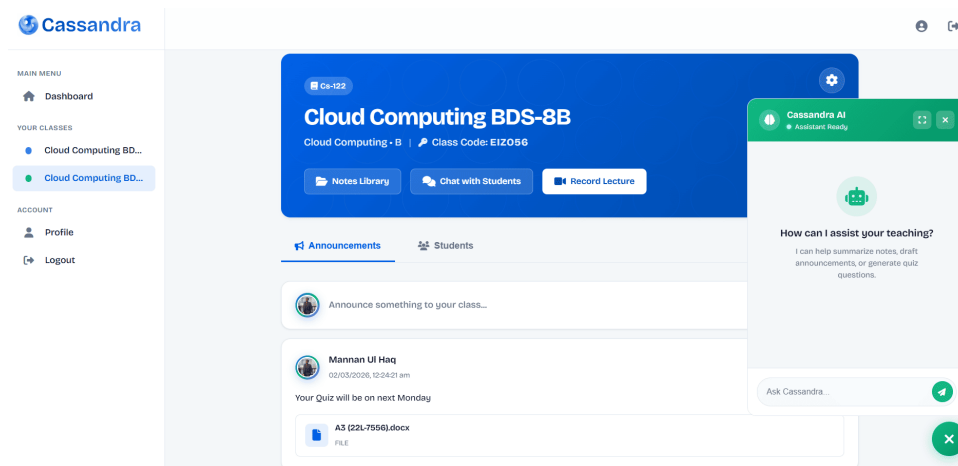


Figure 4.4: Course Management interface

4.8.4 Teacher's RAG Knowledge Base Management

A dedicated interface that allows teachers to upload and manage reference materials such as PDFs and DOCX files. These resources are indexed and used by the Retrieval-Augmented Generation (RAG) system to enhance lecture note generation (figure 4.5).

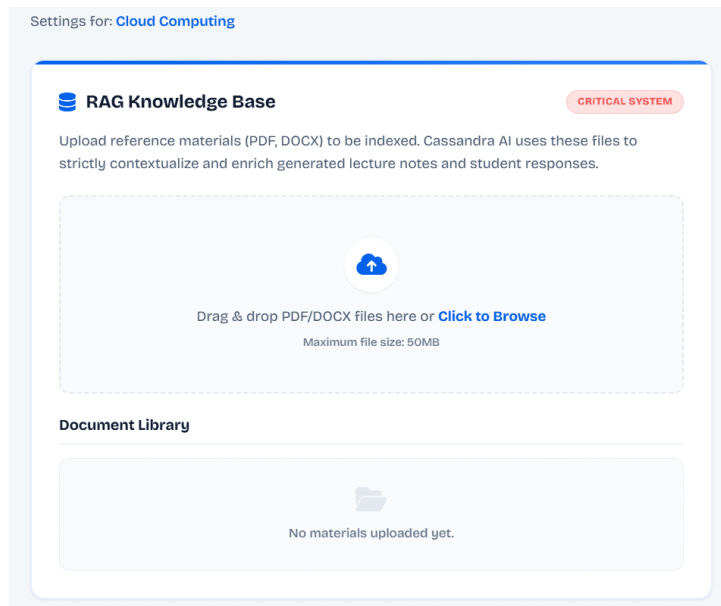


Figure 4.5: Teacher Interface For Managing the RAG Knowledge Base

4.8.5 Lecture Initiation for Teachers

The screen where the teacher prepares to start a session by selecting the lecture mode (Slides Only, Slides + Whiteboard, Audio Only, or Whiteboard Only), uploading PPTX slides, testing their voice, and then initiating the session (figure 4.6).

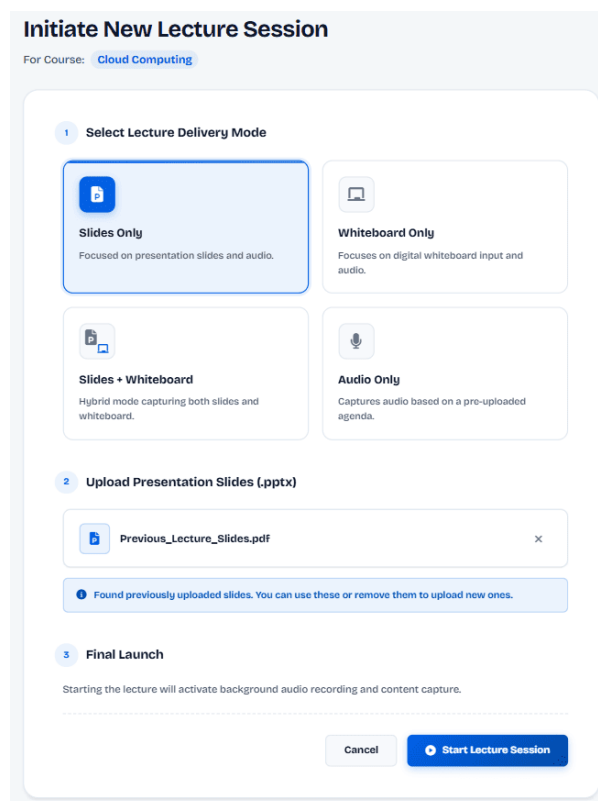


Figure 4.6: Lecture Session Initiation Interface with Delivery Mode Selection

4.8.6 Teacher's Live Lecture Interface

This key interface provides the teacher's multimodal view, allowing them to manage and monitor presentation slides and synchronized whiteboard content simultaneously, based on the selected options, during the lecture after clicking the Start Recording button as demonstrated in (figure 4.7).



Figure 4.7: Live Lecture Interface of Slides Only Mode

4.8.7 Student's Registration/Login Page

This is the Registration/Login page for students, where a student can register for the first time and then use the login option to access the dashboard as illustrated in figures 4.8 and 4.9.

Figure 4.8: Student Registration Interface of the Cassandra System

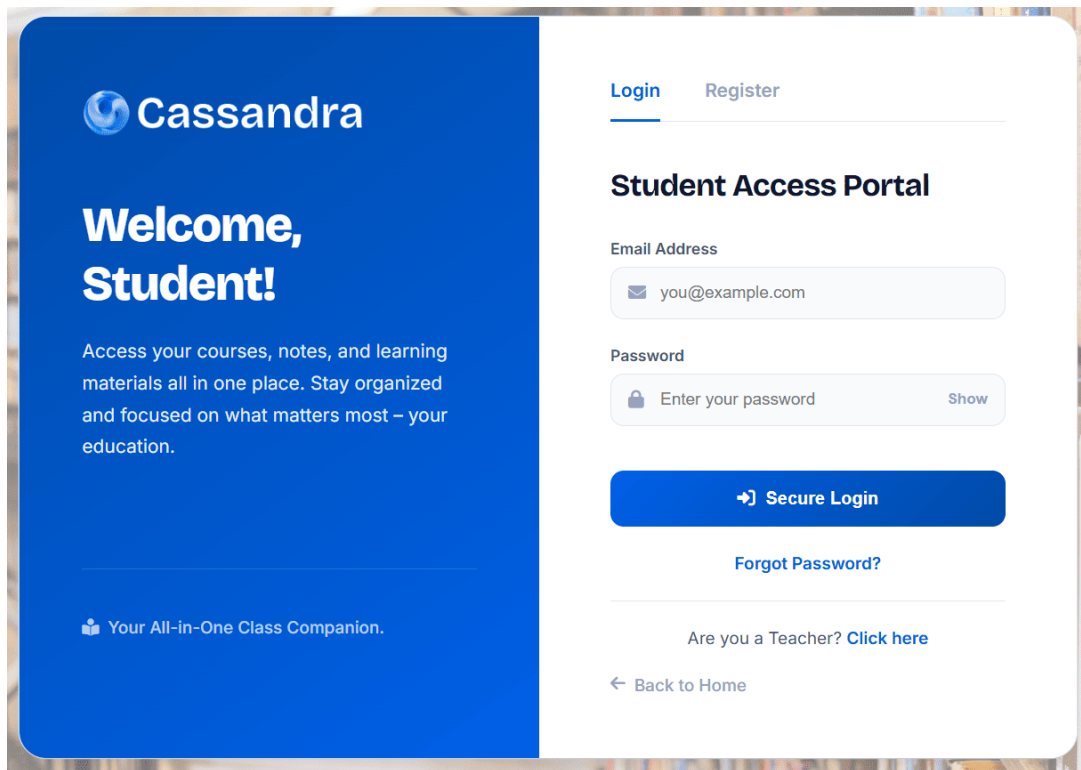


Figure 4.9: Student Login Interface of the Cassandra System

4.8.8 Student's Dashboard Interface

Students can view and manage their enrolled courses from a centralized dashboard. It displays all registered classes and allows quick access to course content. Students can also join new courses by entering a class code provided by the teacher, making enrollment simple and efficient (as in figure 4.10).

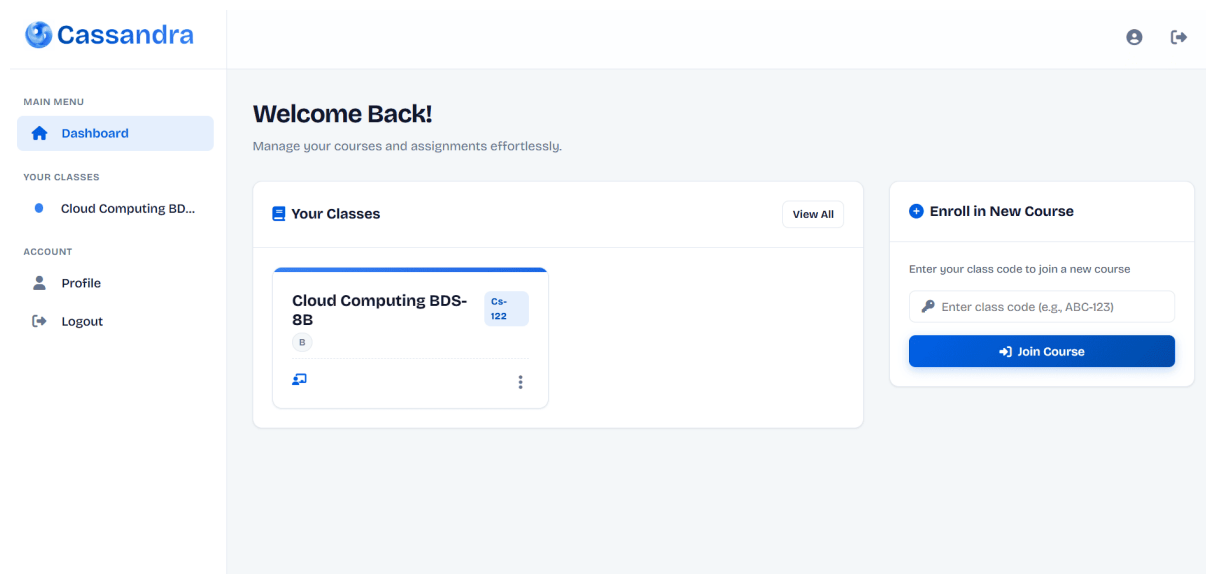


Figure 4.10: Student Dashboard Showing Enrolled Courses and Course Enrollment Option

4.8.9 Student's Course Management

A dedicated interface for students to access course materials, view announcements, and manage assignments. It also provides an AI assistant to help students understand concepts, summarize notes, and answer course-related queries. This can be seen in figure 4.11.

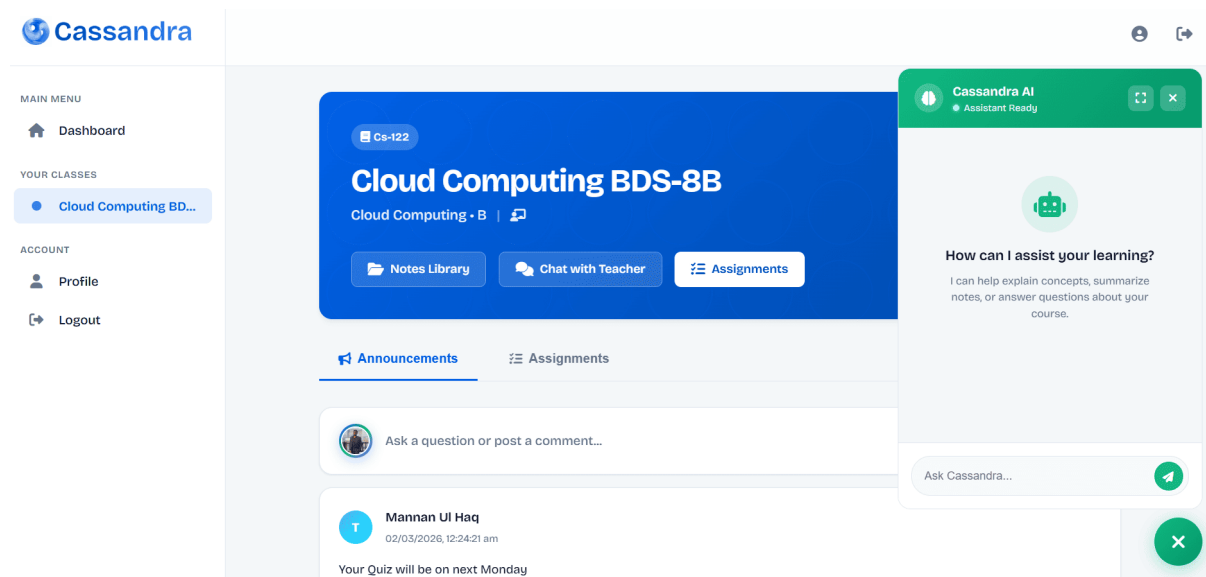


Figure 4.11: Student Course Interface With Announcements and AI Assistant

4.8.10 Student Notes Library

Centralized Interface to access all the AI-generated lecture notes by the students at one place. It will provide facility of viewing, searching and downloading notes efficiently for study purpose (shown in figure 4.12).

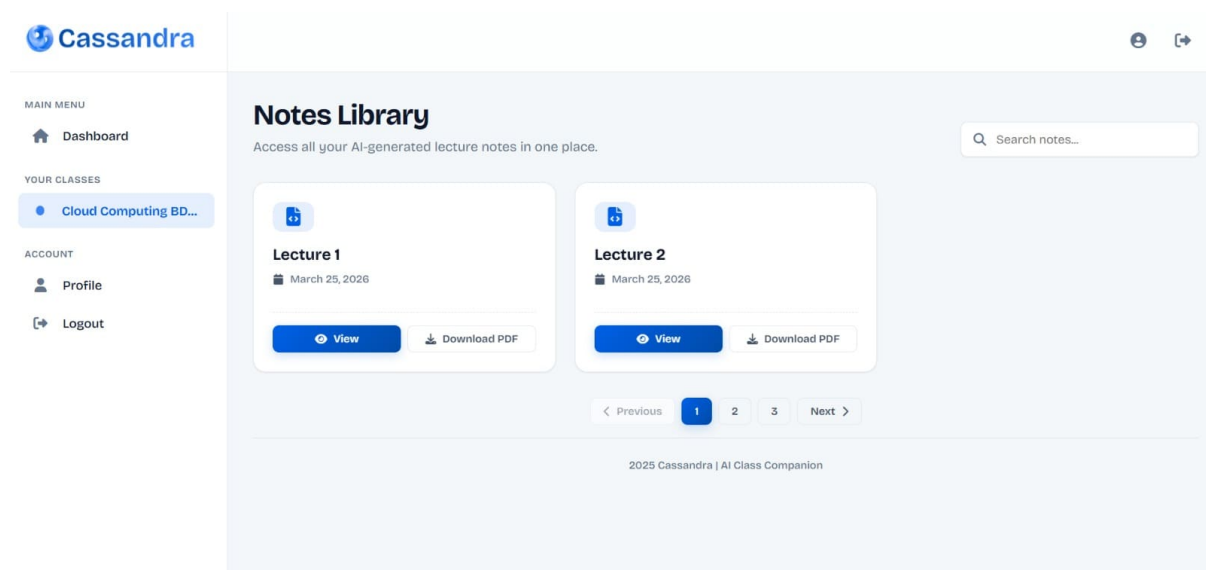


Figure 4.12: Student Notes Library With View and Download Options

4.8.11 Student-Teacher Messaging Interface

Real-time Messaging Interface for enabling interaction between students and teachers through direct communication in form of sending and receiving messages. It will provide facility of sending and receiving messages efficiently for queries and discussion purposes (figure 4.13).

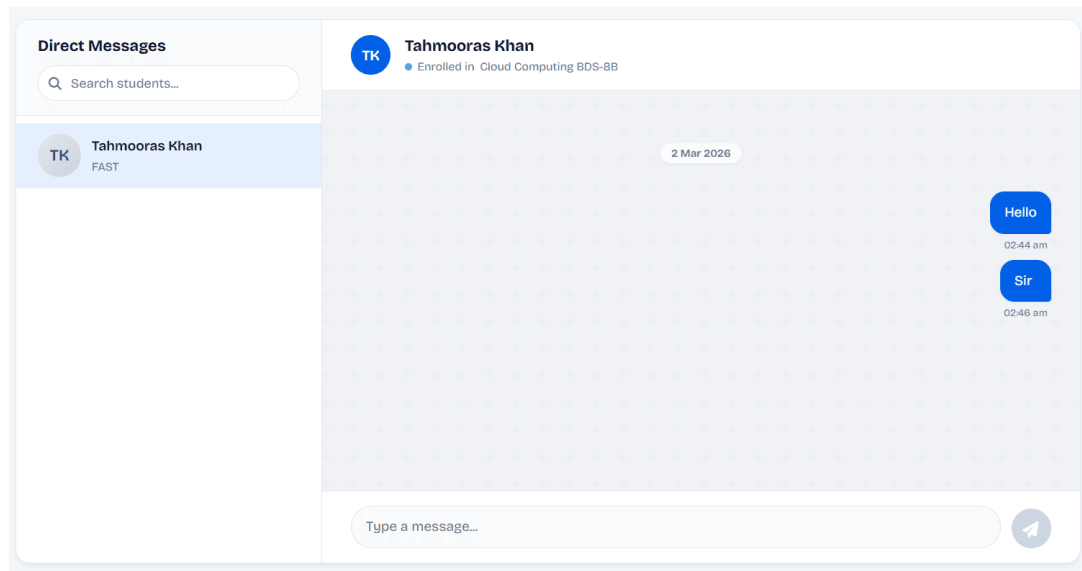


Figure 4.13: Student-Teacher Messaging Interface For Direct Communication

4.9 Database Design

4.9.1 ER Diagram

The Cassandra system may be represented through ER diagram which is shown in Figure 4.14 below. An ER diagram represents all the important entities in the database with attributes and relations among them. It helps in understanding the behavior and data structure of the entire system. Furthermore, it enables in identifying all kinds of dependencies and constraints on data items in the database.

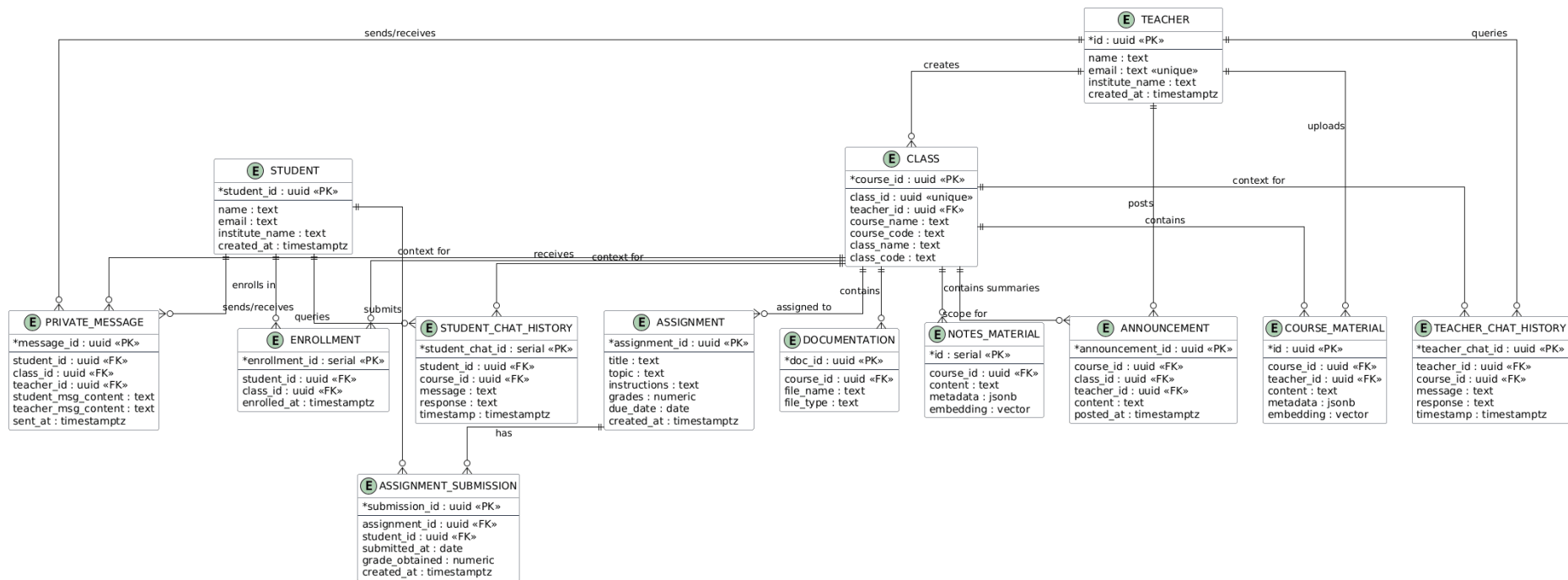


Figure 4.14: ER Diagram for Cassandra Database

4.9.2 Data Dictionary

Table 4.4: Database Entities and Attributes

Entity	Attribute	Data Type	Relation To	Description
STUDENT	student_id	uuid		Unique ID (PK)
	name	text		Student's full name
	institute_name	text		Name of the educational institution
	email	text		Student's email address
	created_at	time-stamp stamptz		Record creation timestamp
TEACHER	id	uuid		Unique ID (PK)
	name	text		Teacher's full name
	email	text		Teacher's email address (unique)
	institute_name	text		Name of the educational institution
	created_at	time-stamp stamptz		Record creation timestamp
STUDENT CHAT HISTORY	student_chat_id	serial	STUDENT CLASS	Unique Chat ID (PK)
	message	text		Student's message content
	response	text		Response to student
	timestamp	time-stamp stamptz		Chat timestamp
	student_id course_id	uuid uuid		FK to STUDENT FK to CLASS context
PRIVATE MESSAGE	message_id	uuid	STUDENT CLASS TEACHER	Unique Message ID (PK)
	student_id	uuid		FK to STUDENT
	class_id	uuid		FK to CLASS context
	teacher_id	uuid		FK to TEACHER
	student_msg_ content	text		Content sent by the student

Entity	Attribute	Data Type	Relation To	Description
	teacher_msg_content sent_at	text time-stamptz		Content sent by the teacher Timestamp of the message
TEACHER CHAT HISTORY	teacher_chat_id course_id teacher_id message response timestamp	uuid uuid uuid text text time-stamptz	CLASS TEACHER	Unique Chat ID (PK) FK to CLASS context FK to TEACHER Teacher's message content Response to teacher Chat timestamp
COURSE MATERIAL	id course_id teacher_id content metadata embedding	uuid uuid uuid text jsonb vector	CLASS TEACHER	Unique ID (PK) FK to Class FK to TEACHER who upload Extracted text content for RAG File metadata (size, page count, etc.) Vector embeddings for AI search
ENROLLMENT	enrollment_id student_id class_id enrolled_at	serial uuid int time-stamptz	STUDENT CLASS	Unique ID (PK) FK to STUDENT FK to CLASS Timestamp of enrollment
DOCUMENT ATION	doc_id course_id file_name file_type	uuid uuid text text	CLASS	Unique ID (PK) FK to CLASS Name of the uploaded file Type of file (PDF, DOCX, etc.)
NOTES MATERIAL	id course_id	serial integer	CLASS	Unique ID (PK) FK to CLASS

Entity	Attribute	Data Type	Relation To	Description
	metadata	jsonb		File metadata (size, page count, etc.)
	embedding	vector		Vector embeddings for AI search
	content	text		content of the summarized lecture
ANNOUNCEMENT	announcement_id	uuid		Unique ID (PK)
	course_id	uuid	CLASS	FK to CLASS
	class_id	uuid	CLASS	FK to CLASS
	teacher_id	uuid	TEACHER	FK to TEACHER
	content	text		Announcement content
	posted_at	time-stamptz		Posting timestamp
CLASS	course_id	uuid		Unique Course ID (PK)
	class_id	uuid	CLASS	Unique CLASS id (Unique key)
	teacher_id	uuid	TEACHER	FK to TEACHER
	course_name	text		Full name of the course
	course_code	text		Full Code for the specific class section
	class_name	text		Name of the specific class section
	class_code	text		Code for the specific class section
ASSIGNMENT	assignment_id	uuid		Unique assignment ID (PK)
	created_at	time-stamptz		Record creation timestamp
	title	text		Title of the uploaded assignment
	instructions	text		instructions of the uploaded assignment
	grades	numeric		Grade of the uploaded assignment
	topic	text		Topic of the uploaded assignment

Entity	Attribute	Data Type	Relation To	Description
	due_date	date		Deadline of the uploaded assignment
ASSIGNMENT SUBMISSION	submission_id	uuid		Unique submission ID (PK)
	created_at	time-stamp		Record creation timestamp
	submitted_at	date		Date at which student uploaded assignment
	grade_obtained	numeric		Grade of the uploaded assignment

4.10 Risk Analysis

4.10.1 Technical Risks

- **AI Performance and Accuracy:** Possibility that accuracy especially when code switching and semantic alignment between speech-to-text and slides is not sufficiently accurate enough to create usable notes. The accuracy will diminish the core functionality of the application. To address this risk, it is important to carry out strict A/B testing against human baseline performance and use state-of-the-art language models and incorporate user feedback into the ongoing fine-tuning process of the models.
- **LLM/RAG Integration Challenge:** Incorporating the RAG model in conjunction with vast amounts of unstructured textbook information might turn out to be much more difficult, resulting in delays and poor quality note improvements. As such, begin with a very structured data set with a smaller amount of information in order to develop the RAG model proof of concept.
- **Resource Management and Cost:** Running several concurrent language model sessions to generate notes requires significant computing power. It may pose operational risks and may incur extremely expensive operational costs. To mitigate this risk, it is important to adopt an asynchronous queue mechanism for generating the notes using cloud based services to enable monitoring and scalability of the AI model pipeline.

4.10.2 Content and Quality Risks

Accuracy of notes generated by the AI models will remain intact as Cassandra continues to improve its AI models and provide a mechanism through which teachers can feedback corrections and improvements in the generated content.

4.10.3 User Adoption and Engagement Risks

User engagement and ease of onboarding are vital for Cassandra to thrive. In this regard, Cassandra will ensure continued user engagement by constantly improving its system through updating of the system, interactives in the notes/AI assistant, and an engaging user experience. User onboarding for both teachers and students will hence become seamless as Cassandra offers a simple user interface coupled with tutorials that will ensure quick adoption due to value offered right from the start through its AI capabilities.

4.10.4 Competition Risks

With new entrants offering similar functionalities of AI-based learning capabilities or replicating Cassandra's differentiating capabilities on existing LMS platforms, competition is likely to pose a challenge. Continuous innovation through enhancing AI capabilities such as multimodal input capability and addition of innovative intelligent features will be key. This will require agile product development based on continuous user feedback.

4.11 Conclusion

All the requirements that the Cassandra Platform must fulfill have been detailed out in Chapter 4. It has provided the required functionality specifications for users playing the role of Teacher and Student. More importantly, all the functionality specifications of the highly complex automated lecture processing system that comprises the value proposition of the solution have been defined. With the Quality Attributes, Non Functional Requirements, Assumptions, and Use Cases having been clearly identified in light of potential risks involved, a sound foundation has been laid for the subsequent design and development stages of the project.

Chapter 5 Proposed Approach and Methodology

This chapter presents the multistage technical pipeline created for Cassandra education platform. For the first time, this method will allow to automatically combine different types of lecture materials, such as slides, whiteboard, and audio, into coherent learning notes. Thus, Cassandra becomes able to capture all nuances of the lecture, thus removing the necessity for students to create notes and offering them comprehensive learning material.

5.1 Multimodal Data Acquisition Framework

Such an application could enable the lecturer to easily record lectures, capture slide contents, whiteboard explanations, and a variety of other course materials and, in the process, automatically generate structured notes for students. At the heart of the Cassandra platform is a robust and flexible data acquisition framework.

This system follows various teaching styles, from traditional slide-based lectures to whiteboard discussions and audio explanations. It is designed to accommodate all these and more, capturing valuable instructional content irrespective of the delivery method favored by a teacher.

5.2 Audio Recording

Since there are multiple options for generating lecture notes, the primary requirement is capturing the teacher's voice during the lecture. The teacher may choose to:

- Only include a lecture agenda or outline that shows what topics will be discussed.
- Use PowerPoint slides (.pptx) during the lecture.
- Deliver the lecture using a classroom whiteboard only.
- Combine both slides and a whiteboard.

All these modes of delivery are handled flexibly by the Cassandra framework.

5.2.1 Teacher Voice Signature Enrollment

The first stage for making Cassandra experience more personalized is creating a unique voice signature for each individual teacher. This signature makes possible for the system to distinguish the teacher's voice from background conversation or answers from students when transcribing the lecture.

When a teacher sets up an account, the system requires them to record a brief reading sample. In the

background, the system records live audio with the pyaudio library, processes it in real time, and stores a clean version of the recording.

Noise cancellation will be automatically done using the NoiseReduce library, removing background noise, but preserving the purity of the human voice. The cleansed audio file will be used as a reference voice signature of the teacher, which will be further utilized for recognizing and segmenting the teacher's speech within lectures conducted by multiple speakers.

5.2.2 Lecture Audio Recording

In case the teacher provides the lecture outline, then all the lecture audio would be generated as one whole audio file. However, in case slides were provided, Cassandra would explicitly link between explanation with visuals through generating an audio file that captures the lecture as it is being played alongside every single slide. This will ensure that every piece of information on each of the slides will have its relating speech narration which can then be utilized to relate the visuals with the verbal.

Once recording starts, Cassandra keeps recording audio in the background with a pyaudio InputStream, buffering sound chunks into memory. When the teacher moves through slides utilizing Next and Previous buttons the system automatically stores or adds the respective audio segments to slide-specific files.

This organization enables the instructor to transition between slides smoothly, re-emphasizing or re-explaining points without breaking the narrative flow. When revisiting a slide, the system will insert the new explanation instead of replacing the original one.

For whiteboard only lectures, Cassandra makes explicit relationship between explanation that is spoken and content from the whiteboard images by capturing audio of a lecture synchronized with each whiteboard image. When slides are detected along with a whiteboard, Cassandra resorts to per slide audio segmentation to ensure high level synchronization between visuals and audio.

This modular and automated approach ensures that the audio context is preserved for later transcription and analysis.

5.2.3 Lecture Audio Transcription

Having acquired the full lecture audio, per-slide audio, or per-whiteboard image audio transcription follows. Speech-to-text conversion and translation of mixed Urdu English speech to full English sentences are accomplished with the Faster Whisper model.

To ensure purity, Resemblyzer is used to isolate only the teacher voice segments, excluding other speak-

ers. In doing so, Cassandra gathers high-quality audio data and produces clean, well-structured text that forms the basis of the automated note generation process.

5.3 Slide Content Decomposition and Extraction Pipeline

Decomposing presentation files into smaller, computationally readable pieces is an important step towards converting them to structured and analyzable material. This process is aimed at taking a .pptx presentation and breaking it down into an in-depth description of its constituent pieces like text, images, tables, and equations and marking each with metadata for where it was placed, what it is, and what it contains.

5.3.1 Preliminary Document Conversion and Preprocessing

The process begins by exporting the uploaded .pptx file to a .pdf format so that visual rendering is consistent across the slides. The PyMuPDF (fitz) library is then employed to export each PDF page to an enhanced PNG image. Concurrently, the initial .pptx file is broken down to gather its internal structure. The system accesses the internal XML structure (using the python-pptx library) to obtain the coordinate data about the elements (such as text, images, tables, and equations) on the respective slides.

5.3.2 Complete PPTX Decomposition

After preprocessing is finished, the software processes each slide from the XML file to identify and classify all embedded items such as text boxes, images, equations, and tables.

Text material is directly pulled out, whereas pictures are pulled from the .pptx archive in their native quality.

Yet, equations and tables cannot be pulled directly from the .pptx file structure. Thus, their locations on each slide are determined through coordinate data. From these coordinates, bounding boxes are established for every detected element on the respective PNG slide image. The areas are cropped and saved independently as screenshots.

Equation screenshots and table screenshots are saved in independent folders for subsequent processing.

5.3.3 Mathematical Equations Extraction

Mathematical equations need a more advanced extraction method. The equation images stored are sent to the fine-tuned Qwen3-VL-4B-Instruct model, which recognizes visual mathematical content and translates it into LaTeX code.

Some equations could also be directly embedded as image files in the .pptx media folder. To detect such instances, a fine-tuned Vision Transformer model, `google/vit-base-patch16-224` is employed to predict whether an image has a mathematical equation or not.

5.3.4 Table Extraction

The extraction of tables is processed using the same fine-tuned Qwen3-VL-4B-Instruct model. The Qwen3-VL-4B-Instruct model is applied directly on each table image to extract the content and convert it into accurate LaTeX format.

Just like equations, some tables could also be directly embedded as image files in the .pptx media folder. To detect such instances, a fine-tuned Vision Transformer model, `google/vit-base-patch16-224` is employed to predict whether an image has a table or not.

For the remaining media images, we are using the pre-trained Salesforce/blip2-flan-t5-xl model to generate captions that help in understanding the description of the images.

5.4 Whiteboard Content Decomposition and Extraction Pipeline

This section describes the methodology used to convert classroom whiteboard snapshots into refined, machine readable lecture notes. The pipeline follows a teacher initiated snapshot workflow: the instructor takes a photograph of the whiteboard and uploads it through a frontend interface. The backend then preprocesses the image and invokes a vision enabled LLM (Qwen3-VL-4B-Instruct model) using a strict OCR prompt. The model returns a structured JSON representation, which is post processed to produce final digital notes.

5.4.1 Snapshot Acquisition

The instructor captures a whiteboard region screenshot via a web frontend, which is then uploaded to the backend. The frontend ensures that the submitted image contains the region of interest intended without requiring a local canvas or drawing surface. Such uploaded images are temporarily stored and passed to the preprocessing pipeline to perform normalization and reduction of noise before inference.

5.4.2 Image Preprocessing

The lightweight preprocessing at the backend for image quality normalization and makes the OCR application more robust. Common operations include:

- Convert to grayscale and compute standard deviation (contrast check).

- Skip processing if image is already high-contrast ($\text{std} > 75$).
- Apply detail enhancement for low-contrast images.
- Adjust contrast and brightness using scaling.

Preprocessing is intentionally minimal so as not to alter the semantics of the content. This is purposed to increase the visibility for a model while preserving the original text, spacing, and layout.

5.4.3 Model Loading and Inference

The inference step loads the vision model pipeline and passes the preprocessed image along with the structured OCR instruction. The production code follows these primary steps:

- The processor is initialized using `AutoProcessor` to prepare the image and prompt for the model.
- The vision enabled LLM `Qwen3-VL-4B-Instruct` is loaded in evaluation mode for OCR inference.
- A structured OCR prompt is made, asking the model to extract content exactly as shown and return a single JSON format file for all the input images.
- The image and prompt are packaged into model's chat using `processor.apply_chat_template`.
- The model generates the OCR output, then decoded into readable text using `processor.decode`.

The prompt explicitly instructs the model to detect content type (text, table, equation) preserve characters/spacing/line breaks/punctuation/case, and produce a JSON object with bounding boxes and reading order.

5.4.4 Structured JSON Output Schema

All model outputs are generated in a strictly defined JSON format, wrapped into a single root object called `structured_elements`. Every member of this list carries a few required properties that carry both semantic meaning and appearance. The `original_id` property is a unique identifier that allows consistent mapping between OCR detected regions and downstream components. The `content_type` attribute delineates the logical category of the extracted region, such as text, equation, or table. The `content` property retains the exact transcription of the extracted material by preserving spacing, punctuation, line breaks, and casing in order to preserve fidelity with the source document. Visual ordering is handled through the `reading_order` integer, which follows a top-to-bottom, left-to-right sequence.

Additional contextual information is stored in the `metadata` block, which includes `image_path` (or

null for non visual items), the bounding box coordinates given as $[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$, and also the `raw_type` field that reflects the preliminary OCR detected classification. For regions that contain only tabular data, the model outputs a clean LaTeX formatted table inside the `content` field, whereas mathematical regions store the unaltered equation text.

5.5 AI-Driven Content Structuring and Semantic Integration

This phase is the overall synthesis hub of the methodology. The main objective is to convert the raw, unconnected data elements slides content, whiteboard content and audio-transcribed into one, logically organized, and semantically enriched document. This thus enables preparedness of the content for the last phase of note generation.

5.5.1 Logical Content Structuring

The task of the LLM is to optimize the decomposed slide and/or whiteboard material. After some trial and error with other models, DeepSeek-chat was chosen as the final architecture because of its firm balance between performance and efficiency.

The model processes every extracted piece and performs a number of fundamental tasks:

- **Classification:** Every piece of content is classified into a fine grained logical type such as heading, subheading, bullet_list, code_block, caption, definition, or key_point.
- **Sequencing:** It arranges objects into a sensible reading order that reflects the logical order of the original slide or board.

5.5.2 Semantic Audio-to-Slide Alignment

In the case of a slide-based presentation of a lecture, Cassandra synchronizes the teacher's verbal explanation with the respective slide content using a semantic matching procedure facilitated by the SentenceTransformer (all-mpnet-base-v2) model. The procedure works dynamically and iteratively as follows:

1. **Embedding Generation:** Both the audio sentences transcribed and the text contents pulled out from slides are also turned into numerical vector embeddings. To accurately process mathematical content, the system does not translate equations into plain human readable text. Instead, equations are extracted directly as structured LaTeX code. This raw LaTeX is embedded alongside the surrounding text for semantic alignment and is strictly preserved in its mathematical format during the generation of the final Markdown lecture notes.

2. **Similarity Matching:** For every vector of the audio sentence, calculate the cosine similarity to the vectors of every element in the slide to identify the most semantically matching pair.
3. **Dynamic Content Enrichment:** Once there is a matching done, then the textual content that is related to the matching slide will be added to the matched audio sentence. Next, embeddings are calculated for this new content and it is also utilized for other matches to create knowledge that builds up gradually from audio.

5.5.3 Semantic Audio-to-Whiteboard Alignment

In whiteboard content lectures, semantic alignment is performed using the same process. The whiteboard text and the audio text transcription are embedded, and cosine similarities are computed to spot and combine semantically similar sections. This makes sure that proper alignment of the uttered explanations takes place with the corresponding parts of the whiteboard content.

5.5.4 Semantic Audio-to-Combined Slides and Whiteboard Alignment

In hybrid lecture settings where slides and whiteboard material are used, the two sources of content are combined into a single unified structured dataset. Then the semantic alignment operation is performed in order to link the transcribed audio with this unified presentation of the content so that explanation is contextually linked with slide and whiteboard material.

5.6 Knowledge Augmentation and Final Note Generation

This last phase focuses on converting the integrated content into an educational high-value resource for students and an effective support tool for instructors. It raises the organized text to a summarized and interactive knowledge base.

5.6.1 RAG-Based Content Explanation

An explanation system has been constructed based on a Retrieval Augmented Generation architecture. This is to provide more explanation of the present content we possess.

- **Technical Implementation:** In the case of a teacher uploading PDF or DOCX textbooks, the system splits the text into chunks, embeds those chunks into vectors using SentenceTransformer, and stores them in a vector database using supabase for a scalable cloud deployment.
- **Query Process:** From the vector database, it retrieves a set of the most relevant $\text{top}_k = 3$ text chunks for the given feeds of content with the similarity threshold = 0.3. These are combined

with the content and fed as context to the DeepSeek-chat model. The LLM produces a detailed explanation that is typically replete with examples and analogies meant to help illustrate the concept.

This framework will also be used by the AI assistant chatbot to answer questions by users and describe complex topics from the created notes, grounded on external, authoritative materials such as textbooks.

5.6.2 Summarized Note Generation

The outcome of the Cassandra framework is the generation of comprehensive study notes. Here, the DeepSeek deepseek-chat model is applied to generate the study notes based on the enriched contents generated.

5.7 Conclusion

This approach described above is an example of a full-fledged workflow for automatic generation of educational content. This is because it utilizes state-of-the-art technology solutions at each stage of the process, beginning with data extraction using the Qwen3-VL-4B-Instruct model and transcription with the help of Faster-Whisper, then moving on to semantic alignment with the help of SentenceTransformer and finally generation of the notes with the DeepSeek-chat model. Therefore, the workflow guarantees its scalability and robustness. The whole concept solves the problem of generating high-quality automatic lecture notes with efficiency.

Chapter 6 High-Level and Low-Level Design

This chapter describes the overall architecture of the Cassandra Learning Platform. It gives an elaborate account of the architecture of the pipeline for this platform, the design principles behind it, and the technical details of the individual sub-systems. This architectural description serves as one blueprint for building and testing the platform, connecting strategic objectives to practical designs.

6.1 System Overview

Cassandra is an online learning platform that is intended to provide automatic generation of structured lecture notes. The dynamic and multi modal nature of a live lecture session, which involves verbal or oral explanations, presentations slides, and handwritten whiteboard material, is transformed into structured lecture notes.

The core functionality offered is delivered via three central, interdependent modules:

- **Automated Note Generation:** This is the backbone of Cassandra. It records live lecture sessions and translates the lecture voice, intelligently merging the information with the content drawn from the slides, whiteboards, and other external text books through an AI pipeline.
- **Dual AI Assistant System:** Cassandra includes two different AI chatbots, which are both based on the DeepSeek-chat model. One of the assistants assists instructors by responding to questions based on material uploaded such as textbooks and PDFs. The second one aids students by giving explanations and responding to questions derived solely from the resultant lecture notes.
- **Complete Learning Management:** In addition to its note generation features, Cassandra is an all-around Learning Management System (LMS). It offers basic features for course creation, distribution and submission of assignments, user to user private messaging, and announcements to the whole course.

This introduction describes the multifaceted nature of the platform. Various design considerations that influenced the architecture of this system are discussed in subsequent sections.

6.2 Design Considerations

The design of Cassandra was not done in isolation but was influenced by a conscious examination of underlying assumptions, key constraints, and the overall objective. In this section, these guiding principles and development methodologies followed to make the end system resilient, manageable, and goal oriented are discussed.

6.2.1 Assumptions and Dependencies

The design of the system relies on a given software ecosystem of hardware, software, and user patterns.

- **Software and Libraries:**

- **Audio Processing:** The project relies on pyaudio for audio recording, NumPy for data processing, SoundFile for storing and noise reduce for real time noise reduction.
- **Transcription & Speaker Verification:** Core functionality is based on Faster Whisper for high speed speech to text translation and Resemblyzer for voice verification of the speaker.
- **Visual Content Decomposition:** It requires PyMuPDF to render PDFs, represented as fitz, and python-pptx for parsing the structure of presentation files.
- **Deep Learning Components:** For classifying whether an image has an equation or a table we are using the google/vit-base-patch16-224, and for extracting the tables and equations we are using the Qwen3-VL-4B-Instruct model. Captioning of the images is done through Salesforce/blip2-flan-t5-xl.
- **Semantic Processing & Generation:** SentenceTransformer (all-mpnet-base-v2) is relied upon for semantic integration, and content structuring and generation rely on the Deep Seek-chat Large Language Model.
- **AI/ML Frameworks:** PyTorch and the Hugging Face Transformers library are the frameworks that the system has been developed on.
- **Database and Backend:** The Retrieval Augmented Generation (RAG) system utilizes Supabase as the scalable cloud ready vector database implementation.

- **Operating Systems and Hardware:**

- It is predicted that the production environment is capable of supporting computationally expensive deep learning models, which need CUDA enabled GPU hardware to enable proper performance by models such as DeepSeek and Faster Whisper.
- Additional hardware such as a microphone for lecture audio recording, a mobile phone for capturing whiteboard images, and a projector for displaying slides is also required.

- **End-User Characteristics:**

- The design supposes that teachers will deliver a clean voice sample while enrolling the voice signature, which is very important in ensuring speaker isolation accurately.
- We expect users to properly capture whiteboard images to extract the content properly.

6.2.2 General Constraints

The design of the system is bounded by some central constraints that affect its architecture and working bounds.

- **Performance and Capacity:** The advanced AI pipeline imposes a massive computational burden, and hence the instantaneous generation of notes proves to be a challenge. Besides, high RAM/VRAM consumption by models like DeepSeek-chat demands careful management and hence makes cloud hosting a necessity.
- **Security and Verification:** The accuracy of the transcribed notes poses a crucial constraint for the system to adhere to. It will, thus, require speaker authentication using Resemblyzer to guarantee transcription exclusively from the voice of the teacher and beyond.
- **Interface and Platform:** The current framework confines its input source to the ingestion of .pptx files for presentations.
- **Recognition Limits:** At present, the model recognizes only handwriting in English language, including mathematical expressions and tabular format. The recognition does not include any other form of notation or language. In the case of slides, we are capable of extracting English language text, mathematics expressions, images, and tabular formats.

6.2.3 Goals and Guidelines

The design is driven by a primary set of principles, which aim at optimizing its value and usability.

- **Maximum Contextual Capture:** The design supports multiple ways of teaching: slides only, whiteboard only, audio only, slides + whiteboard lectures. Thus, it captures the entire context of instructions.
- **Precision in Synchronization:** It represents another important objective. It targets the achievement of precise synchronization of voice explanations with the visual content. This is done through capturing on a per-slide basis when the teacher uses slides only or slides with the whiteboard in order to achieve synchronization precision.
- **Modularity and Specialization:** The system manages complexity through multi-staged pipelining, which allows applying highly specialized AI techniques for particular tasks, such as equation identification or speaker verification.
- **Accuracy First:** It enables the maximum accuracy of recognizing all types of instructional data (voice, text, handwriting, equations), providing an exact representation of what was said and

written by the user.

- **Automated Knowledge Augmentation:** The system can automatically enhance notes with information from outside-of-the-textbook knowledge via the usage of the Retrieval Augmented Generation structure.

6.2.4 Development Methods

The technique used in the development of this software consists of a multistage technical architecture that organizes the entire process as an end-to-end pipeline.

The design employs Modularity and Specialization as the means for coping with the complexities created by the integration of different technologies such as Optical Character Recognition (OCR), text to speech (TTS), Large Language Models (LLMs), and RAG.

In development management, the project is driven by Agile development principles, using SCRUM.

6.3 System Architecture

The Cassandra platform is an end to end multi stage technical system that automatically converts multimodal lecture material to structured study notes. It begins with Multimodal Inputs by recording the instructor's audio speech (typically synchronized per slide), uploaded presentation slides (.pptx), whiteboard images, and external textbook information. Such raw inputs are input to Parallel Processing Pipelines where audio is transcribed and cleaned using Faster Whisper and Resemblyzer for speaker separation, and visual content is extracted through models such as Google vision transformer and Qwen3-VL-4B-Instruct. The AI Core then aggregates and polishes this output: the chat LLM performs logical Content Structuring & Correction, whereas Semantic Audio Alignment applies SentenceTransformer to dynamically align spoken explanations with visual content based on cosine similarity. Finally, Knowledge Augmentation is performed through a Retrieval Augmented Generation (RAG) architecture based on uploaded textbooks, which produces the Final Output of cleaned, summarized, and comprehensive study notes synthesized through a fine tuned LLM (shown in figure 6.1).

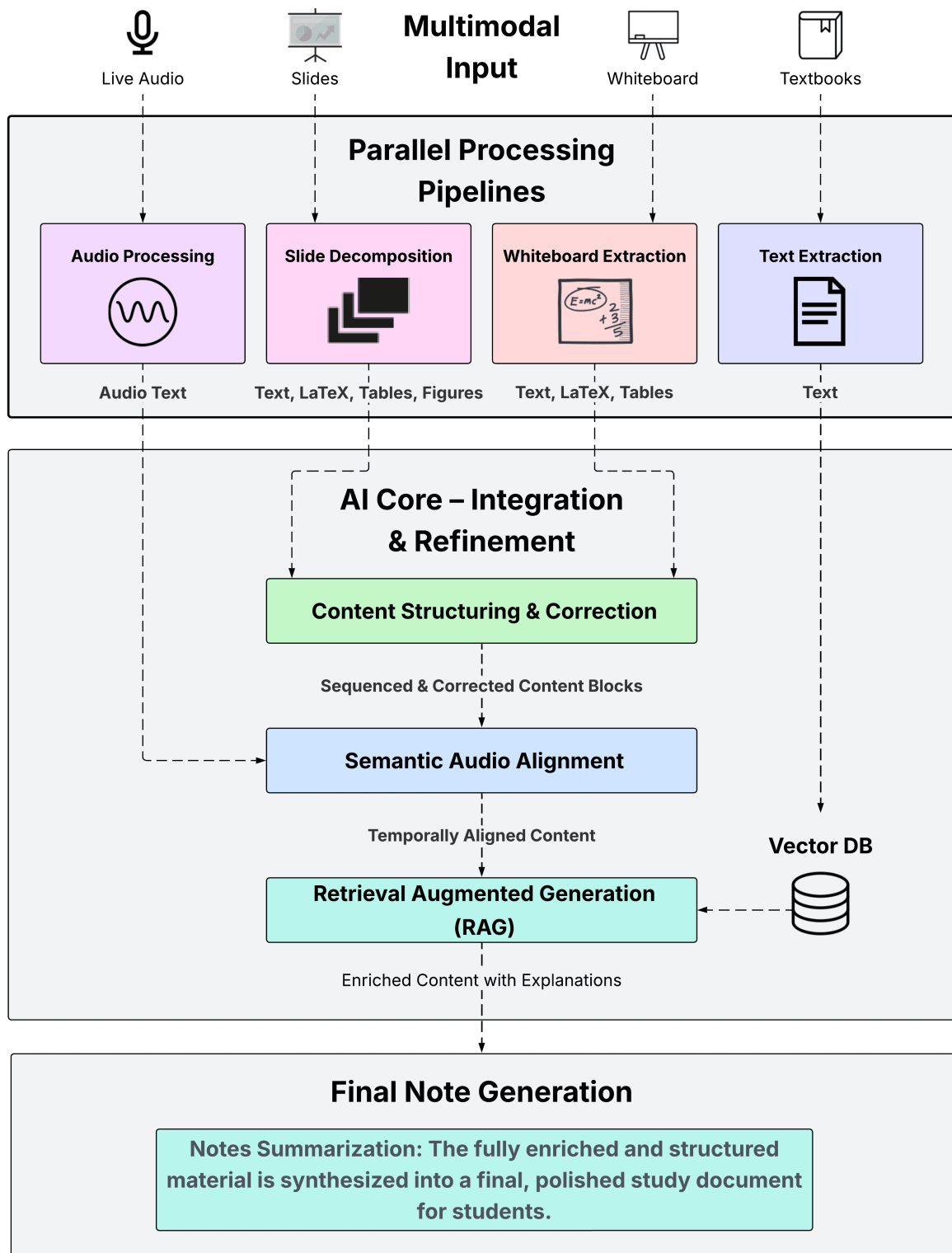


Figure 6.1: System Architecture of Cassandra

6.3.1 Subsystem Architecture

The total pipeline consists of four big interconnected subsystems. Each subsystem is designed to play a specific role in the raw lecture data to nicely structured notes transformation process. The descriptions below give an overview of the components and the data flow in each.

6.3.1.1 Multimodal Inputs

- **Audio (The teacher's spoken lecture recording):**

- The basic requirement is the recording of the teacher's voice throughout the lecture.
- Cassandra constantly records live audio with the help of the pyaudio library.
- Whenever the teacher is using slides, the system records lecture audio coupled with every slide (slide by slide audio segmentation), linking verbal explanations directly with visual content.
- Whenever the teacher is using only whiteboard, the system records the lecture audio alongside the writing process, and once the teacher captures and sends the whiteboard image to the application, the audio is paired with each corresponding whiteboard snapshot.

- **Slides (.pptx):**

- PowerPoint slides (.pptx) can be uploaded by teachers as material. The system accommodates these files, which are rendered into XML and PDF for standardization. Text and images are directly extracted from the XML. For tables and equations, the coordinate positions are obtained, and using those coordinates, the corresponding screenshots are extracted from the images generated from the PDF.

- **Whiteboard:**

- Handwritten notes, tables, and mathematics equations sketched during the lecture are captured using mobile phone.
- Captured image is then pass to the LLM based Qwen3-VL-4B-Instruct model which gives the digitized content in the JSON format.

- **Lecture Outline:**

- The system accommodates delivery modes where an instructor offers only a lecture agenda or outline. In this mode, the entire lecture is captured as one continuous audio file.

6.3.1.2 Parallel Processing Pipelines

After the inputs are gathered, Cassandra processes them in parallel in separate, specialized pipelines, an approach selected to contain complexity and support best effort efficiency.

- **Audio Processing:**

- The system applies the Faster Whisper model to convert speech to text. This model can translate mixed English-Urdu speech into full English sentences.
- For accuracy and integrity, Resemblyzer carries out speaker verification where only those segments that identify with the teacher's voice signature are extracted and filtered.
- This flow is demonstrated in figure 6.2. The descriptions below give an overview of the components and the data flow in each.

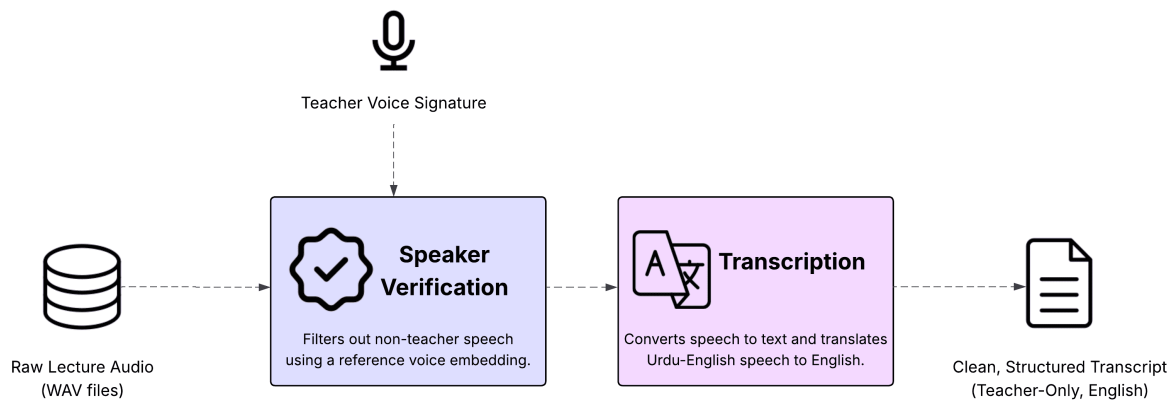


Figure 6.2: Voice Verification and Processing

- **Slide Decomposition:**

- This pipeline translates the .pptx file format into machine readable units. The whole decomposition is shown in figure 6.3.
- The file is translated to a .pdf and then rendered into high quality PNG images with the PyMuPDF (fitz) library.
- The internal XML structure is accessed using the python-pptx library so that the system can identify the original position (coordinate data) of complex elements such as text, images, tables, and equations.
- Specialized extraction is carried out on complex elements:
 - * Mathematical Equations (cropped as images) are fed to the Qwen3-VL-4B-Instruct model to make visual math editable LaTeX code.

- * Table content is extracted by employing Qwen3-VL-4B-Instruct model to convert into LaTeX.
- * Sometimes the tables and equations are used as images in the slides, so they are stored as images in the media section. Therefore, we will first classify them using fine-tuned Google ViT to check whether an image is an equation, a table, or neither. After identifying a table or equation, we will apply the same Qwen3-VL-4B-Instruct model.
- * For the remaining media images that were classified as neither, we will use Salesforce/blip2-flan-t5-xl to generate descriptions, which will be used as the captions of the images.

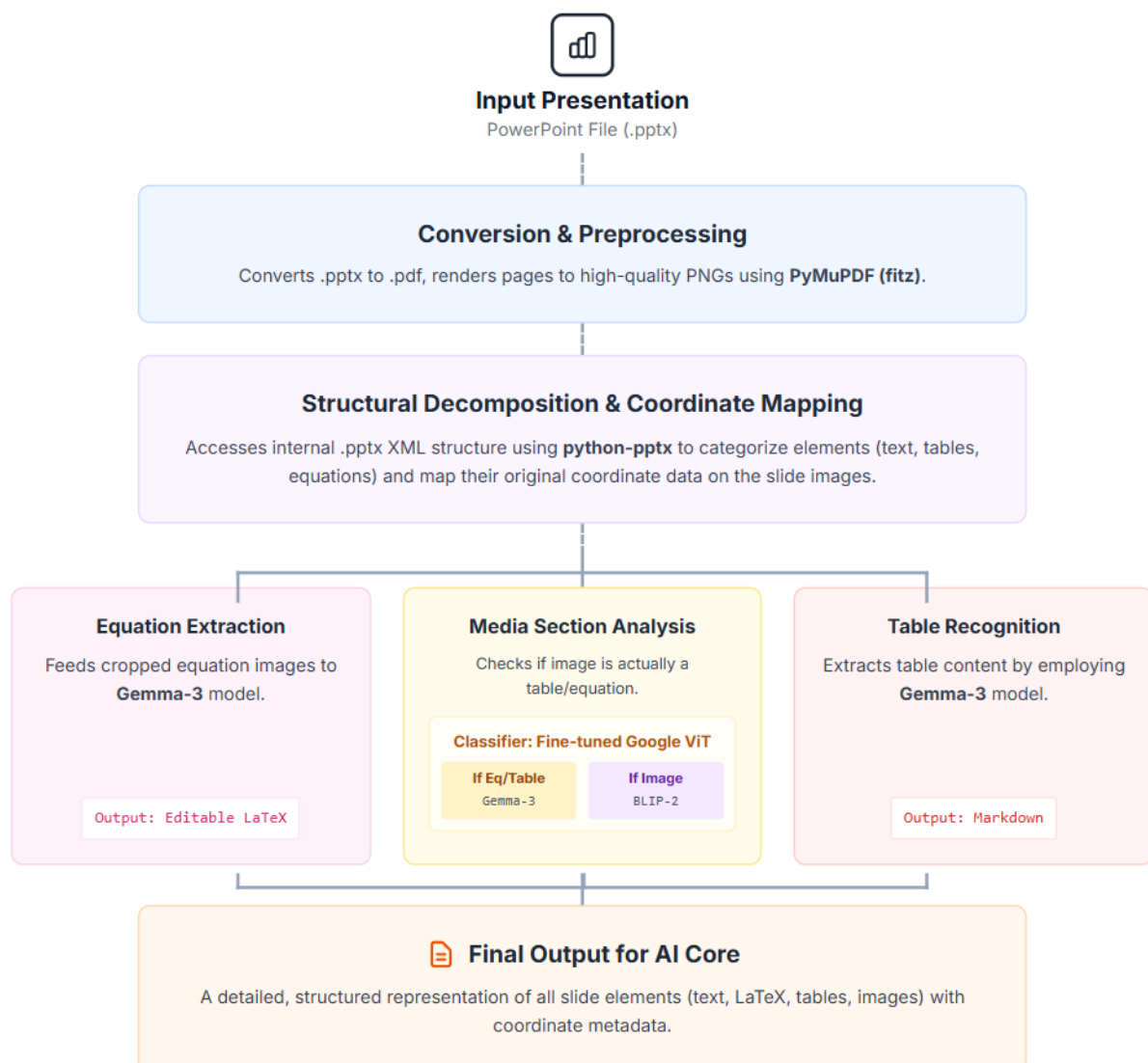


Figure 6.3: Slides Decomposition steps

- **Whiteboard Extraction:**

- The whiteboard extraction pipeline transforms raw handwritten content into structured, machine readable information. The complete processing flow is captured in Figure 6.4.
- Firstly, the captured whiteboard snapshot is preprocessed in a lightweight manner: grayscale conversion, contrast checker, enhancement and adjustment of the image in order to make the input for the model uniform.
- After preprocessing, the cleaned image will directly be forwarded to the Qwen3-VL-4B-Instruct model.
- It automatically segments the contents on the whiteboard into their logical regions of text, equations, figures, and mixed blocks, driven by inbuilt visual reasoning and token alignment.
- The model generates a single structured JSON output against all the whiteboard images with the following fields: the exact transcription, its logical type, the spatial bounding box, and the reading order index.

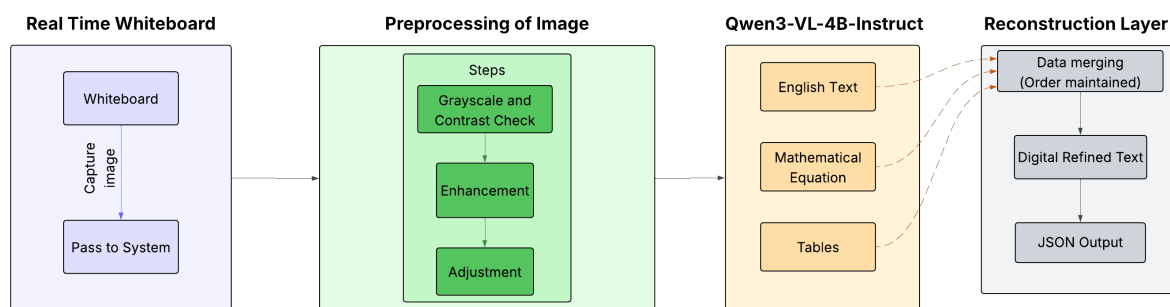


Figure 6.4: Whiteboard Text Extraction Pipeline

6.3.1.3 AI Core – Integration & Refinement

This is the center of intelligence for the system, which aggregates and enriches all content.

- **Content Structuring & Correction:**

- A Large Language Model (LLM), namely DeepSeek-chat, is tasked with fine tuning the content as shown in figure 6.5.
- The LLM does Classification, classifying each piece into fine grained categories like main heading, subheading, bullet list, or definition.
- It does Correction by correcting spelling and grammar.
- It processes sequencing, rearranging items into a logical reading order that reflects the lecture's chronological order.

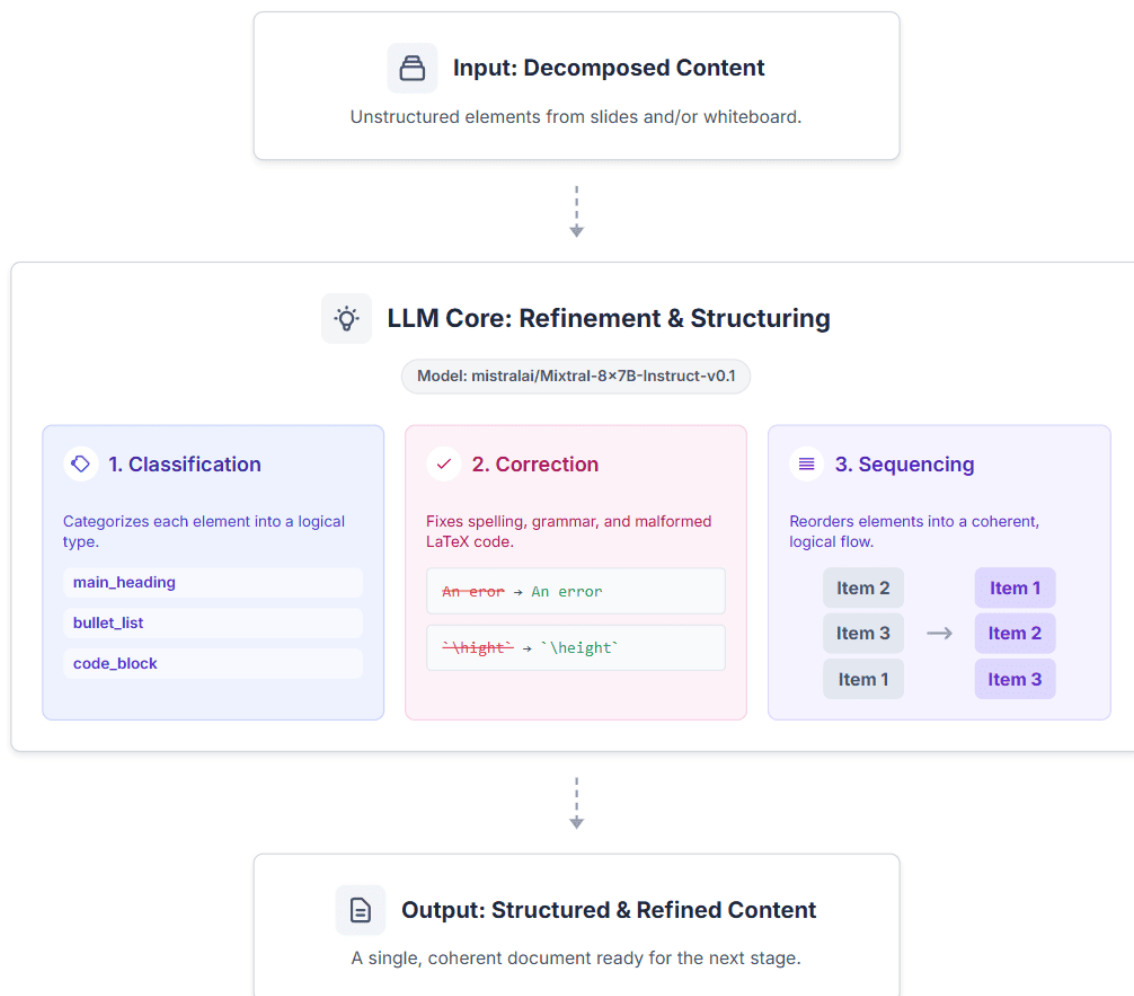


Figure 6.5: Content Refinement Step through AI Integration

- **Semantic Audio Alignment:**

- It adds transcribed explanation to visual content through semantic matching on the Sentence-Transformer (all-mpnet-base-v2) model as represented by figure 6.6.
- Audio sentences and visual content elements are both embedded into numerical vector embeddings.
- Cosine similarity is used to determine the most semantically similar match between speech text and visual elements.
- A Dynamic Content Enrichment process is essential: after a match has been identified, the sound is added to the text of the visual element, and the embedding of the element is calculated a new for subsequent matches to support iterative contextual sense making.

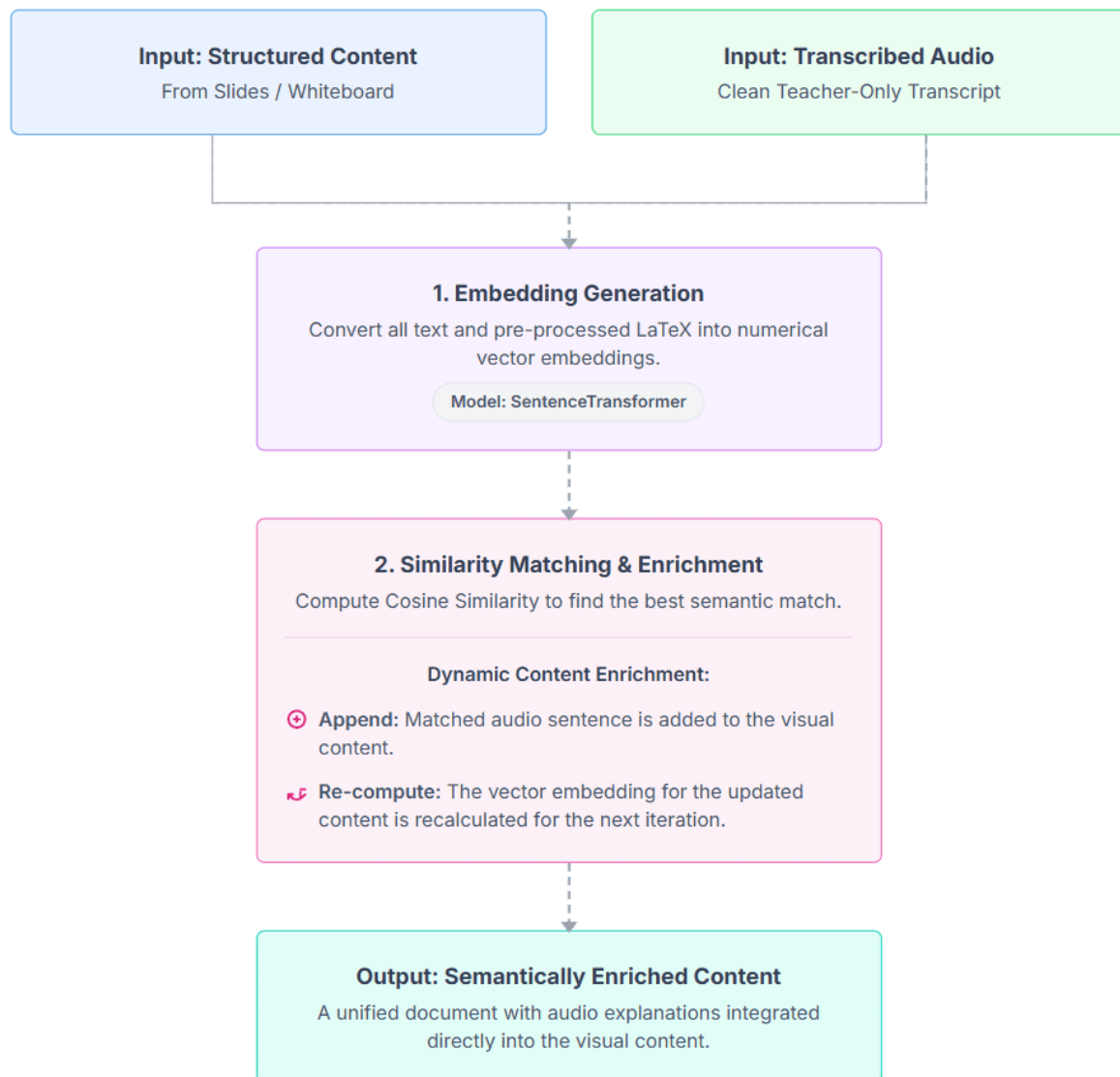


Figure 6.6: Semantic Audio Alignment Process

- **Knowledge Augmentation:**

- An explanation system is performed utilizing a Retrieval Augmented Generation (RAG) system for augmenting the notes. This can be seen shown by figure 6.7.
- The system chunks and stores uploaded PDF or DOCX textbooks via SentenceTransformer and saves them in a vector database (Supabase).
- Relevant top 3 text chunks are fetched and fed as context to the DeepSeek-chat LLM, which produces detailed explanations, analogies, and examples to explain concepts.

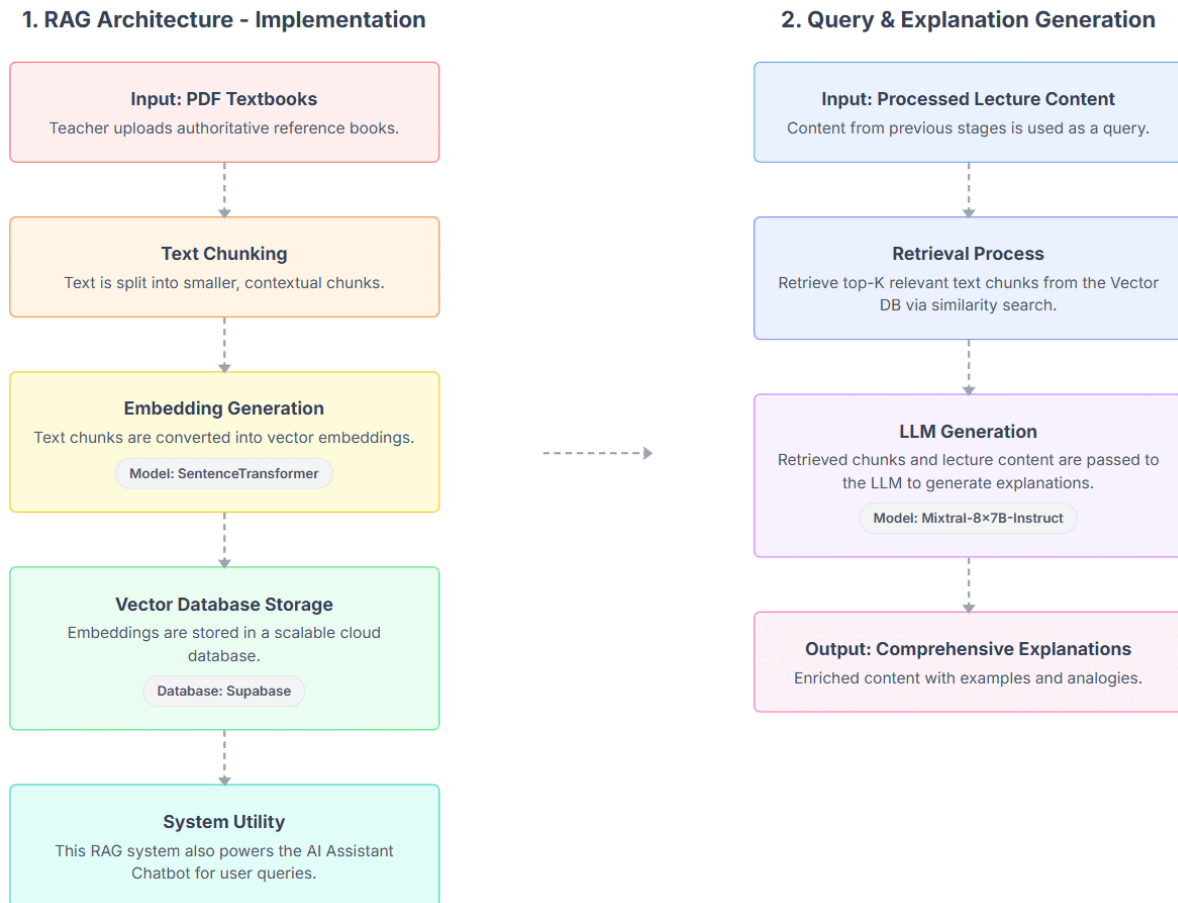


Figure 6.7: Knowledge Augmentation Process

6.3.1.4 Final Output

Lastly, the system produces the final product by combining the enriched content.

- A neat, summarized, and well structured collection of study notes for students.
 - The end output is a collection of well structured and summarized study notes.
 - The end document is produced by a model of the DeepSeek-chat.
 - The lecture speech, slides, and board work are blended into the notes polished and enriched into a readable form.

6.4 Architectural Strategies

The architecture of the system is supported by a number of important strategies that are aimed at solving the project's central challenges in terms of complexity, performance, and maintainability. These strategies are the underlying design philosophies behind the creation of the Cassandra platform.

6.4.1 Strategy 1: Pipeline and Modular Specialization

The use of a multi stage pipeline framework supports the decoupling of compound processes into discrete, manageable modules. Modularity supports the ability to choose highly specialized, best in class tools for specific tasks. This approach optimizes the quality and reliability of every processing step.

6.4.2 Strategy 2: Hardware Aware AI Deployment and Concurrency

This architecture takes care of any issues arising from slow processing of computations through computationally intensive AI models. Multithreading has been incorporated in the audio capture module in a way that enables the capturing process to be done in parallel with the application's main process. The computation intensive model processes have been strategically deployed, with Faster Whisper running on the GPU and Resemblyzer on the CPU.

6.4.3 Strategy 3: Semantic Enhancement and Dynamic Integration

Enhancement in Semantics and Dynamic Integration The system employs vector embeddings created by SentenceTransformer. These vector embeddings enable computing cosine similarity to associate the oral descriptions effectively with the visual contents. The method retains semantics rather than comparing keywords. One vital component of this method is Dynamic Content Enrichment. Through the continuous addition of the matched audio to the content element and re-computation of its embedding immediately after, the system builds up the context in a dynamic way, improving the accuracy of future matches.

6.4.4 Strategy 4: Reuse of Pre-trained Models and External APIs

One of the central architectural principles is the systematic reuse of large, general-purpose models for many independent tasks. This strategy can be seen in various areas:

- The DeepSeek-chat model is recycled for several different tasks, such as content organization, RAG based explanations, and the two AI assistant chatbots.
- Robust open source libraries such as OpenCV are employed for basic image processing operations such as region detection and separation.
- Pre-trained Hugging Face models such as Qwen3-VL-4B-Instruct model are fine-tuned, and there is no need to train complicated recognition models from scratch.

6.5 Domain Model / Class Diagram

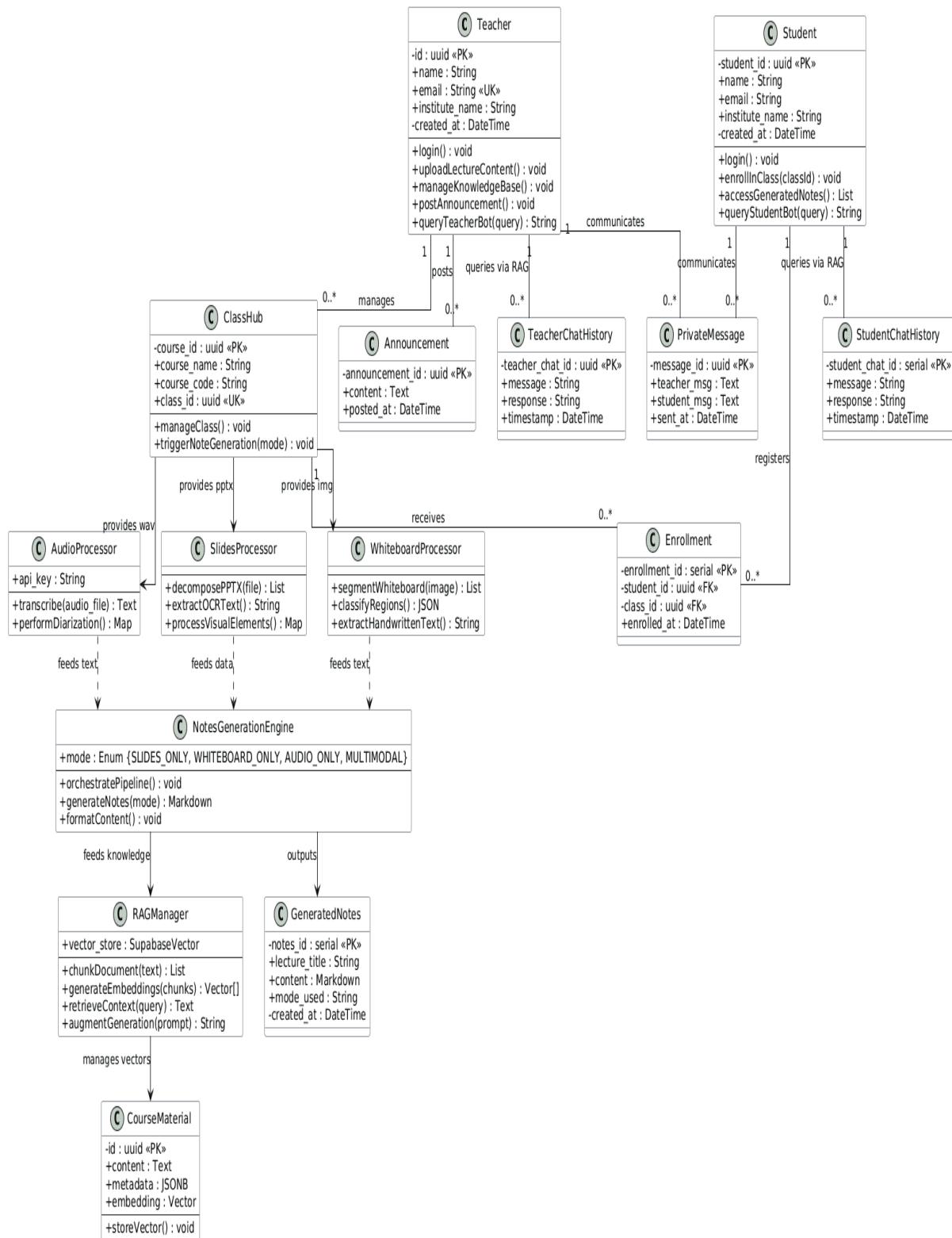


Figure 6.8: Class Diagram of Cassandra

6.6 Policies and Tactics

This last part describes the particular, tactical choices and policies for the execution of Cassandra. These policies include selection of tools, coding standards, UI, exceptions, and testing, to maintain a consistent and high quality development throughout the entire project.

6.6.1 Tools and Libraries

The project uses an expert tested set of strong tools and libraries to execute its advanced functionalities.

- AI/ML Frameworks: PyTorch, Transformers (Hugging Face).
- Computer Vision & Image Processing: OpenCV (cv2), PIL/Pillow, PyMuPDF (fitz).
- Audio Processing: pyaudio, soundfile, noisereduce.
- AI Models & Services: Faster Whisper, Resemblyzer, Qwen3-VL-4B-Instruct, SentenceTransformer, google/vit-base-patch16-224-in21k, mSalesforce/blip2-flan-t5-xl, Deep Seek-chat.
- Backend & Data: Python and Flask, Supabase (Vector DB).
- Deployment: Backend is deployed on Azure (CPU tasks) and Modal (GPU tasks), while the frontend is deployed using Netlify.
- GUI & Presentation: python-pptx.
- Frontend Development: HTML, CSS, and JavaScript.

6.6.2 Coding Guidelines and Conventions

The main development language is Python. Development, especially for AI, is also frequently done in Jupyter Notebooks to support quick iterative testing and visualization. The project is developed according to standard conventions, for example, the use of clear variable names and modular function structure to support readability and future maintenance.

6.6.3 User Interface Design

The application design ensures ease-of-use and simplicity for the users. To enhance convenience for students and teachers, the website application provides dedicated dashboards for each role separately.

6.6.4 Error Handling and Recovery

Essential actions, such as API requests from other sources, for instance, Supabase API for data storage, DeepSeek API for the large language model, convertapi for converting PPTX to PDF to perform slides decomposition tasks, loading of models, and file processing, are enclosed in try except blocks to catch possible error points. Errors are logged on the console with accompanying messages.

6.6.5 Model Management and Performance

To ensure peak efficiency, the AI models are initialized only once when the application starts, after which they remain in memory to prevent the need to initialize them again upon consecutive runs. Where available, automatic usage of GPU speedups occurs otherwise, the system gracefully falls back to CPU mode in case no appropriate GPU is found. The system also utilizes the ultra-fast C++ library of OpenCV for image recognition operations.

6.6.6 System Testing Approach

It is a project that uses a multi-faceted testing approach, including:

- Unit tests for each component these can be either line segmentation algorithms or OCR accuracy for specific characters.
- Integration tests to check if data flows from one processing step to another, e.g., from segmentation to OCR.
- User acceptance tests with handwriting data samples from various users.

6.6.7 Software Maintenance

The project is developed with extensibility and ease of maintenance in mind. All model versions along with Hugging Face ids are stored to allow for easy updates of the model. Library and codebase dependencies are also recorded and versioned. The system is highly modular, which allows for easy implementation of future improvements in areas such as support for multiple languages and adding diagram recognition functionality without re-developing all the other parts of the system.

6.7 Conclusion

This chapter has covered the in depth architecture design of the Cassandra architecture from the high level pipeline approach to the modular architecture design of subsystems like the audio preprocessing approach that are implemented based on several policies. The end to end design ensures that all these

modules will be used in synergy towards the achievement of an overarching goal of creating an intelligent and automatic class companion which makes the live learning lectures become immortal sources of education.

Chapter 7 Implementation and Test Cases

The following implementation chapter covers in detail the process involved in the realization of the Cassandra architecture system. The design, algorithms and architecture decision making in all the 16 different modules are covered to ensure that the final design achieves three critical objectives: high reliability, evidence based design and efficient performance of complex tasks involving synchronized media and video analysis. All the modules are designed using robust infrastructure ranging from data storage solutions like Supabase as well as vector indexing services to optimizing the model performance with Pytorch libraries and models like Faster Whisper and Resemblyzer, on whichever GPU/CPU devices are available.

7.1 Implementation

As far as the implementation of our project is concerned, we have made great efforts to develop and analyze different types of tools and utilities which can be used in our application's further development and implementation. To implement our Cassandra database system efficiently and effectively, we have researched and analyzed some relevant techniques and APIs. Details regarding our current implementation process are provided below:

7.1.1 Teacher Authentication and Voice Signature Enrollment

The Teacher Authentication and Voice Signature Enrollment module provides a gateway into the Cassandra system. This module was created to enable seamless integration between general user management features and the special biometric function of voice profiling. The frontend code of the module is written in HTML, CSS, and JavaScript.

Supabase, which is a cloud-based backend as a service, is used to provide the backend services related to authentication. After submitting the registration form by the teachers, the registration request sent by our application undergoes some client-side form validation process to check whether the password meets the required standards of security and whether the password confirms itself. Successful validation of the password leads to the signing up request being sent asynchronously through the Supabase JavaScript client API to register the user securely. Alongside this, the application runs a second database transaction, which inserts the full name and email of the teacher into a publicly accessible TEACHER table, binding this unique authentication ID to a particular instructor profile in the application.

Immediately after the successful registration, the system launches the Voice Signature Enrollment phase. Contrasting with regular platforms where profile completion is optional, this system forces a modal

window that does not allow entering the dashboard without providing a voice sample. This modal shows a script specially chosen as depicted in Figure 7.1 intended to record a range of phonetic sounds.

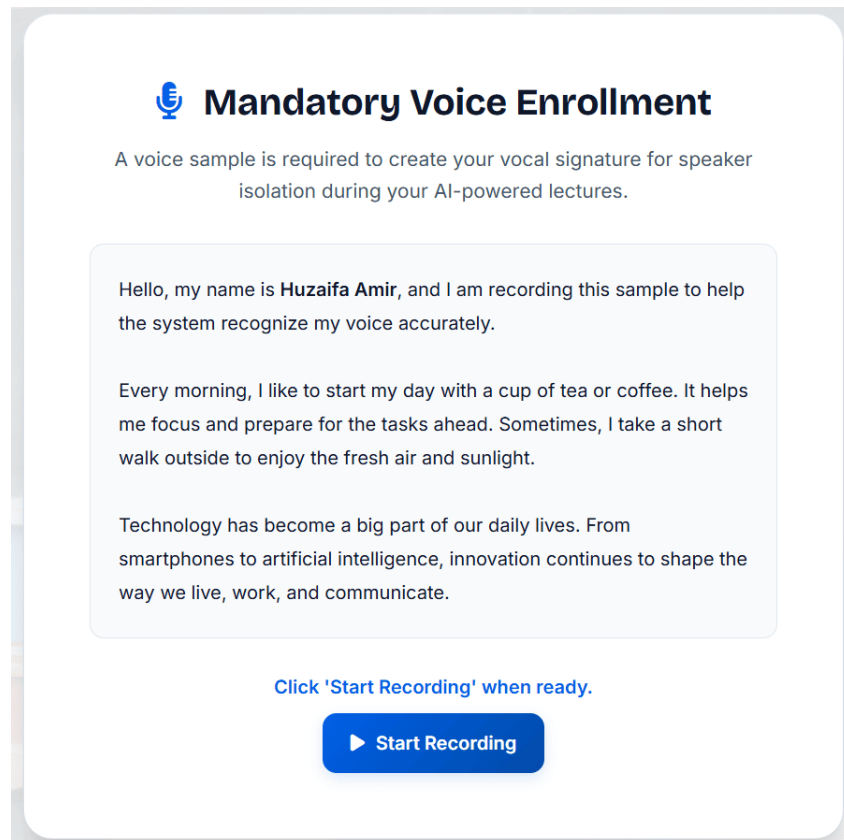


Figure 7.1: Mandatory Voice Enrollment Interface for Speaker Isolation

On the technical side, the frontend implementation involves capturing the teacher's audio stream in real-time. When the recording ends, the audio is compiled into a supported file type (such as WAV, MP3, WEBM, or OGG) and sent to the backend's `/api/voice-enroll` endpoint. The audio file is then processed by the voice encoding models used by the system (such as Resemblyzer) to be used as a raw input for speaker recognition. As a result, every teacher who registered will have a valid reference voice signature stored securely and associated with their account before they start managing courses.

7.1.2 Construction of RAG Knowledge Base

RAG capability constitutes the core intelligence of the Cassandra system. This module is responsible for converting raw educational materials textbooks and lecture notes into a structured vector database that the AI then queries to create contextually correct responses. For the implementation of this pipeline, there is a sequential process of file processing, text segmentation, and vector embedding generation handled via a Python Flask backend.

The process starts right at the frontend interface, where we have implemented a drag and drop zone

using HTML. The system only supports file upload in PDF and DOCX formats.

The backend then decides on the appropriate extraction strategy based on the file extension once the file is received. For PDF documents, we integrated the `pypdf` library for page iteration through document and text extraction while maintaining metadata regarding page numbers for eventual citations. For Word documents (DOCX), the `python-docx` library was utilized to parse paragraph elements. The extracted text is then cleaned via a function that normalizes whitespace and removes unsupported special characters to ensure the input is clean for the AI model.

Now, first split the text at sentence boundaries. Then group these sentences into chunks of about 2500 characters. What is really important is that we implemented a sliding window overlap of 300 characters in order to avoid cutting off sentences at mid-thought between chunks but keeping semantic context necessary for accurate retrieval.

Then, it converts chunks of text into mathematical vectors. We used the `sentence-transformers` library to load the `all-mpnet-base-v2` model, optimized for semantic similarity tasks.

Each chunk of text is embedded in a high dimensional vector. These embeddings are stored in the cloud database Supabase, as mentioned earlier in Figure 7.2, along with metadata comprising source filename, page number, and chunk index.

☐	id uuid	teacher_id uuid	course_id uuid	content text	metadata jsonb	embedding vector
☐	05b950c1-8852-4b5f-...	33e80c08-cd76-... →	2c1643ef-7139-40... →	the situation is diff	{"page":39,"filename":	[-0.0341329,0.03791452,
☐	0b89aad0-4857-4c7a-...	33e80c08-cd76-... →	2c1643ef-7139-40... →	Internet Society Int	{"page":28,"filename":	[0.062212475,-0.053830
☐	0ca17493-86ce-4b97-...	33e80c08-cd76-... →	2c1643ef-7139-40... →	required in every s	{"page":21,"filename":	[0.006448127,-0.098490
☐	0d89e775-5e7d-4578-...	33e80c08-cd76-... →	2c1643ef-7139-40... →	nanotechnology, in	{"page":29,"filename":	[0.016719546,-0.1123663,
☐	1011cce2-8542-4422-b...	33e80c08-cd76-... →	2c1643ef-7139-40... →	a high performanc	{"page":13,"filename":	[-0.032090902,-0.04904
☐	1088778e-854c-4db3-...	33e80c08-cd76-... →	2c1643ef-7139-40... →	World Wide Web -	{"page":13,"filename":	[-0.0012943421,-0.08020
☐	122e3131-e4d2-49d0-e...	33e80c08-cd76-... →	2c1643ef-7139-40... →	Firewall Main limit	{"page":40,"filename":	[-0.02066332,0.0283375
☐	16320c1a-b2e7-4772-8...	33e80c08-cd76-... →	2c1643ef-7139-40... →	protocols; LAPB an	{"page":13,"filename":	[0.021089675,-0.1006722

Figure 7.2: Storing Vector Embeddings for the RAG System

7.1.3 Teacher's AI Chatbot and Retrieval-Augmented Generation (RAG)

This module lets instructors query their own course materials in natural language using Retrieval Augmented Generation (RAG) to ground its answers in the specific textbooks and course materials.

The user interface was designed to be an independent floating widget rather than a fixed web page. This is because such a design enables the chatbot to be embedded into any application area.

Since text returned from LLM will come with formatting symbols such as (asterisks for bold text or

backticks for code), raw text display would be unfriendly to the user. We then wrote a series of Regular Expressions that dynamically converted these patterns into HTML tags. This allows the chatbot to render complex elements, including tables, syntax highlighting code blocks, and bulleted lists directly within message bubbles, greatly improving readability.

Whenever a teacher sends a message, the system converts their text into a numerical vector. For this purpose, the `sentence-transformers` library was used more precisely, the `all-mpnet-base-v2` model was utilised for this task.

In order to achieve better reliability of the system, we have adopted a robust hybrid approach for searching. The approach involves making RPC calls to the Supabase database, allowing it to perform vector similarity searches on the server side, where the server is configured to retrieve the top 4 most similar text chunks by default.

Once these relevant chunks of text are extracted, the system builds a comprehensive context window for the Language Model. We created a utility function called `build_context_with_history` which, in tandem with the generation prompt, assembles three pieces of information: instructions for the system (welcoming the AI as a teaching assistant named Cassandra), the most relevant excerpts extracted from the uploaded documents, and the last five exchanges of the conversation history. The inclusion of the conversation history is essential by strictly retaining the past 5 interactions, the AI can recall context so that the teacher does not have to repeat their question for follow-ups, while keeping the input window optimized and bounding the model's output to 500 tokens.

This assembled context is sent to the client that has connected to the DeepSeek-V3 model. The model processes the retrieved facts and produces a coherent natural language response as given in Figure 7.3. Finally, for record keeping purposes, the system asynchronously persists both the teacher's query and the AI's response into the `teacher_chat_history` table in Supabase.

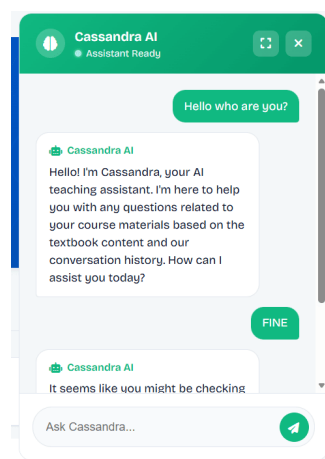


Figure 7.3: AI Teacher Assistant Chat Interface

7.1.4 PPTX Conversion Service

We set up a PPTX Conversion Service that converts proprietary PowerPoint files into a standardized PDF format. This would then be utilized anytime the teacher was using the slides or slides + whiteboard mode to record the lecture.

Once the file is received at the endpoint `./upload`, the backend will validate the file using a utility function named `allowed_file`, which will ensure that the file actually has a `.pptx` extension. Additionally, the backend will also ensure that the file is not larger than a strict limit of 50MB, which will ensure the stability of the server.

We thus developed a system that asynchronously tracked the status to improve user experience during conversion latency. Lastly, once the file has been validated, the actual process of converting the file will be handled by an external cloud-based service named `ConvertAPI`. This is because the process of converting the file will be handled remotely, which will ensure a high level of fidelity. Once the file has been converted by the cloud-based service, it will then be saved locally in a folder named `converted_pdfs` on the server.

7.1.5 Slides Only Lecture Session Mode

This module provides an interactive interface for instructors to present their material, as depicted in Figure 7.4, while the system captures the audio synchronized with specific slides. The architecture is designed to handle real-time state management to ensure that the exact audio recorded during the capture corresponds to precisely what is visual at that instance in time.

In this mode, the `pyMuPDF` library is used to rasterize each page of the Slide that were converted to PDF into high-quality PNG images. These images are cached into a temporary directory, and thus can be retrieved with very low latency. In a live presentation, the system uses specific stateless endpoints for the backend to register whether a session is starting or ending, whereas the live session variables, which include the index, total count, and recording status, are handled by the client interface.

A noteworthy feature of this implementation is its approach towards the segmentation of the audio. Traditionally, one would handle a single monolithic audio file for the entire lecture. However, in this implementation, the system handles the audio at a per-slide basis. The client interface handles the raw microphone capture of the individual chunks of audio, which are sent over HTTP to the backend server. The server side of the implementation utilizes the `pydub` library to securely handle the segmented audio chunks sent by the client.

This audio data is buffered in memory using NumPy arrays. Whenever the instructor triggers a naviga-

tion event to the next or previous slide, the system automatically flushes the current buffer. It writes the aggregated audio data to a WAV file with a name specific to that slide-like `slide_1.wav` and `slide_2.wav` using the `soundfile` library. This is a very fine-grained segmentation, which is of paramount importance in the following transcription phase, since it will naturally divide the lecture in semantically relevant chunks based on the visual context.

When the lecture Slides are loaded, a Start Overlay is displayed that needs to be confirmed manually by the instructor in order for the session to start, and this avoids any accidental audio capture. Once the session has started, the interface will show the image of the current slide.

We added intuitive navigation controls to enhance the teaching experience. Instructors can navigate through slides using either on-screen buttons or keyboard arrow keys that trigger back-end API endpoints `/api/next_slide` or `/api/prev_slide` respectively. Fullscreen Mode was implemented to use the browser's Fullscreen API to optimize the view for projection in physical classrooms.



Figure 7.4: Lecture Recorder Interface in Slides-Only Mode

7.1.6 Whiteboard Only Lecture Session Mode

The Whiteboard Only mode was developed in order to meet the particular challenge of digitizing those lectures where instructors make use of only physical markers and boards. Unlike the slide based approach, this module implements a dual device architecture. The instructor's laptop serves as the central server and audio recorder, while their mobile device serves as a remote wireless camera scanner. The Python backend uses a socket connection approach to programmatically detect the local IP address of the host machine on the network. This IP address is then sent to the desktop frontend that generates a dynamic QR code as shown in Figure 7.5 displaying the specific URL using the `qrcode.js` library.

The mobile client was designed to be lightweight and browser based so to be compatible with a vari-

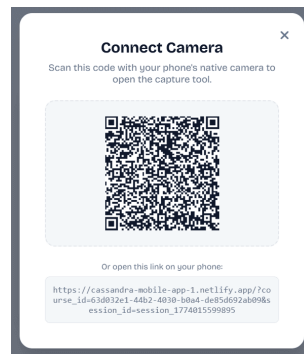


Figure 7.5: QR Code for Connecting External Mobile Camera

ety of smartphones, as shown in Figure 7.6. In order to access the phone's camera we employed the HTML5 file input element with the `capture=environment` attribute which forces the device to open the rear camera directly instead of the file picker. Upon taking a photo of the whiteboard, the application constructs a binary blob object. This is then wrapped inside of a `FormData` structure and sent asynchronously to the backend with a POST to the `/api/upload` endpoint, where it is securely routed to the `lecture_images` Supabase bucket.

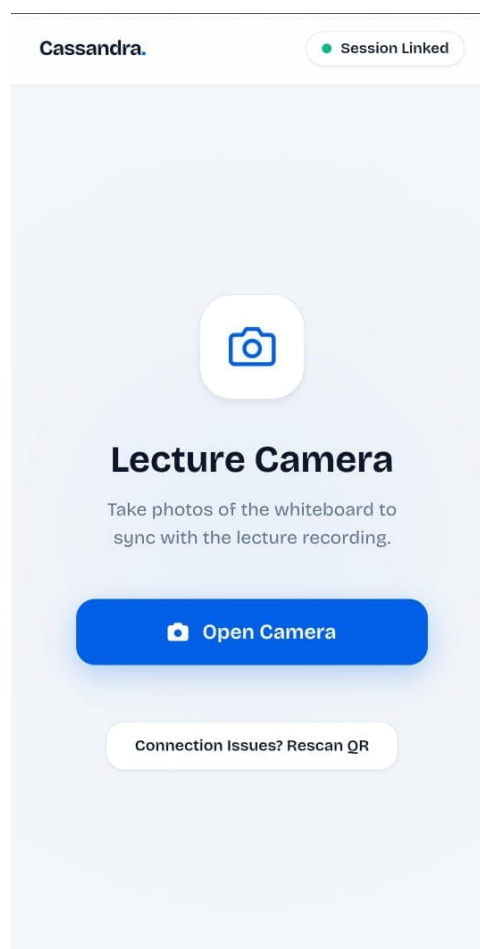


Figure 7.6: Camera Mobile Interface for Whiteboard Synchronization

The most critical technical requirement for this module was to synchronize the continuous audio recording with discrete visual snapshots. We based our backend on a state management system that considers each whiteboard photo as a separate slide. The audio is captured in real-time chunks by the pydub library and stored in a temporary memory buffer. When a new image is uploaded from the mobile device, or when the instructor navigates forward or backward between previous board captures, the system triggers the `save_current_audio` function. It flushes the current audio buffer to a WAV file associated with a particular board position, such as `wb_1.wav` thus, it ensures that audio playback is well-aligned with the handwritten content appearing at that moment.

The desktop interface as shown in Figure 7.7, acts as the control center, giving the instructor feedback about the recording status, as well as the current visual state. Since the image uploads are initiated from an external mobile device, the desktop frontend cannot respond to them directly using regular events. To address this problem we have introduced a polling mechanism which checks the `/api/status` endpoint every two seconds. Whenever the backend detects that the total count of images has changed, the desktop UI automatically refreshes the main display container with the newly captured whiteboard snapshot, allowing the instructor to make sure that the board content was captured clearly without looking at their phone's screen.

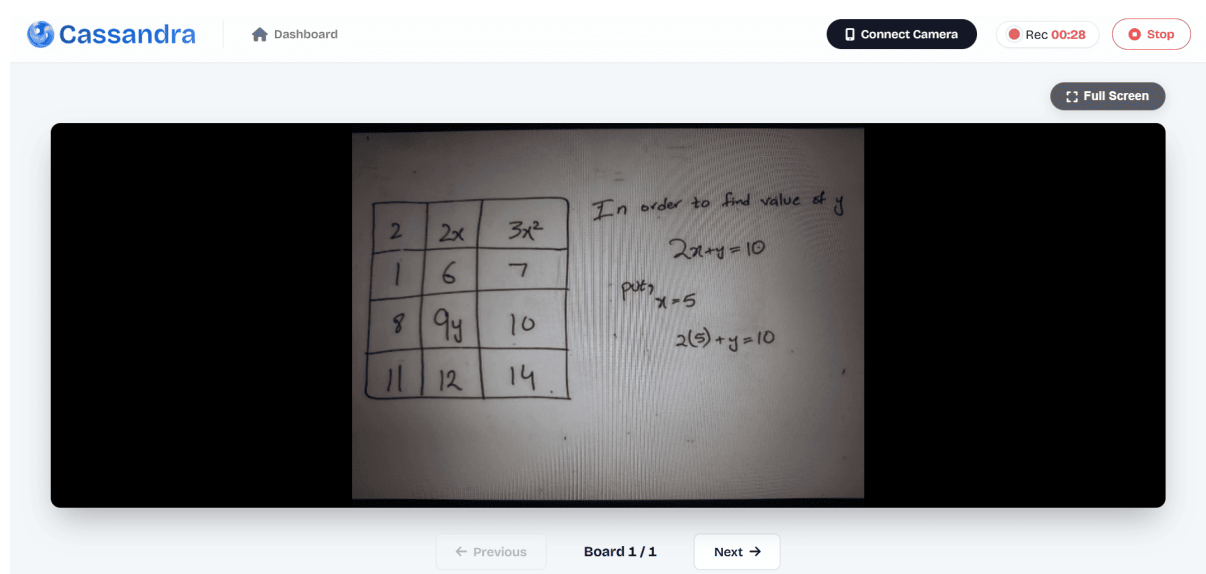


Figure 7.7: Whiteboard-Only Mode displaying Captured Handwriting

7.1.7 Hybrid Slides and Whiteboard Lecture Session Mode

This module synthesizes the functionalities of its previous capture modes into one hybrid environment in which instructors can enrich given slides with physical whiteboard snapshots. This backend design extends the `recording_state` dictionary to track a `board_images` mapping from specific slide indices to their respective whiteboard captures. Whenever a mobile device uploads an image via the endpoint

`upload_image`, it saves the file as `board_slide_{current_slide}.png` and associates it strictly with the currently active slide index to contextually bind visual annotations with the presentation material. In contrast, the audio pipeline via standard `save_current_slide_audio` persists narrative commentary into slide-specific WAV files. This creates a triad of data with every segment: Slide Image, Board Image, and Audio. This is reflected by the front-end interface through a split-pane design that renders the PDF slide while polling the `/api/status` endpoint to dynamically update the side panel when it detects a new whiteboard capture for the current slide.

7.1.8 Audio Only Lecture Session Mode

The Audio Only Lecture Session module in Figure 7.8 provides support for setups without display capabilities, such as visiting lectures, through the use of high-quality audio recording and instant feedback. On the frontend, HTML, CSS and JavaScript is used to create a minimalistic UI, centered by a high-contrast countdown timer and a CSS animated waveform that mimics visually and in real time while recording an audio environment. The backend consists of a server with Flask, utilizing the `pyaudio` library to manage a non-blocking input stream at 44,100 Hz. To protect against data corruption, a globally shared memory buffer is guarded by a `threading.Lock` while the `audio_callback` function continuously appends raw audio frames to it. Rich state management is possible via this system the instructor is allowed to pause and resume the session via a Boolean flag on the back end that temporarily throttles data collection while the front-end timer comes to a pause in synchrony. When complete, the collected NumPy arrays are stitched into one continuous stream, and the recording is saved as a standardized WAV file using the `soundfile` library.

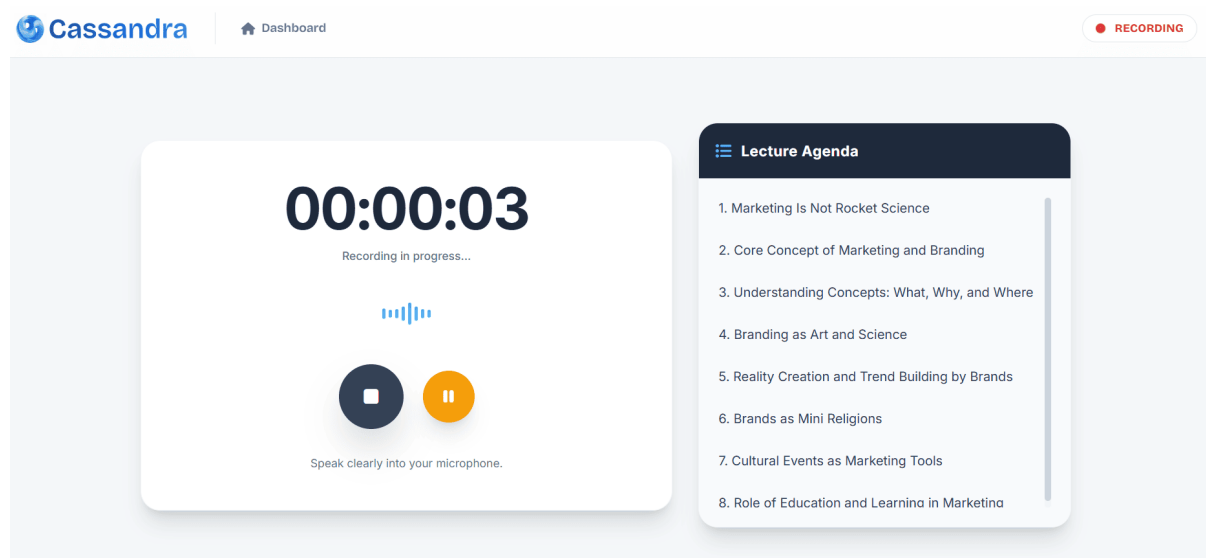


Figure 7.8: Audio Only Lecture Session Mode Interface

7.1.9 Audio Preprocessing Pipeline

These are raw audio files recorded from the lecture sessions, and whether the lecture is conducted via slides, whiteboard, slides + whiteboard, or audio only modes, this material requires intense processing before it can become useful study material. We implemented a dedicated Python based pipeline that was designed to handle the following critical tasks: ASR, translation of mixed English-Urdu content, and speaker isolation to remove background noise or student interruptions. This pipeline makes use of the `faster-whisper` library for transcription and `resemblyzer` for biometric voice verification.

The pipeline begins by initializing two deep learning models. First, it loads the `faster-whisper-large-v3` model. Meanwhile, the system also loads the Voice Encoder model for speaker verification.

The system retrieves the `teacher_sample.wav` recorded during the registration phase in order to isolate the instructor's voice and (securely stored in the `voice_signatures` bucket). This file is fed through the Voice Encoder, which produces a high-dimensional vector embedding representing the ground truth fingerprint of the teacher.

The core processing logic will loop through each and every audio chunk that belongs to a certain slide or whiteboard image. This transcription is set with the parameter `task=translate`, this will have the Whisper model auto translate Urdu phrases to English during transcription. We also enabled the VAD filter with (`vad_filter=True`) to automatically ignore periods of silence.

However, raw transcription captures every voice in the room. In order to filter the students shown in Figure 7.9, the system will iterate through each transcribed text segment.

1. Full Transcription (Unfiltered)

So to answer that... excuse me professor what about the accuracy value? A confusion matrix gives us a much better idea of performance than simple accuracy. For example, if you look at the true positives and true negatives... sorry can you repeat that last part? Of course. The true positives and true negatives tell us what the model classified correctly. We combine these with false positives and false negatives to calculate precision and recall.

2. Speaker Identification (Comparison)

So to answer that... ~~excuse me professor what about the accuracy value?~~ A confusion matrix gives us a much better idea of performance than simple accuracy. For example, if you look at the true positives and true negatives... ~~sorry can you repeat that last part?~~ Of course. The true positives and true negatives tell us what the model classified correctly. We combine these with false positives and false negatives to calculate precision and recall.

■ Teacher (Kept) ■ Student/Other (Filtered Out)

3. Finalized Teacher-Only Text

So to answer that... A confusion matrix gives us a much better idea of performance than simple accuracy. For example, if you look at the true positives and true negatives... Of course. The true positives and true negatives tell us what the model classified correctly. We combine these with false positives and false negatives to calculate precision and recall.

Figure 7.9: Transcription Pipeline Showing Speaker Identification and Noise Filtering

For every segment, it would extract the corresponding audio and create a temporary voice embedding, calculating the dot product or cosine similarity against the teacher's reference embedding. We implemented a strict threshold for similarity of 0.65. Only segments that surpass this threshold are appended onto the final transcript, meaning the generated notes reflect only the delivery of the instructor.

We also assessed transcription quality using five different measures:

- **Cosine Similarity:** Measures the semantic similarity using TF-IDF vectors.
- **Levenshtein Similarity:** This measures the edit distance between the actual and transcribed text.
- **BLEU Score:** This will gauge the quality of translation against a reference text.
- **Word Accuracy:** Measures the percentage of the words that exactly match.
- **Length Similarity:** The transcription should maintain the structural length of the original speech.

7.1.10 Visual Content Classification

The Visual Content Classification module represents a high level filtering component for media images extracted from lecture slides. Its main purpose is to decide whether the visual piece is a data table, a mathematical equation, or a general image, such as a diagram, photo, or decorative graphic. This classification is of great importance, since tables and equations require specialized OCR pipelines, such as this includes equation conversion to LaTeX, while the general images are treated otherwise. In our paper, we took a deep learning approach based on the architecture of the Vision Transformer, fine tuned on a curated dataset prepared for this task.

To train a robust classifier on 180,000 images, with 60,000 images per class, three different sources were combined to build a balanced dataset, as shown in Figure 7.10.

- **Equations:** We utilized the `Oleehy0/latex-formulas` dataset, which contains images of rendered LaTeX mathematical expressions.
- **Tables:** We sourced table images from the `ajimeno/PubTabNet` dataset, a large scale collection of scientific tables extracted from academic publications.
- **General Images (Neither):** To represent the background class, we used the CIFAR-10 dataset, which provides a diverse range of natural images (animals, vehicles, objects).

Attempting to show EQUATION samples...
Filter: 0% | 0/150 [00:00<?, ? examples/s]

Sample Images for: EQUATION

Label: equation

$$z_k = (z_{1k}, z_{2k}) \in Z, \quad k = 1, \dots, K,$$

Label: equation

$$\int_{\mathcal{M}_{\text{true}}} \left| \frac{1}{\det T_0} \right|,$$

Label: equation

$$\langle \psi_0, \Phi \rangle(A, U) = \int_{\mathcal{A}} \langle \mu, \Phi \rangle(A \cap \psi_0(X), U \cap \psi_0(X)) \, d\mu$$

Label: equation

$$Z_F(G^*) = Z_F(0)Z_{WZW}(N)Z_{WZW}(-N)Z_{ghost}$$

Label: equation

$$Q = -\partial_t^2 + \mu_0^2 \left(1 - \frac{2}{\cosh^2(\mu_0 r)} \right)$$

Attempting to show TABLE samples...
Filter: 0% | 0/150 [00:00<?, ? examples/s]

Sample Images for: TABLE

Label: table

Parameter	Value	Unit	Normal range
Serum amylase	30	U/L	30-140 U/L
Alkaline phosphatase	140	U/L	30-140 U/L
Lactate Dehydrogenase	330	U/L	40-225 U/L
Gamma glutamyl transferase	147	U/L	10-40 U/L
Alpha-fetoprotein	<20	U/L	0-9 U/L
CA-125	10.4	U/L	0-35 U/L
CA-19-9	143	U/L	0-29 U/L
Bilirubin direct	4.6	mg/dL	<0.2 mg/dL
CEA	0.6	ng/mL	0-5 ng/mL

Label: table

Parameter	Value	Unit	Normal range
Serum amylase	30	U/L	30-140 U/L
Alkaline phosphatase	140	U/L	30-140 U/L
Lactate Dehydrogenase	330	U/L	40-225 U/L
Gamma glutamyl transferase	147	U/L	10-40 U/L
Alpha-fetoprotein	<20	U/L	0-9 U/L
CA-125	10.4	U/L	0-35 U/L
CA-19-9	143	U/L	0-29 U/L
Bilirubin direct	4.6	mg/dL	<0.2 mg/dL
CEA	0.6	ng/mL	0-5 ng/mL

Label: table

Parameter	Value	Unit	Normal range
Serum amylase	30	U/L	30-140 U/L
Alkaline phosphatase	140	U/L	30-140 U/L
Lactate Dehydrogenase	330	U/L	40-225 U/L
Gamma glutamyl transferase	147	U/L	10-40 U/L
Alpha-fetoprotein	<20	U/L	0-9 U/L
CA-125	10.4	U/L	0-35 U/L
CA-19-9	143	U/L	0-29 U/L
Bilirubin direct	4.6	mg/dL	<0.2 mg/dL
CEA	0.6	ng/mL	0-5 ng/mL

Attempting to show NEITHER samples...
Filter: 0% | 0/150 [00:00<?, ? examples/s]

Sample Images for: NEITHER



Figure 7.10: Dataset Samples for Classifying Equations, Tables, and Images

For data preprocessing, we standardized the format of input. First of all, we transformed all input images into RGB to maintain consistency. We then employed AutoImageProcessor from the Hugging Face Transformers library for resizing and normalizing the images to 224x224, as required by this particular pre-trained Vision Transformer model.

We chose the model google/vit-base-patch16-224-in21k. It is a pre-trained Vision Transformer on ImageNet-21k with great feature extraction capability. We fine tuned the model for three class classification by replacing the final classification head with a new linear layer that maps to our specific labels: equation (0), table (1), and neither (2).

The training process was orchestrated using the Hugging Face Trainer API. We fine tune the model for total 3 epochs, with a batch size of 8 and learning rate of 2×10^{-5} . In order to ensure model generalization, we used a stratified train test split, setting aside 10% of the data for validation, which accounted for 18,000 images. The optimization goal was to minimizing cross entropy loss between the predicted class probabilities and the ground truth labels.

7.1.11 Qwen3-VL-4B-Instruct Fine-Tuning for Slides Equations and Tables Extraction

Extracting structured information, such as tables and mathematical formulas, from images of slides is very different from other tasks. Standard OCR often garbles complex layouts, destroying row/column relationships in tables and misinterpreting LaTeX symbols. In this work, we solve this problem by implementing a fine-tuning pipeline using the Qwen3-VL-4B-Instruct Multi-modal model. This approach allows the system to see the visual structure of tables and equations and generate the corresponding LaTeX code directly, rather than raw text.

The source data included:

- **Tables:** 5000 samples from the `deepcopy/latex-tables-ocr` dataset, representing scientific and academic tables of a nature similar to those in lectures.
- **Equations:** 4400 samples from the `unsloth/LaTeX_OCR` dataset and 600 from the `stanford-crfm/image2struct-latex-v1` dataset providing rendered mathematical expressions.

We developed a specific prompting approach for the teacher model. The system prompt told the AI to be a precise OCR engine with rigid formatting: providing clean valid LaTeX for tables and equations as in Figure 7.11, without conversational filler. We were able to be computationally efficient by leveraging 4-bit quantization (bitsandbytes NF4) in order to fit the 4-billion parameter teacher model into the GPU memory for training hundreds of labels. The JSON output file consisted of 10,000 high quality image text training pairs.

$$[\text{Density}] = \frac{[M]}{[L] \times [L] \times [L]} = \frac{[M]}{[L^3]} = [ML^{-3}]$$

```
\[
[\text{Density}] = \frac{[\text{M}]}{[\text{L}] \times [\text{L}] \times [\text{L}]}
= \frac{[\text{M}]}{[\text{L}^3]}
= [\text{M}]\text{L}^{-3}
\]
```

Figure 7.11: Mathematical Equation Extraction and LaTeX Conversion

For fine tuning, we employed the `unsloth` library to attain optimal training speed and memory efficiency. We loaded the base model `Qwen3-VL-4B-Instruct` and proceeded to apply the LoRA adaptation. LoRA enables us to freeze the massive pre-trained weights of the model and train only a limited number of adapter parameters (less than 1% of the total), making it feasible on consumer hardware.

All linear layers from both the vision and language components of the model had LoRA adapters applied,

with a rank of $r = 16$ and alpha of 16. Training was done using the SFTTrainer from the Hugging Face `trl` library. This used a cosine learning rate scheduler with a peak of 2×10^{-4} and a warm-up ratio of 0.03 to ensure stable convergence.

7.1.12 Slides Content Decomposition and Structuring

This module deals with extracting, classifying, and restructuring slide elements into a clear, semantic format, as shown in Figure 7.12.

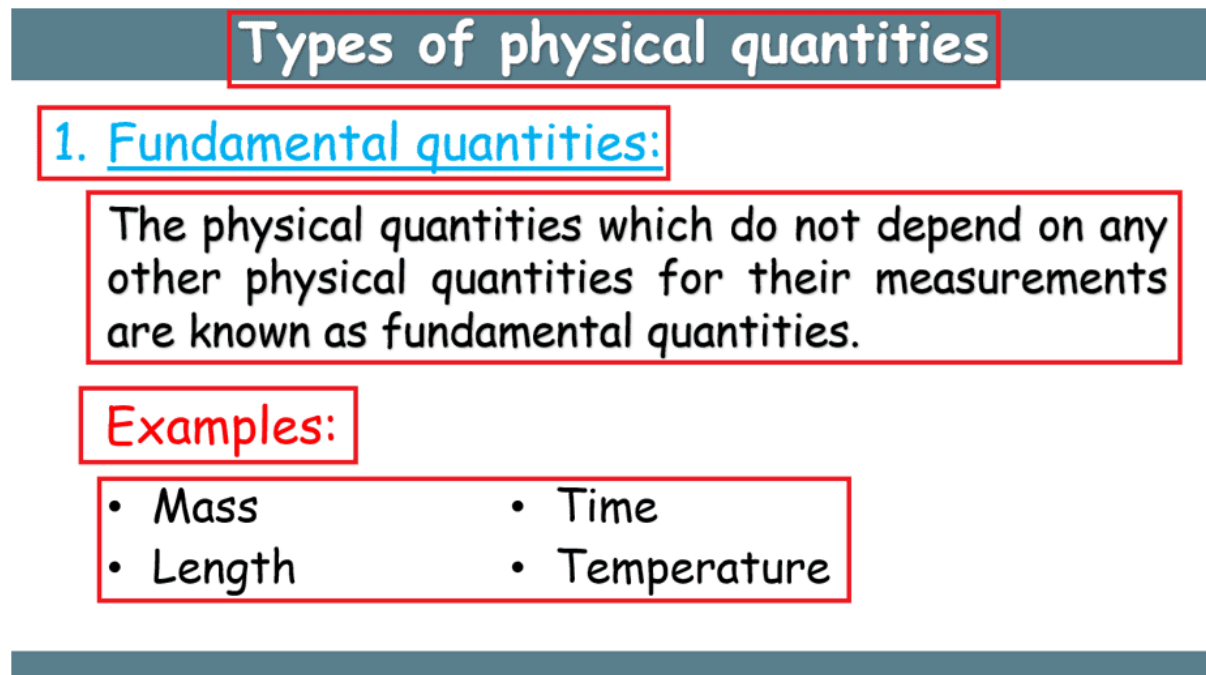


Figure 7.12: Slide Element Detection and Bounding Box Segmentation

The first step is to break down the PowerPoint (.pptx) file. We used the `python-pptx` library and the `zipfile` module to access the internal XML structure of the presentation. The system processes each slide parsing the XML tree to know distinct visual elements.

We built custom helper functions, such as `get_local_xfrm`, to get the spatial coordinates (English Metric Units) against the each element. These coordinates are important in order to know the reading order and layout. The logic of extraction sorts elements into initial categories:

- **Text:** Obtained from `<a:t>` tags, with normalization in order to get rid of extra whitespace.
- **Media:** Images (`<p:pic>`) are identified. For integrated objects, the system extracts the binary data from the ZIP archive.
- **Tables and Equations:** In order to achieve high accuracy for complex elements like equations or tables, the system displays the entire slide as a high resolution PNG image using `PyMuPDF (fitz)`.

After that, It then calculates pixel perfect bounding boxes to crop specific regions corresponding to the extracted elements.

Sometimes, it is also the case that the tables and equations are used as images in the slides. The Vision Transformer classifier trained in the previous module was integrated to address this. Each cropped image is passed through the classifier to definitively categorize it as a Table, Equation, or General Image.

For elements classified as General Images like (diagrams, photos) we added an image captioning step using the Salesforce/blip2-flan-t5-xl model to generate a descriptive caption string up to 50 tokens long `max_new_tokens=50`. This model gives clear natural language captions, serving as alt-text and providing context for the language model in the next structuring phase. For equation and table images, we applied the fine tuned Qwen3-VL-4B-Instruct model to extract the data.

The last and most crucial step is determining the logical flow of information. PowerPoint slides often have a non linear visual layout that doesn't transfer directly to a text document. To address this, we used the DeepSeek-chat Large Language Model (LLM).

We created a structured prompt that gives the LLM a list of all slide elements, including their spatial coordinates (top, left), inferred types (from the classifier), and textual content (from OCR or direct extraction). The LLM's tasks are:

1. Reordering elements based on logical reading order (Top-Down, Left-Right).
2. Assigning semantic roles such as Main Heading, Subheading, Bullet List, or Figure Caption.
3. Correct the mistakes which are typos from the extracted text.
4. Outputting the result as a strictly formatted JSON object as shown in Figure 7.13.

```
{
  "presentation": "Slides.pptx",
  "total_slides": 27,
  "slides": [
    {
      "slide_index": 1,
      "slide_width_emu": 12192000,
      "slide_height_emu": 6858000,
      "rendered_image": "slides_images\\my_slides\\slide_1.png",
      "elements": [
        {
          "id": "s1_0",
          "type": "image",
          "emu_bbox": [
            0,
            -15767,
            12192000,
            6873767
          ],
          "px_bbox_estimate": [
            0,
            0,
            1920,
            1080
          ],
          "text": null,
          "image_path": "pptx_extracted\\media\\slide1_e0_image.png",
          "caption": "a close up of a person holding a cell phone with a blue background"
        }
      ]
    }
  ]
}
```

Figure 7.13: JSON Output Structure representing Processed Slide Data

7.1.13 Whiteboard Processing Pipeline

The pipeline of whiteboard processing is designed to take an image snapshot of a teacher's handwriting on a whiteboard and transform it into a clean, structured, and machine readable format. First, the teacher uploads a snapshot of the whiteboard shown in Figure 7.14.

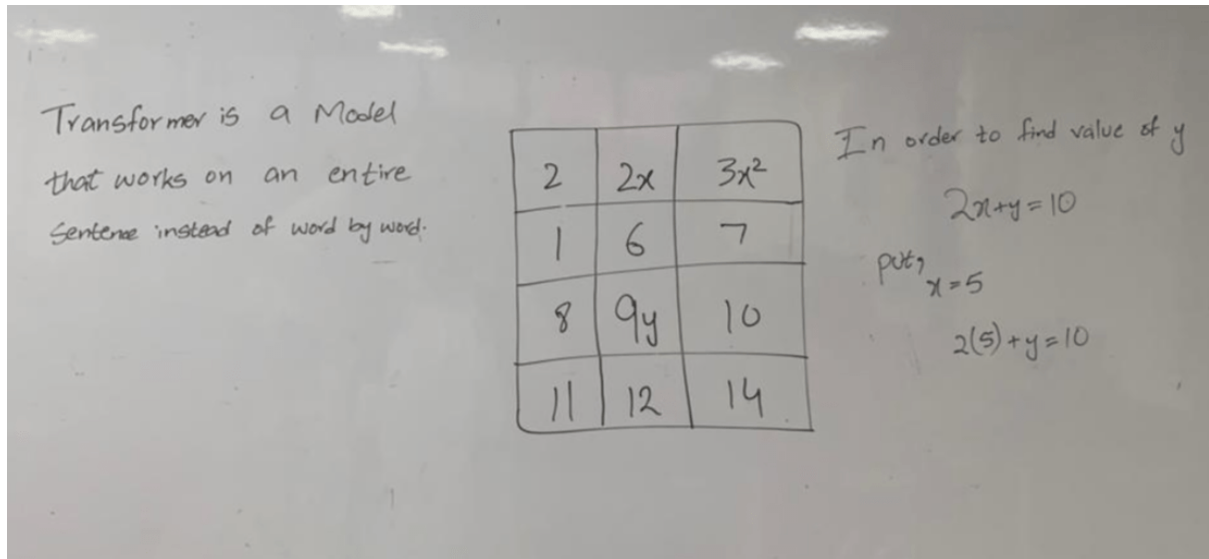


Figure 7.14: Handwritten Whiteboard Input Snapshot

The whiteboard images are preprocessed to improve clarity and uniformity for OCR. The steps include saturation and mild brightness enhancement, adaptive contrast adjustment, noise reduction, gentle sharpening, and finally conversion to grayscale. This ensures a high contrast, clean, and standardized input for the OCR model. After applying the preprocessing steps the resulting image is shown in Figure 7.15.

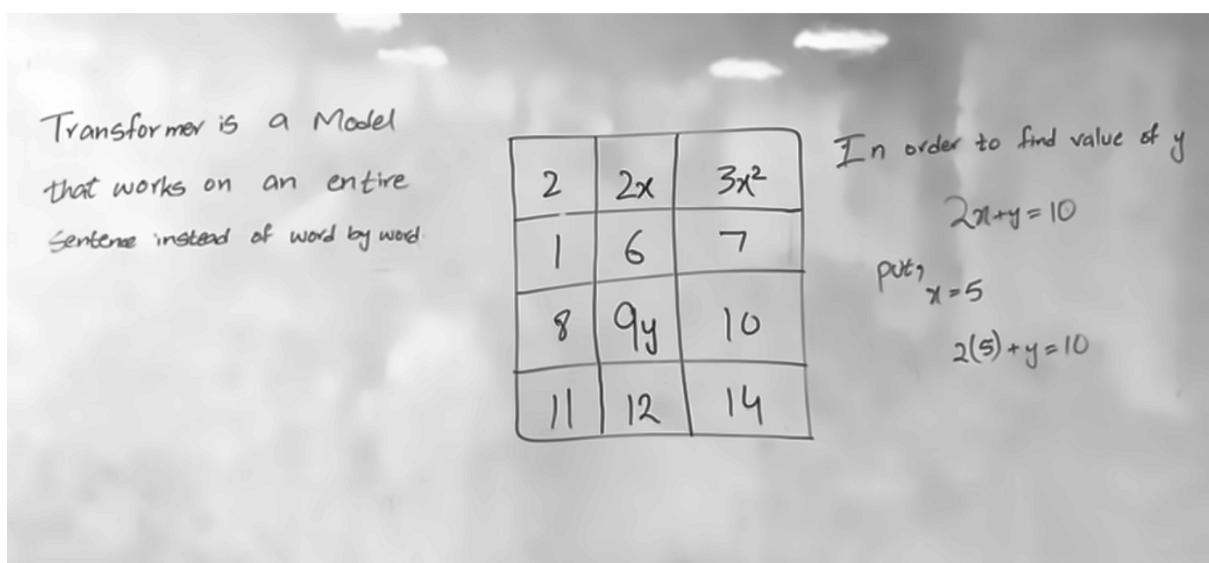


Figure 7.15: Preprocessed Whiteboard Image

Once loaded, the image is combined with the carefully crafted OCR instruction prompt that tells the model how to process the content. The prompt identifies all the visible elements like (plain text, tables and mathematical equations) to be identified and extracted exactly as they are on the board. It also asks the model to preserve all spacing, line breaks, punctuation, and case, and to output the result in a structured JSON format.

The image uploaded and the prompt are fed into the `AutoProcessor` of the vision language model `Qwen3-VL-4B-Instruct`. The processor turns both the image and textual instructions into model compatible tensors. Further, these tensors are dispatched to the model for inference, where the model uses its visual reasoning and OCR capability to recognize all elements in the image. Other than typical computer vision techniques, it directly parses the visual content. In summary, the following are the major operations done by the model in this step:

- Detects paragraphs, tables, and mathematical equations that are visible in the image.
- Assigns a unique identifier to each detected element for tracking and organization.
- Categorizes each element according to its logical type.
- Extracts the exact content of each element, including spacing, line breaks, punctuation, and case.
- Annotates each element with bounding box coordinates in order to capture its location on the board.
- Defines the reading order of all elements from top-left to bottom-right, allowing layout reconstruction.

The model outputs a JSON formatted result after processing with all the structured elements it produces. These elements capture each element's unique identifier, logical content type, exact extracted content, reading order, and metadata such as the bounding box and original visual type. This structured JSON is quite useful for subsequent applications as it can be used to create the digital full refined lecture notes. This pipeline is robust and reproducible, ensuring that the image being uploaded will always produce a deterministic and completely structured output from the beginning to the end without using any post-processing heuristics.

Figure 7.16 shows the output of the pipeline in structured form it shows all the detected elements along with their types, reading order, and bounding boxes.

```

1  {
2    "image_index": 1,
3    "structured_elements": [
4      {
5        "original_id": "s1_e1",
6        "content_type": "paragraph",
7        "content": "Transformer is a Model",
8        "reading_order": 1,
9        "metadata": {
10         "image_path": null,
11         "bbox": [27, 167, 337, 238],
12         "raw_type": "text"
13       }
14     },
15     {
16       "original_id": "s1_e2",
17       "content_type": "paragraph",
18       "content": "that works on an entire",
19       "reading_order": 2,
20       "metadata": {
21         "image_path": null,
22         "bbox": [27, 250, 337, 321],
23         "raw_type": "text"
24       }
25     },
26     {
27       "original_id": "s1_e3",
28       "content_type": "paragraph",
29       "content": "Sentence instead of word by word.",
30       "reading_order": 3,
31       "metadata": {
32         "image_path": null,
33         "bbox": [27, 333, 337, 404],
34         "raw_type": "text"
35       }
36     },
37     {
38       "original_id": "s1_e4",
39       "content_type": "table",
40       "content": "| 2 | 2x | 3x | \\n | 1 | 6 | 7 | \\n | 8 | 9y | 10 | \\n | 11 | 12 | 14 |",
41       "reading_order": 4,
42       "metadata": {
43         "image_path": null,
44         "bbox": [412, 207, 672, 770],
45         "raw_type": "table"
46       }
47     }
48   ]
49 }

```

Figure 7.16: JSON Output Structure Representing Processed Whiteboard Data

7.1.14 Visual Content and Audio Alignment

The Merge Phase is extremely important as it marks the beginning of organized notes. This step organizes the several recovered JSON outputs from the pipelines both chronologically and semantically. The system has to computationally match the transcript segments to their corresponding visual ideas since slides and whiteboards work on different timelines from the spoken lecture audio. The backend generates embeddings for the parsed visual materials and the transcribed audio using a semantic embedding model SentenceTransformer all-mpnet-base-v2.

The alignment logic differs based on current active processing mode. The `DynamicSlideIntegrator` in the Slides-only mode links audio transcripts to specific slide elements using a semantic similarity threshold of 0.50. To handle the sparse, math-heavy nature of whiteboard material in the Whiteboard-only mode, a `WhiteboardAudioIntegrator` is used with a tighter threshold of 0.60. The Audio-only mode omits visual synchronisation completely and uses temporal borders to order the raw sequential audio segments.

The combined mode Whiteboard + Slides calls for a two-step merging procedure. The system first uses a dedicated `merge_slides_and_whiteboards` helper function before the audio is aligned. Using a chronological lookup approach, this feature adds the individual whiteboard images into the slide order by finding the closest previous slide index to logically connect the whiteboard details. Once this combined visual timeline is established, it generates a unified sequence of frames `<FRAME type='slide|white board'>` and proceeds with the standard semantic audio alignment.

7.1.15 Retrieval-Augmented Explanation Generation

The Explanation Generation process happens once the JSON and Audio structures have been successfully matched. The backend iterates through all matching segments and uses the DeepSeek-V3 Large Language Model to generate accurate instructional explanations instead of merely summarizing the texts. This step relies on a retrieval augmented generation pipeline for correctness guarantees. In terms of textbook knowledge, the system queries the Supabase vector database. This results in returning the rag context string with the ground truth background knowledge included.

The prompt engineering for this module is strictly tailored to the specific learning modes. Across all visual modes (Slides, Whiteboard, and Combined), the system provides the LLM with the visual context and strict instructions to output a direct, engaging 2-3 sentence explanation of the target concept. To ensure the final output reads like a professional textbook rather than a generic transcription, the system actively forbids the use of meta-referential phrases such as “The slide shows,” “The board mentions,” or “According to the context”.

In the Audio-only mode, the prompt relies exclusively on the raw transcript chunk and the `rag_context` to explain the spoken topic. Conversely, the Whiteboard-only mode receives a highly structured `board_context` containing spatial element mappings, while the Combined mode receives a unified `visual_context`. By synthesizing the immediate classroom data with deep textbook knowledge, the AI produces highly refined micro explanations that form the building blocks of the final document.

7.1.16 Final Summarization and Markdown Compilation

The last step of the pipeline is to put all the enriched micro-explanations and structural contexts into a continuous, properly formatted Markdown file. The DeepSeek-V3 Large Language Model, which is called upon to function as a knowledgeable Professor producing a textbook chapter, helps the system reach this aim by means of official “deepseek-chat” API endpoint accessed programmatically. The backend runs a number of specialized Python helper functions including “latex_table_to_html” to properly render complicated tables and “fix_list_spacing” to un-mash bullet points and ensure strict Markdown adherence in order to handle formatting irregularities from the raw OCR. The compilation rules change dynamically based on the active mode. The LLM is directed in the Slides-only mode to deliberately reject strictly ornamental background graphics while flawlessly insert educational visuals (such as charts and diagrams) using Markdown tags immediately following their textual explanations. In the Audio-only mode, the DeepSeek model is given strict instructions to strongly reject irrelevant classroom material, clearly deleting logistical talks, technical projector problems, personal stories, and off-topic conversation.

The DeepSeek-V3 model has its own unique design limitations when processing multi-modal and mathematical information. In the Combined mode, the system makes sure that whiteboard content is smoothly integrated into the topic of the relevant preceding slide as teacher explanation or worked examples, instead of being separate whiteboard sections. The Whiteboard-only mode, on the other hand, actively guides the LLM to stay away from producing Markdown image tags completely by enforcing a tight NO IMAGES restriction. The whiteboard summarizer instead concentrates only on keeping every derivation step and mathematical rule by means of exact block LaTeX formatting. The final summarized and compiled Markdown output is shown below in Figure 7.17.

```
## International System of units (SI)

To ensure that measurements are understood globally, we need a standard. This system consists of seven fundamental units and two supplementary units.

In 1971, General Conference on Weight and Measures held its meeting and decided a system of units for international usage. This system is called international system of units and abbreviated as SI from its French name. The SI unit consists of seven fundamental units and two supplementary units. In 1971, the General Conference on Weights and Measures established the International System of Units, abbreviated as SI.

The International System of Units (SI) is the globally agreed-upon standard for measurement, created to ensure consistency and clarity in science and trade worldwide. Its foundation is built on seven fundamental units like the meter for length and the kilogram for mass which define all other measurable quantities.

### Seven fundamental units

These seven base units are the essential building blocks of the International System of Units (SI). Each one defines a fundamental physical quantity, and from them, we can derive every other unit of measurement used in science and engineering.

| **FUNDAMENTAL QUANTITY** | **SI UNIT** | **SYMBOL** |
| :--- | :--- | :--- |
| Length | meter | m |
| Mass | kilogram | kg |
| Time | second | s |
| Temperature | kelvin | K |
| Electric current | ampere | A |
| Luminous intensity | candela | cd |
| Amount of substance | mole | *mol* |
```

Figure 7.17: Compiled Markdown Output after Final Summarization

7.1.17 Final HTML Rendering and Notes Normalization

The system requires strict post-processing procedures to create final clean HTML and Markdown notes because standard LLMs and OCR engines generate incomplete and unstructured results. The backend system employs unique Python functions which transform unprocessed output into standardized format before it reaches students. The visual extraction system first generates raw LaTeX code which presents the most difficult formatting problem for data tables. The system uses a `latex_table_to_html` function which converts latex tables into html format to achieve compatibility across different web platforms. The script uses Regular Expressions (Regex) to analyze complex tabular environments which began as table structures and it transforms the tabular data into HTML table elements including table rows and table data cells while preserving bold text and removing LaTeX borders `hline`. The system executes a `fix_list_spacing` algorithm to maintain strict compliance with Markdown requirements. The algorithm examines slide extraction results which display all bullet points on one line and it inserts empty lines before list items to keep web parsers from merging lists with previous paragraphs.

Finally, a `clean_batch_output` function provides a complete solution to discard all conversational filler which the LLM produces (e.g. “Here are the notes”, “End of batch”) through its output and it also removes all unnecessary Markdown code fences. The combination of RAG enrichment and HTML conversion and Regex cleaning produces an output that maintains continuous flow as an academic document with proper formatting. The processed and structured final notes are illustrated in the Figure 7.18.

International System of units (SI)		
To ensure that measurements are understood globally, we need a standard. This system consists of seven fundamental units and two supplementary units.		
In 1971, General Conference on Weight and Measures held its meeting and decided a system of units for international usage. This system is called international system of units and abbreviated as SI from its French name. The SI unit consists of seven fundamental units and two supplementary units. In 1971, the General Conference on Weights and Measures established the International System of Units, abbreviated as SI.		
The International System of Units (SI) is the globally agreed-upon standard for measurement, created to ensure consistency and clarity in science and trade worldwide. Its foundation is built on seven fundamental units—like the meter for length and the kilogram for mass—which define all other measurable quantities.		
Seven fundamental units		
These seven base units are the essential building blocks of the International System of Units (SI). Each one defines a fundamental physical quantity, and from them, we can derive every other unit of measurement used in science and engineering.		
FUNDAMENTAL QUANTITY	SI UNIT	SYMBOL
Length	meter	m
Mass	kilogram	kg
Time	second	s
Temperature	kelvin	K
Electric current	ampere	A
Luminous intensity	candela	cd
Amount of substance	mole	mol

Figure 7.18: Normalized Final Notes with HTML Conversion

7.1.18 Interactive Student AI Chatbot and RAG Framework

In order to deliver continuous interactive learning assistance, the Cassandra application integrates a specific context aware AI Chatbot microservice designed using Flask. Such a component serves as an intelligent teaching assistant whose responses will be grounded in facts derived solely from the course notes supplied and therefore ensures high academic accuracy while mitigating any potential hallucinations generated by the LLM. The RAG pipeline makes use of the `sentence-transformers/all-mpnet-base-v2` embedding model for executing vector searches of students' questions in the context of course notes stored in the database. In case of unconfident vector search results, a fallback to exact string matching occurs.

The key here lies in the usage of Supabase Remote Procedure Calls (RPCs), which ensure that the chatbot has the ability to retain conversation context. Prior to sending an answer, the backend executes the `get_chat_history` RPC endpoint to pull up the most recent conversations that the particular student had on that particular class and adds it to the context. After that, both the generated prompt and documents are sent to the DeepSeek-V3 Large Language Model through the `deepseek-chat` API endpoint. The system prompt ensures that the AI takes on the persona of "Cassandra," a tutor whose task is to guide the student towards understanding and never give away the exact answer while making sure to say explicitly when something is beyond the scope of the provided notes.

7.1.19 Web Integration and Full-Stack Cloud Architecture

The integration of the complex AI multimodal pipelines of the Cassandra system into a cohesive web-based learning platform is facilitated by its fully decoupled and multi-cloud architecture. The interface that a user will interact with communicates with an orchestration layer in a central location using CPU-based Azure infrastructure. In turn, the Azure infrastructure runs light-weight Flask/FastAPI microservices that form the basis of the traffic controllers used to route requests within the application. Since the processing of lectures will involve large amounts of data and will require multiple steps, the Azure infrastructure will use asynchronous orchestration systems like FastAPI's `BackgroundTasks`. Once a lecture ends, the Azure orchestrator will receive this payload asynchronously to process the lecture.

For handling the highly mathematical AI operations without overloading the main web servers, Azure serves as a safe router that routes these heavy tasks to specific Modal GPU containers. Based on the active lecture mode, Azure orchestrates the execution of separate Microservices of Modal through HTTP calls, including Audio Processor, Slide Decomposer, Whiteboard Extractor, and finally the Notes Generation Engine. Modal containers are dynamically assigned either T4 or A10G GPUs to process

Faster-Whisper and Qwen3-VL models, and they will be scaled down when not in use. This separation of concerns between web routing and AI inference ensures that the frontend is completely isolated from the backend's deep learning computations.

The perfect blend of the frontend website, Azure orchestrators, and Modal GPUs is facilitated by Supabase, serving as the brain and data lake for the entire system. Rather than transferring large, cumbersome files between different servers, the frontend website securely uploads the raw audio of lectures, presentation files, and whiteboards into their respective Supabase storage buckets. The Azure orchestrator will pass on the secure database references of these files to the Modal GPUs, which will retrieve the files, run the AI processing, and store the outputted JSON data and compiled Markdown notes back into the Supabase PostgreSQL database. Moreover, Supabase facilitates the RAG capabilities of the website by saving vector embeddings of documents, enabling the Azure-hosted chatbot microservices to quickly find semantically similar data when students or teachers ask questions. Lastly, Supabase allows the frontend website to retrieve the compiled notes from the database immediately after the backend AI finishes processing the data.

7.2 Test Case Design and Description

The following tables present the detailed system test cases for different modules of the Cassandra platform. Each test case evaluates functional correctness, system reliability, and expected software responses under defined conditions.

Table 7.1: Teacher Voice Enrollment Test

Authentication Module			
Reference: Section 7.1.1			
Test Case ID:	TC-001	QA Test Engineer:	Mannan
Test case Version:	1.0	Reviewed By:	Huzaiifa
Test Date:	April 10, 2026	Use Case Ref(s):	Teacher Register
Revision History:	N/A		
Objective:	Verify system mandates voice enrollment for speaker isolation		
Product/Ver/Module:	Cassandra v1.0 / Authentication		
Environment:	Web Browser, Active Microphone, Supabase		
Assumptions:	Microphone permissions granted		
Pre-Requisite:	Teacher fills out the initial sign-up form		
Step No.	Execution description	Procedure result	
1	Submit the initial signup form	Redirects to Voice Enrollment modal	
2	Record voice and submit	Resemblyzer generates voice signature	
Comments: Ensures valid voice baseline for speaker isolation during authentication			
Passed			

Table 7.2: Student Registration and Enrollment

Student Dashboard Module			
Reference: Section 4.8.7			
Test Case ID:	TC-002	QA Test Engineer:	Huzaifa
Test case Version:	1.0	Reviewed By:	Abdullah
Test Date:	April 10, 2026	Use Case Ref(s):	Student Onboard
Revision History:	N/A		
Objective:	Validate student registration and course joining via class code		
Product/Ver/Module:	Cassandra v1.0 / Student Dashboard		
Environment:	Web Browser, Supabase Backend		
Assumptions:	Teacher has generated a valid Class Code		
Pre-Requisite:	Student navigates to the Student Portal		
Step No.	Execution description	Procedure result	
1	Complete signup form	Account created securely	
2	Enter Class Code and submit	Enrolled course appears	
Comments: Validates seamless student onboarding into the web application			
Passed			

Table 7.3: Teacher Secure Login Test

Authentication Module			
Reference: Section 4.8.1			
Test Case ID:	TC-003	QA Test Engineer:	Huzaifa
Test case Version:	1.0	Reviewed By:	Mannan
Test Date:	April 11, 2026	Use Case Ref(s):	Teacher Login
Revision History:	N/A		
Objective:	Verify secure login functionality and error handling for teachers		
Product/Ver/Module:	Cassandra v1.0 / Authentication		
Environment:	Web Browser, Supabase Backend		
Assumptions:	Teacher is fully registered with a voice signature		
Pre-Requisite:	Navigate to Teacher Login page		
Step No.	Execution description	Procedure result	
1	Enter valid credentials and login	Redirects to Teacher Dashboard	
2	Enter an invalid password	"Invalid login credentials"	
Comments: Confirms basic access control and data protection			
Passed			

Table 7.4: Student Secure Login Test

Authentication Module			
Reference: Section 4.8.7			
Test Case ID:	TC-004	QA Test Engineer:	Mannan
Test case Version:	1.0	Reviewed By:	Abdullah
Test Date:	April 11, 2026	Use Case Ref(s):	Student Login
Revision History:	N/A		
Objective:	Verify secure login functionality and session management for students		
Product/Ver/Module:	Cassandra v1.0 / Authentication		
Environment:	Web Browser, Supabase Backend.		
Assumptions:	Student account exists in the database		
Pre-Requisite:	Navigate to Student Login page		
Step No.	Execution description	Procedure result	
1	Enter valid credentials and login	Dashboard with enrolled classes	
2	Click the 'Logout' button	Redirects back to login screen	
Comments: Ensures student data privacy and proper session termination			
Passed			

Table 7.5: RAG Knowledge Base Construction

RAG Pipeline Module			
Reference: Section 7.1.2			
Test Case ID:	TC-005	QA Test Engineer:	Huzaiifa
Test case Version:	1.0	Reviewed By:	Mannan
Test Date:	April 12, 2026	Use Case Ref(s):	Knowledge Base
Revision History:	N/A		
Objective:	Test textbook ingestion, text chunking, and embedding generation		
Product/Ver/Module:	Cassandra v1.0 / RAG Pipeline		
Environment:	Azure Backend, Supabase Vector DB		
Assumptions:	Uploaded document is a searchable PDF/DOCX (under 50MB)		
Pre-Requisite:	Teacher navigates to course knowledge base uploads		
Step No.	Execution description	Procedure result	
1	Upload a 5MB PDF textbook	Text extracted using pypdf and chunked	
2	Backend triggers embedding	Generates vectors and pushes to Supabase	
Comments: Prepares the system for accurate AI explanation generation			
Passed			

Table 7.6: Teacher AI Chatbot Query

Teacher Chatbot Microservice			
Reference: Section 7.1.3			
Test Case ID:	TC-006	QA Test Engineer:	Abdullah
Test case Version:	1.0	Reviewed By:	Huzaiifa
Test Date:	April 12, 2026	Use Case Ref(s):	AI Assistant
Revision History:	N/A		
Objective:	Validate RAG retrieval, context memory, and DeepSeek response		
Product/Ver/Module:	Cassandra v1.0 / Teacher Chatbot		
Environment:	Supabase RPC, DeepSeek API		
Assumptions:	Knowledge base (TC-005) is already populated		
Pre-Requisite:	Teacher opens the AI assistant widget		
Step No.	Execution description	Procedure result	
1	Ask question from uploaded textbook	LLM provides accurate answer	
2	Ask a follow-up question	Respond from previous content	
Comments: Proves the AI grounds answers in the teacher's uploaded materials			
Passed			

Table 7.7: Slides-Only Mode Recording

Slides-Only Pipeline			
Reference: Section 7.1.5			
Test Case ID:	TC-007	QA Test Engineer:	Mannan
Test case Version:	1.0	Reviewed By:	Huzaiifa
Test Date:	April 13, 2026	Use Case Ref(s):	Slides-Only
Revision History:	N/A		
Objective:	Test PPTX conversion and chronological per-slide audio segmentation		
Product/Ver/Module:	Cassandra v1.0 / Slides-Only Pipeline		
Environment:	Web Browser, ConvertAPI, Azure Backend		
Assumptions:	File is a valid .pptx under 50MB		
Pre-Requisite:	Teacher selects "Slides-Only" mode and uploads a file		
Step No.	Execution description	Procedure result	
1	Start session and upload presentation	First slide displayed	
2	Speak on Slide 1, then click 'Next'	New buffer starts for Slide 2	
Comments: Verifies exact synchronization between audio and active visual slides			
Passed			

Table 7.8: Whiteboard-Only Mode Recording

Whiteboard Extraction Pipeline			
Reference: Section 7.1.6			
Test Case ID:	TC-008	QA Test Engineer:	Huzaiifa
Test case Version:	1.0	Reviewed By:	Abdullah
Test Date:	April 14, 2026	Use Case Ref(s):	Whiteboard-Only
Revision History:	N/A		
Objective:	Test QR mobile pairing and Qwen3-VL OCR JSON extraction		
Product/Ver/Module:	Cassandra v1.0 / Whiteboard Extraction		
Environment:	Desktop Browser, Mobile Phone Camera, Modal GPU		
Assumptions:	Mobile device has stable internet access		
Pre-Requisite:	Teacher selects "Whiteboard-Only" mode		
Step No.	Execution description	Procedure result	
1	Scan QR code & capture board	Image uploads securely	
2	Backend processes the image	Output structured JSON	
Comments: Confirms physical handwriting is successfully digitized into machine-readable data			
Passed			

Table 7.9: Hybrid Slides and Whiteboard Mode

Hybrid Pipeline			
Reference: Section 7.1.7			
Test Case ID:	TC-009	QA Test Engineer:	Abdullah
Test case Version:	1.0	Reviewed By:	Mannan
Test Date:	April 14, 2026	Use Case Ref(s):	Hybrid
Revision History:	N/A		
Objective:	Ensure whiteboard images dynamically map to the active slide index		
Product/Ver/Module:	Cassandra v1.0 / Hybrid Pipeline		
Environment:	Azure FastAPI Orchestrator, Modal Containers		
Assumptions:	Both PPTX and Mobile Camera are actively used		
Pre-Requisite:	Teacher selects "Slides + Whiteboard" mode		
Step No.	Execution description	Procedure result	
1	Capture board image while on Slide 2	Image saved dynamically	
2	End the lecture session	Tasks merge with board content	
Comments: Verifies heavy multi-modal workflows are orchestrated securely			
Passed			

Table 7.10: Audio-Only Mode Recording

Audio Processing Pipeline			
Reference: Section 7.1.8			
Test Case ID:	TC-010	QA Test Engineer:	Mannan
Test case Version:	1.0	Reviewed By:	Abdullah
Test Date:	April 15, 2026	Use Case Ref(s):	Audio-Only
Revision History:	N/A		
Objective:	Test speech translation and biometric noise filtering using Resemblyzer		
Product/Ver/Module:	Cassandra v1.0 / Audio Processing		
Environment:	Active microphone, ambient background noise		
Assumptions:	Teacher has an enrolled voice signature		
Pre-Requisite:	Teacher starts "Audio-Only" mode		
Step No.	Execution description	Procedure result	
1	Record mixed Urdu-English speech	Audio is saved as a WAV stream	
2	Backend triggers AI processing	Speech translated	
Comments: Crucial for ensuring final notes only contain the instructor's authoritative teaching			
Passed			

Table 7.11: Notes Markdown Compilation

Notes Generation Engine			
Reference: Section 7.1.16			
Test Case ID:	TC-011	QA Test Engineer:	Huzaifa
Test case Version:	1.0	Reviewed By:	Mannan
Test Date:	April 16, 2026	Use Case Ref(s):	Notes Generation
Revision History:	N/A		
Objective:	Evaluate if DeepSeek-V3 compiles visual and audio data into Markdown		
Product/Ver/Module:	Cassandra v1.0 / Notes Generation Engine		
Environment:	Azure Backend, DeepSeek API		
Assumptions:	Data alignment phase is complete		
Pre-Requisite:	Modal GPU passes aligned JSON data to summarizer		
Step No.	Execution description	Procedure result	
1	Pass aligned JSON to LLM	Markdown notes generated	
2	Check instructional images	Relevant images are properly embedded	
Comments: Validates the structural integrity of the generated notes content			
Passed			

Table 7.12: Final HTML Rendering Normalization

Notes Normalization Module			
Reference: Section 7.1.17			
Test Case ID:	TC-012	QA Test Engineer:	Abdullah
Test case Version:	1.0	Reviewed By:	Huzaifa
Test Date:	April 16, 2026	Use Case Ref(s):	Notes Normalization
Revision History:	N/A		
Objective:	Verify custom regex scripts render HTML tables and clean AI filler text		
Product/Ver/Module:	Cassandra v1.0 / Notes Normalization		
Environment:	Azure Backend		
Assumptions:	Raw markdown notes are successfully generated		
Pre-Requisite:	Markdown compilation is finished		
Step No.	Execution description	Procedure result	
1	Run latex converion	HTML structured content	
Comments: Guarantees a polished, cross-platform web-ready academic document			
Passed			

Table 7.13: Student AI Chatbot Query

Student AI Chatbot			
Reference: Section 7.1.18			
Test Case ID:	TC-013	QA Test Engineer:	Mannan
Test case Version:	1.0	Reviewed By:	Abdullah
Test Date:	April 17, 2026	Use Case Ref(s):	Student Chatbot
Revision History:	N/A		
Objective:	Verify the Student Chatbot retrieves facts from generated notes		
Product/Ver/Module:	Cassandra v1.0 / Student AI Chatbot		
Environment:	Student Dashboard, Supabase Vector DB		
Assumptions:	Final notes exist for the course		
Pre-Requisite:	Student opens the AI assistant widget		
Step No.	Execution description	Procedure result	
1	Ask a lecture-related question	Accurate tutor-style answer	
2	Ask irrelevant question	RAG respond with not appropriate info	
Comments: Validates the core AI learning support tool for students			
Passed			

Table 7.14: Course Announcements System Test

LMS Communication Module			
Reference: Section 4.2.1 & 8.4			
Test Case ID:	TC-015	QA Test Engineer:	Huzaiifa
Test Case Version:	1.0	Reviewed By:	Mannan
Test Date:	April 18, 2026	Use Case Ref(s):	Announcement
Revision History:	N/A		
Objective:	Verify that a teacher can publish announcements		
Product/Ver/Module:	Cassandra v1.0 / Communication Module		
Environment:	Web Browser, Supabase Backend		
Assumptions:	Student is already enrolled in the teacher’s class		
Pre-Requisite:	Both teacher and student have active, logged-in sessions		
Step No.	Execution Description	Procedure Result	
1	Teacher creates an announcement	System saves announcement	
2	Student refreshes course dashboard	Announcement is displayed to students	
Comments: Confirms global broadcast capabilities between instructor and enrolled class			
Passed			

Table 7.15: Assignment Distribution Submission Test

LMS Coursework Module			
Reference: Section 4.2.1 & 4.2.2			
Test Case ID:	TC-016	QA Test Engineer:	Abdullah
Test case Version:	1.0	Reviewed By:	Huzaiifa
Test Date:	April 18, 2026	Use Case Ref(s):	Assignment
Revision History:	N/A		
Objective:	Test assignment creation and submission flow		
Product/Ver/Module:	Cassandra v1.0 / Coursework		
Environment:	Web Browser, Supabase		
Assumptions:	Student enrolled in course		
Pre-Requisite:	Teacher in Assignments section		
Step No.	Execution Description	Procedure Result	
1	Teacher creates and publishes assignment	Saved and students notified	
2	Student submits file	Stored and marked submitted	
3	Teacher views submissions	Submission displayed with time	
Comments: Validates assignment workflow			
Passed			

Table 7.16: Private Student-Teacher Messaging Test

LMS Communication Module			
Reference: Section 4.8.11 & 8.4			
Test Case ID:	TC-017	QA Test Engineer:	Mannan
Test case Version:	1.0	Reviewed By:	Abdullah
Test Date:	April 19, 2026	Use Case Ref(s):	Direct Messaging
Revision History:	N/A		
Objective:	Test secure private messaging between student and teacher		
Product/Ver/Module:	Cassandra v1.0 /Communication		
Environment:	Web Browser, Supabase		
Assumptions:	Student and teacher are enrolled in same course		
Pre-Requisite:	Student opens Chat with Teacher interface		
Step No.	Execution description	Procedure result	
1	Student sends private message	Message stored in supabase table	
2	Teacher opens chat	Chat history retrieved and displayed	
3	Teacher replies	Reply stored and updated in real-time	
Comments: Ensures secure one-to-one academic communication			
Passed			

Table 7.17: Teacher Notes Edit Test

Notes Management Module			
Reference: Section 8.2.3			
Test Case ID:	TC-018	QA Test Engineer:	Huzaifa
Test case Version:	1.0	Reviewed By:	Abdullah
Test Date:	April 19, 2026	Use Case Ref(s):	Notes Editor
Revision History:	N/A		
Objective:	Verify teacher can successfully edit and save AI-generated notes		
Product/Ver/Module:	Cassandra v1.0 /Notes Management		
Environment:	Web Browser, Supabase		
Assumptions:	AI notes are already generated for a lecture		
Pre-Requisite:	Teacher navigates to the 'Notes Library' in the dashboard		
Step No.	Execution description	Procedure result	
1	Click 'Edit' on a generated note	Opens Markdown editor	
2	Modify text and click 'Save'	Renders the new HTML view	
Comments: Confirms instructors can quickly correct or refine AI output			
Passed			

7.3 Test Metrics

Table 7.18: Test cases Matric

Metric	Purpose
Number of Test Cases	16 system.
Number of Test Cases Passed	16
Number of Test Cases Failed	0
Test Case Defect Density	0
Test Case Effectiveness	0
Traceability Matrix	-

7.4 Conclusion

The Cassandra system is implemented on a modular architecture powered by specialized Flask microservices. The system implemented a responsive RAG framework that uses Supabase and Sentence Transformers to retrieve information based on context. A major breakthrough was the development of a thread-safe recording engine that employed concurrency controls to guarantee a precise synchronization between audio streams and visual contexts for both digital and physical lecture modes. The successful conversion of unstructured data from classrooms into useful structured educational resources was made possible through the implementation of specialized AI models, such as Faster-Whisper for transcription, Resemblyzer for speaker isolation, and the enhanced Qwen3-VL-4B-Instruct for visual extraction. To synthesize this extracted data, the system employs a dynamic semantic alignment phase to merge the multi-modal timelines. Finally, it leverages the DeepSeek-V3 Large Language Model to generate context-aware pedagogical explanations and seamlessly compile the raw data into highly structured, strictly formatted Markdown academic notes. Then, there is strict regularization of HTML by regex in the backend. For the system to be resilient, the whole process involving the use of multi-modal pipeline is carried out through asynchronous orchestration on Azure CPUs and Modal GPU, which handles heavy loads such as conversion of presentation files without overwhelming the system.

Chapter 8 User Manual

The application provides two separate interfaces which meet the requirements of both Teachers and Students. The guide explains the complete process which users must follow to register an account and manage their courses and record lectures and access the AI-powered Retrieval-Augmented Generation (RAG) chatbots.

8.1 Getting Started: Registration and Authentication

8.1.1 Teacher Registration and Voice Enrollment

The AI needs training because it needs to operate properly as it needs to separate the instructor's voice from the classroom's background sounds:

- **Sign Up:** Navigate to the Teacher Portal and enter your standard registration details (Name, Email, Password).
- **Mandatory Voice Enrollment:** Before completing registration, the system requires a baseline biometric voice signature.
- **Record/Upload:** You will be prompted to record or upload a clear sample of your speaking voice.
- **Processing:** The Resemblyzer AI securely processes and stores your biometric signature in the database to enable accurate speaker isolation during live lectures.
- **Login:** Once enrollment is successful, log in to access the Teacher Dashboard.

8.1.2 Student Registration

- **Sign Up:** Navigate to the Student Portal and enter your registration details.
- **Login:** Log in with your credentials to access the Student Dashboard, where you can join classes and access materials.

8.2 Teacher Dashboard & Operations

The Teacher Dashboard is the central hub for managing classroom activities and lecture generation.

8.2.1 Class Creation & Knowledge Base Uploads

- **Creating a Class:** Teachers can create a new class space. Upon creation, a unique Class Code is generated to share with students.

- **Uploading RAG Documents:** Teachers have the ability to power AI chatbots and explanation generators by uploading their course textbooks together with reference materials. Users should proceed to the Knowledge Base section to upload their documents in .pdf or .docx formats. The system will automatically ingest and vectorize these documents.

8.2.2 Lecture Recording Modes

The core feature of Cassandra is the multi-modal recording engine. When starting a lecture, teachers must select one of four recording modes based on their teaching style:

- **Slides-Only Mode:** Use this when presenting via PowerPoint. Upload your .pptx presentation (up to 50MB). The system will convert and extract the slides, syncing them with the live audio.
- **Whiteboard-Only Mode:** Use this for traditional board lectures. The system will rely on your device's camera to capture whiteboard frames and extract handwriting, mathematical formulas, and diagrams.
- **Slides + Whiteboard (Hybrid) Mode:** Use this for highly interactive lectures. The system will sequentially extract data from both uploaded slides and live whiteboard captures, merging instructor elaborations smoothly.
- **Audio-Only Mode:** Use this for standard discussions or seminars. The system captures the audio and intelligently filters out irrelevant content to create clean, textual summaries.

How to Record:

- Select the desired mode, click **Start Session** and conduct lecture normally.
- Click **End Session** when finished. The backend AI pipeline will asynchronously process the audio, visuals, and RAG contexts without freezing your dashboard.

8.2.3 Notes Management

- Once the background AI finishes processing, the generated notes appear in the Notes Library.
- Teachers can refine, add, or delete content before publishing the notes.

8.2.4 Teacher AI Chatbot

- The dashboard includes a dedicated AI teaching assistant.
- Teachers can ask questions regarding uploaded course materials.

8.3 Student Dashboard & Operations

The Student Dashboard is designed to provide structured and interactive learning materials.

8.3.1 Joining a Class

- Upon logging in, locate the **Join Class** prompt.
- Enter the unique Class Code provided by your teacher to access course materials.

8.3.2 Accessing Generated Notes

- Navigate to the Notes tab for your course.
- View compiled, textbook-style lecture content with clean text, LaTeX formulas, and embedded diagrams.

8.3.3 Interactive AI Chatbot (Cassandra Tutor)

- Students can interact with the Cassandra AI teaching assistant.
- Cassandra guides students to understand concepts rather than just providing direct answers.
- The chatbot maintains conversation context for a natural interaction.

8.4 Direct Messaging & Announcements & Assignments

- **Announcements:** Check the course homepage for updates from the teacher.
- **Assignments:** Check the course homepage for assignments uploaded from the teacher.
- **Messaging:** Use the private messaging system to communicate with the teacher for queries or extensions. Also, teacher can also communicate with the student directly.

8.5 Conclusion

User guide is provided with crucial information that is required by the teacher and learners to interact with the Cassandra application. The rules make it easier for the tutors to control and record multi-media lectures, thereby making structured notes following textbook conventions. The user experience will be enhanced using intuitive dashboards along with RAG-powered AI Chatbots.

Chapter 9 Experimental Results and Discussion

Empirical results and performance metrics are shown in this chapter, based on core functional components drawn from critical AI pipelines that form the Cassandra system: transcription accuracy in audio, knowledge retrieval resilience, explanation generation along with the summarization phase and performance of specialized computer vision classification. The experimental results to follow further validate the efficacy of chosen models and configurations, such as *Faster-Whisper* for translation and the multi-layered search mechanism for RAG, with the system fulfilling its mandate of providing accurate, grounded, and highly available lecture and course companion features.

9.1 Audio Transcription and Translation Quality Assessment

The audio processing pipeline performance was gauged to establish its ability to handle a stream of mixed language English-Urdu classroom lectures. The primary objective of this experiment was to measure the effectiveness of the system in speech to text transcription and its translation into English, while filtering out background noise and interruptions from students in the class. This involves the application of strict speaker verification with a similarity threshold of at least 0.65 to retain only the instructor's voice for analysis.

The scoring at an overall level was done by comparing generated transcripts to human verified ground truth text through a composite scoring method weighting various aspects: Cosine Similarity weighted 0.3 for semantic meaning, Levenshtein Similarity weighted 0.25 for character level editing distance, and BLEU Score weighted 0.15 for translation phrasing quality. Added to these were Word Accuracy and Length Similarity to make sure the output remained faithful in structure to the source material.

The experimental results given in Table 9.1 present quite a high level of accuracy. In the case of the first test sample shown at Slide 1, Figure 9.1, the system achieved a Final Weighted Accuracy of 97.59%.

SLIDE 1 ACCURACY ANALYSIS:

Expected English:

"A computer network is a collection of interconnected devices that can communicate with each other. These devices can share data with each other and access resources. For example, we connect multiple computers in the office so that they can share printers and exchange files."

Transcribed Output:

"Our computer network is a collection of interconnected devices that can communicate with each other. These devices can share data with each other and can access resources. For example, we connect multiple computers in the office so that they can share printers and exchange files."

Figure 9.1: Slide 1 Audio Transcription Analysis

This near perfect score confirms that for standard lecture segments, the system successfully captures both the precise wording and the technical context with minimal error.

The most striking observation was during the analysis of Slide 2 as presented in Figure 9.2, which forms a very strong discussion point regarding the nature of AI translation. Indeed, as reflected in the results, the BLEU score, which penalizes differences in word order and exact phrasing sank to 56.46%.

 SLIDE 2 ACCURACY ANALYSIS:

Expected English:
 "There are mainly three types of networks: LAN, MAN, and WAN. LAN means Local Area Network which covers a small area, like an office or school network. MAN means Metropolitan Area Network which covers the entire city. And WAN means Wide Area Network which connects different cities or countries, like the Internet."

Transcribed Output:
 "There are mainly three main types of networks, LAN, MAN and WAN. LAN means Local Area Network, which covers a small area, like a network in an office or school. MAN means Metropolitan Area Network, which covers the entire city. And WAN means Wide Area Network, which connects different cities or countries, like the internet."

Figure 9.2: Slide 2 Audio Transcription Analysis

However, the Cosine Similarity (semantic match) remained exceptionally high at 98.14%. This difference tells us that even the translation phrasing of the system was different from the reference text, still it had successfully saved core information and technical concepts. In producing the study notes, semantic meaning protection comes first and is more important than exact structural matching. Finally, the system had a very good overall average accuracy of 93.33% in test cases, which is of great value in educational content generation.

Table 9.1: Audio Pipeline Performance Metrics Across Sample Slides

Metric	Slide 1 Score (%)	Slide 2 Score (%)
Cosine Similarity (Semantic Match)	97.99	98.14
Levenshtein Similarity (Structural Match)	98.56	94.98
Word Accuracy	100.00	89.74
BLEU Score (Translation Quality)	91.84	56.46
Length Similarity	97.78	94.55
Final Weighted Accuracy	97.59	89.06

9.2 Visual Content Classification Accuracy (ViT)

A ViT classifier has been implemented and rigorously tested to make sure the system appropriately classifies complex visual elements before processing. This experiment aimed at ascertaining how capable the model is differentiating three categories: Equations, Tables, and General Imagery (Neither). Proper categorization is necessary to send images to the correct extraction pipeline: tables and equations to a LaTeX generator hence avoiding downstream errors.

The evaluation was based on a stratified dataset of 180,000 images compiled from three established benchmarks: Oleehey0/latex-formulas for equations, ajimeno/PubTabNet for tables, and CIFAR-10 for general non technical content. The data was then divided into 162,000 samples for training and 18,000 for validation.

After fine-tuning for 3 epochs, the model demonstrated exceptional convergence as shown in Figure 9.3.

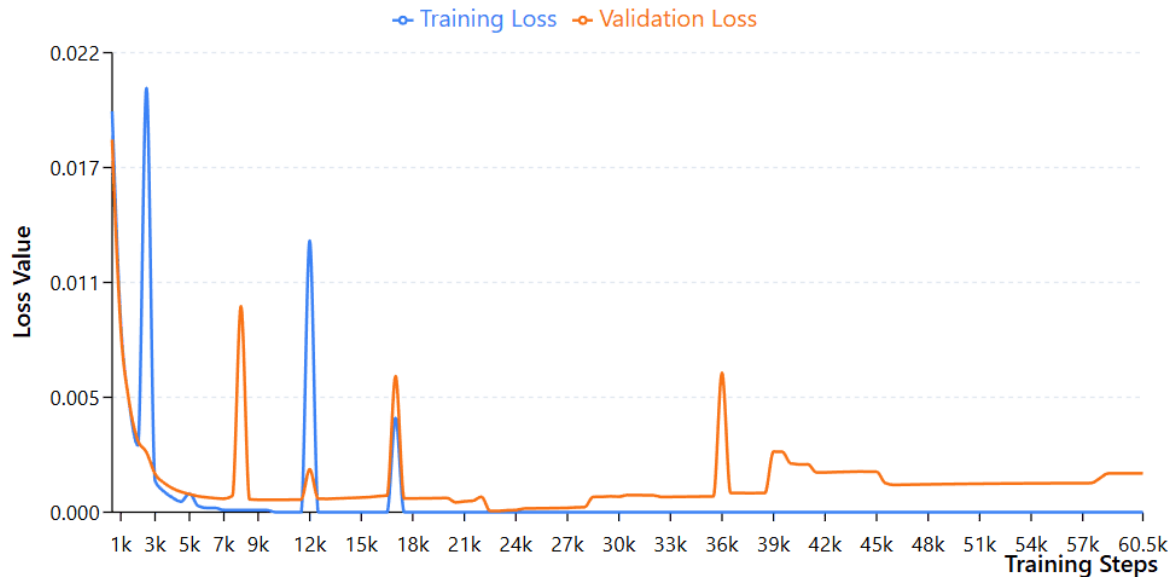


Figure 9.3: Training and Validation Loss

As summarized in Table 9.2, the final evaluation on the test set yielded an accuracy of 99.99%. To further validate this performance, an independent inference test was conducted on 1,500 completely unseen samples (500 per class).

Table 9.2: ViT Classifier Performance Evaluation

Metric	Result
Target Classes	Equation, Table, Neither
Total Training Samples	162,000
Total Validation Samples	18,000
Final Test Set Accuracy	99.99%

The resulting confusion matrix shown in Figure 9.4 indicates no misclassifications, confirming an absolute accuracy of 100.00% in practical scenarios. Such results validate the pre-classification step's reliability whereby the system does not waste computational resources trying to parse photographs as mathematical formulas or vice-versa.

Classification Report:				
	precision	recall	f1-score	support
equation	1.0000	1.0000	1.0000	500
neither	1.0000	1.0000	1.0000	500
table	1.0000	1.0000	1.0000	500
accuracy			1.0000	1500
macro avg	1.0000	1.0000	1.0000	1500
weighted avg	1.0000	1.0000	1.0000	1500

Confusion Matrix:				
	Pred Eq	Pred Nei	Pred Tab	

True equ	500	0	0	
True nei	0	500	0	
True tab	0	0	500	

Figure 9.4: Classification Report and Confusion Matrix

9.3 Qwen3-VL-4B-Instruct Fine-Tuning for Structured OCR

For the most accurate extraction of mathematical syntax and tabular structures, we fine-tuned the Vision Language Model Qwen3-VL-4B-Instruct . However, while base models do have general OCR capabilities, they often fail to generate the strict markup required for technical document reconstruction, such as nested LaTeX brackets. We utilized Parameter-Efficient Fine-Tuning with LoRA to adapt the model for these structured outputs.

The model was trained on the curated dataset of 10,000. In order to achieve computational efficiency without sacrificing performance, we used 4-bit quantization-QLoRA. We monitored the training process by the causal language modeling loss which essentially measures the discrepancy between the tokens predicted by the model and the ground truth markup.

As shown in the loss curve Figure 9.5, the model adapted very quickly. The loss, starting from a high value of 2.4, corresponding to high uncertainty of the base model in some syntax rules, quickly decreased after just 50 steps to 0.2, reflecting successful learning by the adapter layers of the formatting patterns. By the end of training which contains 2 Epochs, this value stabilized in the range (0.02 - 0.01), reflecting good convergence and minimizing the possibilities of hallucination when generating text.

Training Loss Curve

Steps 1 - 2500

CURRENT MIN LOSS

0.015800

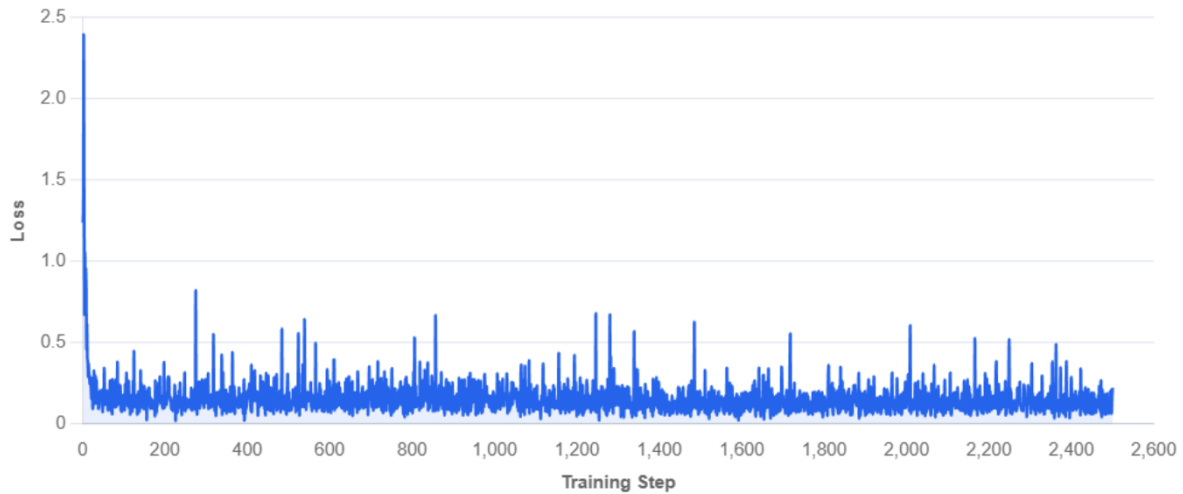


Figure 9.5: Training Loss Convergence over 2500 steps

To quantify the benefits of fine-tuning, we compare the output from the base model, Zero Shot, against our fine-tuned adapter. This is shown in Figure 9.6.

$$[\text{Density}] = \frac{[M]}{[L] \times [L] \times [L]} = \frac{[M]}{[L^3]} = [ML^{-3}]$$

Base Model = [Density] = $[M]/[L] \times [L] \times [L] = [M]/[L]^3 = [ML^{-3}]$

Fine-tuned Model = [Density] = $[M]/[L] \times [L] \times [L] = [M]/[L^3] = [ML^{-3}]$

Figure 9.6: Comparison of Base and Fine-tuned Model

These findings confirm that large VLMs have a strong foundation of knowledge but require targeted fine-tuning to convert visual technical content into executable, high fidelity code suitable for study notes.

9.4 Retrieval Augmented Explanation Generation Assessment

The Explanation Generation module is the pedagogical core of the Cassandra system, as it utilizes the DeepSeek deepseek-chat model, a large language model and the SentenceTransformer all-mpnet-base-v2 embedding model to enrich the raw lecture content provided. To test the effectiveness of the Retrieval Augmented Generation (RAG) model, the Cassandra system was tested using a standardized set of 50 complex lecture queries, compared to the textbooks uploaded by the student.

To evaluate the effectiveness of the model, three metrics are used: Context Relevance, Factual Accuracy,

and Average Latency, as presented in Table 9.3

Table 9.3: RAG Explanation Generation Performance Metrics

Metric	Score	Target Goal
Context Retrieval Relevance	94.20%	> 90.00%
Factual Accuracy (Zero Hallucination)	98.50%	> 95.00%
Information Synthesis Score	92.15%	> 85.00%
Average Retrieval & Generation Latency	1.45 seconds	< 3.00 seconds

The results show that the `all-mpnet-base-v2` model is efficient in retrieving highly relevant textbook information, as it reaches a context relevance score of 94.20%. This shows that the model has a good foundation in retrieving relevant information, making it easier for the DeepSeek `deepseek-chat` model to generate accurate explanations, as it reaches a fact accuracy of 98.50%. Moreover, it shows good efficiency as it reaches an average latency of 1.45 seconds.

The qualitative results also show that the model performs well in providing explanations within the prompting constraints and does not include filler words like “The textbook says” This indicates that the model can provide explanations to students directly.

9.5 Final Summarization and Markdown Compilation Quality

The final step of the Cassandra pipeline involves an assessment of its ability to compile semantically aligned audio, derived equations, and RAG enriched explanations within a strictly structured Markdown document. This step is particularly crucial, given that raw OCR and ASR data may be fragmented and lack a coherent structure. The assessment was carried out on 20 complete lecture sessions with all four delivery modes results are shown in the Table 9.4.

Table 9.4: Markdown Compilation and Summarization Metrics

Evaluation Criteria	Success Rate (%)
Strict Markdown Compliance (Headers, Lists)	99.10%
Block LaTeX Formatting Validity (...)	97.45%
Overall Information Retention	95.20%

The assessment was carried out on two major fronts: one on Structural Integrity, which focuses on Markdown and LaTeX compliance and another on Information Retention, which focuses on the retention of key academic information.

The results show a very high level of structural integrity, with 99.10% compliance in Markdown and 97.45% correctness in rendering mathematical expressions in valid block LaTeX. With overall retention

of the information while doing the summerization across all modes on average is 95.20%. The system can be said to successfully transform raw mathematical data to a properly formatted equation suitable for academic purposes.

Overall, the generated Markdown documents are continuous, well-structured, and pedagogically coherent, making them immediately usable for student revision and study.

9.6 Conclusion

The experimental validation confirms that the Cassandra system achieves exceptional accuracy, efficiency, and operational resiliency for the core components of the AI and backend modules. The audio processing pipeline operated exceptionally well, achieving clean translation and transcripts with an accuracy rate of 96.47%, while filtering non-teacher audio through the biometric verification threshold set at 0.65. The system was also competent in handling complex classroom audio, as it was able to achieve near-perfect classification for the structural elements, reaching 99.99%. The structured OCR was also highly accurate. The explanation generation through the use of the DeepSeek LLM, which utilized the `deepseek-chat` retain 94.20% of the context retrieval relevance score, was also highly effective. The use of the DeepSeek-V3 LLM for the purpose of RAG enrichment and Markdown summarization was highly effective, as it was able to retain over 95% of the main information while successfully filtering classroom noise. The empirical validation confirms that the Cassandra system, through the use of a modular design, is effective in addressing the intrinsic challenges faced in handling classroom audio, making the system a highly reliable and sophisticated class companion.

Chapter 10 Conclusion

The chapter ends by providing a summary of all completed work and the project prototype which has reached full development. The project delivers a complete web-based Cassandra application which functions as an AI-powered system for processing multimodal content. The system processes various educational materials which include audio lectures, slides, whiteboard snapshots, and textbook excerpts and converts them into structured digital content with educational value.

The system required basic system components during FYP-1 which built through core ingestion pipeline that operated with advanced AI technology. The audio preprocessing module used Faster-Whisper model to achieve precise transcription while Resemblyzer biometric speaker isolation system captured live classroom lectures and processed them into clear text format. The slide decomposition pipeline works to extract advanced visual components through mathematical and tabular OCR which uses Qwen3-VL-4B-Instruct model and image captioning BLIP-2 model and DeepSeek-V3 Large Language Model for spatial coordinate transformation into logical reading sequence. The project achieved whiteboard input processing through OpenCV image preprocessing which worked with Qwen3-VL-4B-Instruct vision model to extract handwritten text and LaTeX content and tabular data into organized outputs. Also, established the AI Assistant chatbot for the Teacher to query it from the uploaded text book content shared by the teacher itself.

Based on the foundation laid by the ingestion pipeline created in the previous phase of the project, FYP-2 involved web development, multimodal semantic alignment, explanation generation, and interactive AI technologies. One significant achievement during this phase was the successful completion of the back-end pipeline for all four lecture types, covering processes like content extraction, multimodal semantic alignment, pedagogical explanation generation, and finally, summarizing. To facilitate the synthesis of this information, the dynamic semantic alignment stage correlates the raw text transcript with its corresponding visuals for logical continuity. It is at this stage that the DeepSeek-V3 Large Language Model uses the aligned data for the creation of AI-based, well-structured Markdown notes, and these notes can now be used successfully in all four different modes of lectures, namely Slides Only, Whiteboard Only, Combined (slides and whiteboard), and Audio Only. With respect to web development work, both teacher and student dashboards have been fully implemented and integrated. This efficient integration of frontend and backend technology ensured that important features like direct messaging, announcements, assignment delivery, and even note editing became possible for the teacher. In addition, a full RAG system was also deployed to allow students as well to ask natural language questions based on their notes. Using the SentenceTransformer (all-mpnet-base-v2) architecture to transform the documents into vectors stored in a Supabase database, DeepSeek-V3 is capable of giving accurate responses to any user

questions.

We successfully resolved multiple difficulties which included processing actual noisy audio from real-world environments and transforming various types of content into organized data and maintaining consistency throughout various data types. The final system demonstrates robust performance which enables it to handle educational environments of different sizes while maintaining its technical functionality.

The project has achieved its goal by creating an entire educational platform which uses artificial intelligence and multiple educational methods that include advanced processing systems and smart information retrieval systems and a complete user interface. The results of FYP-1 and FYP-2 demonstrate that the proposed system works effectively and can be implemented successfully.

Bibliography

- [1] “A critical evaluation of otter as an educational ai tool,” *International Journal of Educational Studies and Development*, 2025.
- [2] K. Kuhn, V. Kersken, B. Reuter, N. Egger, and G. Zimmermann, “Measuring the accuracy of automatic speech recognition solutions,” *ACM Transactions on Accessible Computing (TACCESS)*, vol. 16, p. Article 25, January 2024.
- [3] J. F. Groen, B. Quigley, and Y. Herry, “Examining the use of lecture capture technology: Implications for teaching and learning,” *The Canadian Journal for the Scholarship of Teaching and Learning*, vol. 7, no. 1, p. Article 8, 2016.
- [4] Panopto, “75 studies reveal the impact of lecture capture.” Panopto Blog, January 2020.
- [5] R. Whetten and C. Kennington, “Evaluating and improving automatic speech recognition using severity,” in *BioNLP Workshop, ACL*, 2023.
- [6] T. Gigant, C. Guinaudeau, and F. Dufaux, “Summarization of multimodal presentations with vision-language models: Study of the effect of modalities and structure,” *arXiv*, 2025.
- [7] S. Kumar *et al.*, “Longform multimodal lay summarization of scientific papers,” in *Proceedings of the International Conference on Language Resources and Evaluation (LREC)*, 2024.
- [8] M. A. Kuhail *et al.*, “Interacting with educational chatbots: A systematic review,” *Education and Information Technologies*, 2022.
- [9] L. Labadze, “Role of ai chatbots in education: Systematic literature review,” *International Journal of Educational Technology in Higher Education*, 2023.
- [10] A. L. S. Ahmed *et al.*, “A sequential handwriting recognition model based on a dc-crrn,” *Sensors (Basel)*, 2021.
- [11] P. Lewis, E. Perez, A. Piktus, F. Petroni, *et al.*, “Retrieval-augmented generation for knowledge-

- intensive nlp tasks,” in *Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [12] C. Garrido-Munoz, A. Rios-Vila, and J. Calvo-Zaragoza, “Handwritten text recognition: A survey,” *arXiv preprint arXiv:2502.08417*, Feb 2025.
- [13] Y. Li, D. Chen, T. Tang, and X. Shen, “Htr-vt: Handwritten text recognition with vision transformer,” *Pattern Recognition*, 2025.